Adaptive Design of Embodied Robotic Systems for Real-Time Self-Modeling Toward Swarm Coordination

Master's Thesis

Muhammad Zeeshan Asghar
Master's Program in Data Science
HSE University

2026

Abstract

Embodied robotic systems deployed in unstructured environments inevitably experience embodiment drift—the progressive divergence between a robot's internal kinematic model and its physical hardware caused by mechanical wear, calibration slip, thermal deformation, and unmodeled impacts [1, 2]. When a robot's internal model no longer matches its physical embodiment, traditional Jacobian-based inverse kinematics controllers fail catastrophically: end-effectors diverge from targets, oscillations destabilize the system, and contact-rich tasks generate unsafe forces [4, 10]. Maintaining true autonomy requires self-modeling—the ability to infer one's own morphology directly from sensorimotor experience.

Contemporary visual self-modeling architectures based on Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) achieve high morphological expressiveness but suffer from a critical latency paradox [11, 15]. A diagnostic self-model is only valuable if it can provide state estimates fast enough to influence the feedback controller. Modern robotic control loops commonly operate at 1 kHz, permitting a maximum latency of 1.0 millisecond per inference [90]. Existing neural rendering self-models operate at 5–37 FPS—latencies of 27–192 milliseconds—rendering them unusable for real-time control integration. This represents a fundamental architectural mismatch: volumetric reconstruction is computationally incompatible with sub-millisecond control deadlines.

This thesis resolves this bottleneck by introducing NeuroKin, a lightweight fully convolutional direct sensorimotor decoder that completely bypasses explicit 3D volumetric reconstruction. NeuroKin maps joint angles directly to visual silhouettes, achieving 21.88 dB PSNR at 7,400 FPS (0.135 ms latency)—a 1,418× speedup over implicit NeRF baselines with superior fidelity. Training completes in 33.5 seconds on a Tesla T4 GPU. An auxiliary multi-task variant, ResNeuroKin-D, recovers geometric depth while maintaining 2,500 FPS (0.400 ms latency). Integration of NeuroKin's spatial estimates into a Jacobian-based feedback loop enables dynamic error recovery after severe fault injection, demonstrating that sub-millisecond visual self-modeling can actively stabilize a damaged manipulator.

Building on this individual diagnostic capability, we scale to collective fault tolerance via temporal stigmergy—a mechanism of indirect coordination wherein agents leave environmental traces that stimulate adaptive responses in subsequent agents. In a 100-stage sequential swarm under progressive fault injection, the standard inverse-kinematics baseline collapses from 100% pre-fault success to 13% post-fault success. Conversely, NeuroKin-enabled agents read and write a shared failure ledger, autonomously trigger localized recovery

modes, and preserve 88% systemic success.

This thesis provides four primary contributions: (1) formal benchmarking of the latency paradox in neural rendering self-models, (2) the NeuroKin direct decoding architecture achieving sub-millisecond inference, (3) visual closed-loop damage adaptation, and (4) empirical validation that fast individual self-modeling enables decentralized stigmergic coordination—the computational prerequisite for physical swarm deployment.

Acknowledgements

This thesis represents the culmination of a deeply fulfilling academic journey, and it would not have been possible without the profound support of my mentors, colleagues, and family.

First and foremost, I gratefully acknowledge the guidance, patience, and critical feedback of my academic supervisor and the thesis defence committee at the Higher School of Economics (HSE) Master's Program in Data Science. Your rigorous standards and technical insights have fundamentally shaped my approach to research. Reflecting on my academic development, the foundational engineering principles I acquired during my Bachelor of Engineering in Mechatronics, Robotics, and Automation Engineering with a minor in Computing from the National University of Sciences and Technology (NUST) have been absolute prerequisites in grounding my deep learning models in real-world physical and kinematic boundaries.

I also extend my sincere appreciation to my co-founders and colleagues at LangMeet AI. Serving as Technical Lead while managing real-time speech translation infrastructure concurrently with my thesis workload has been a formidable challenge, but it provided an invaluable sandbox for investigating computational latency profiles, hardware-level optimization, and production bottlenecks.

On a deeply personal level, I owe an immense debt of gratitude to my parents. Your shared childhood observations, continuous encouragement, and steadfast support have been my ultimate cognitive anchor. To my sister, whose empathy, kindness, and unyielding dedication to rescuing and adopting stray cats inspire me daily to notice harmony in chaotic systems. To my brother, whose background studying classical violin provided a beautiful, resonant acoustic environment that acted as the backdrop to countless late-night debugging and writing sessions. I am also deeply thankful to my aunt for her kind nature, warmth, and steady reassurance throughout my master's timeline.

Finally, I must acknowledge the quiet but essential companionship of my cat, Alec, who spent countless hours keeping watch beside my workstations while these neural networks were designed, trained, and evaluated.

As this Master's exploration draws to a close, I look forward to expanding this foundation. The synthetic experiments and decentralized control frameworks validated herein represent an indispensable stepping stone. Translating these direct sensorimotor networks from controlled physical constraints into adaptive physical deployments on multi-robot platforms will form the explicit technical objective of my upcoming Ph.D. program.

The author confirms that AI tools were used only to assist with grammar, vocabulary, and to enhance narrative flow; all technical content, experimental design, analyses, and conclusions remain the author's original work. Supplementary materials and the implementation of the NeuroKin project are available at: https://github.com/YoloPopo/neurokin.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 The Challenge of Embodiment Drift

Autonomous robots deployed in remote, unstructured environments—planetary rovers, disaster-response machines, deep-sea manipulators, and industrial inspection drones—must operate for extended periods without human intervention. These systems cannot rely on frequent manual recalibration, hardware replacement, or human-in-the-loop supervision. They must be self-sufficient, adaptive, and resilient in the face of inevitable physical degradation [1, 2].

A fundamental requirement for such autonomy is maintaining an accurate internal representation of one's own morphology, kinematics, and dynamics. Traditional robotic systems rely on predefined kinematic models—typically specified in Unified Robot Description Format (URDF) files—that encode link lengths, joint limits, inertial properties, and collision geometries. These models enable classical control algorithms, such as Jacobian-based inverse kinematics and operational space control, to map intended high-level actions into precise joint torques or positional trajectories [90]. Under ideal conditions, with a perfectly calibrated robot operating in a controlled environment, these methods achieve millimeter-level precision.

However, the assumption that a physical robot remains mathematically identical to its idealized digital model is fundamentally flawed under continuous operational deployment. Physical embodiment subjects robots to the realities of thermodynamics, material fatigue, and mechanical friction. Over extended operational cycles—spanning hours, days, or months of continuous operation—the strict mathematical guarantees of forward and inverse kinematics inevitably decouple from physical reality.

This persistent, accumulating divergence between the internal computational representation of the robot and its actual physical state is formally defined in this thesis as *embodiment drift* [1, 4]. The term captures the essential phenomenon: the robot's sense of its own body drifts away from truth as hardware degrades.

1.1.1 Manifestations of Embodiment Drift

Embodiment drift manifests across a broad spectrum of mechanical and sensory degradations. At the micro-scale, geared transmissions develop unmodeled backlash as teeth wear; actuator drive belts lose optimal tension through creep and fatigue; thermal expansion induced by sustained high-torque operations alters the effective physical length of metallic linkages by micrometers—enough to matter in precision tasks. At the sensory layer, proprioceptive joint encoders may imperceptibly slip from their calibrated zero-points due to vibration or electrical noise; exteroceptive sensors, such as chassis-mounted RGB cameras or depth sensors, experience extrinsic calibration shifts from ambient thermal cycling or minor physical impacts [14].

In more severe, sudden operational scenarios, robots may suffer macroscopic structural damage: an unexpected collision may plastically deform a physical link by several degrees; an actuator may seize entirely due to electrical failure or debris ingress; an asymmetrical payload shift—such as a grasped object suddenly changing weight distribution—may fundamentally alter the system's center of mass and dynamic response [4].

1.1.2 Consequences for Control

When significant embodiment drift occurs, the consequences for closed-loop feedback control are catastrophic. A controller operating under the assumption of a nominal kinematic tree will compute pseudo-inverse Jacobian updates that are mathematically sound relative to the model but physically erroneous relative to reality [90]. The system issues corrective motor commands that inadvertently drive the end-effector away from the target, induce highly unstable resonant oscillations, or, in contact-rich manipulation tasks, exert unsafe, unbounded forces upon the environment. In legged locomotion, embodiment drift leads to gait asymmetry, reduced stability margins, and eventual falls. In humanoid robots operating near humans, the safety implications are even more severe [4].

To maintain true autonomy and operational safety in the presence of embodiment drift, a robotic system must possess the capacity for *self-modeling*—the ability to autonomously infer, update, and deploy an internal representation of its own morphology derived strictly from its own sensorimotor experience, entirely bypassing the need for external, human-in-the-loop manual recalibration [10, 4].

Figure 1.1 summarizes the closed-loop self-modeling pipeline that motivates the rest of this thesis.

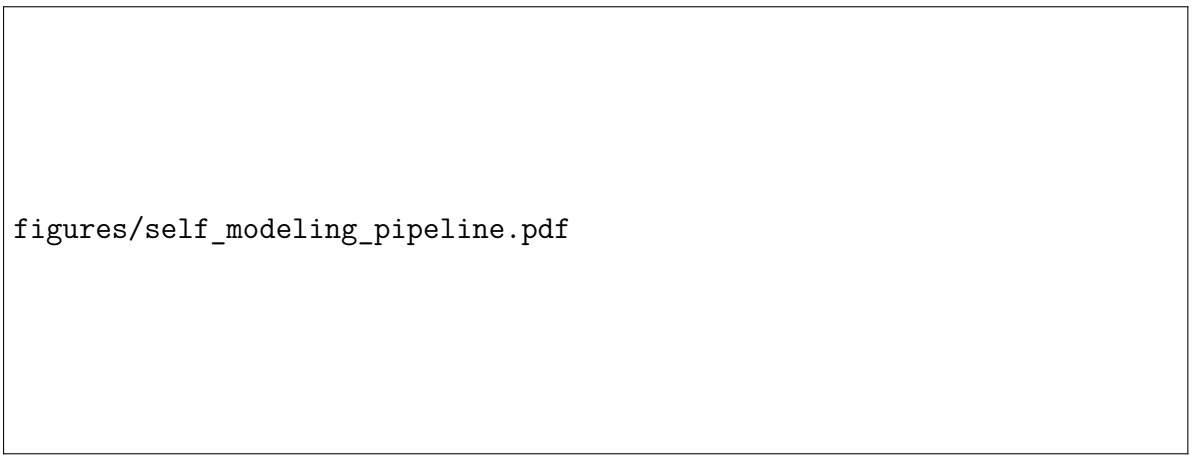


Figure 1.1: Block diagram tracing a five-stage operational architecture for autonomous robots, sequencing raw sensory input, feature state encoding, deep self-model kinematics processing, tracking control planning, and joint execution loops.

1.2 The Evolution of Self-Modeling Paradigms

The formidable challenge of autonomous self-modeling has driven several distinct research paradigms over the past two decades. Early approaches relied heavily on symbolic inference and active physical exploration, utilizing continuous cycles of hypothesis generation to deduce morphological changes [1]. Subsequent paradigms shifted toward behavioral adaptation, bypassing damage diagnosis entirely to search pre-computed behavioral repertoires for compensatory actions [2]. The advent of deep learning introduced task-agnostic learned dynamics forward models [4, 5], while parallel threads explored proprioceptive morphology identification to restore embodiment using internal signals alone [6, 8, 7, 64, 9].

Most recently, the field has entered the visual era, leveraging neural rendering to learn self-models directly from camera observations [10, 11, 15]. Architectures based on implicit coordinate networks like the Free-Form Kinematic Self-Model (FFKSM) and explicit primitives like Kinematic 3D Gaussian Splatting (K-3DGS) achieve high morphological expressiveness. However, as this thesis establishes, the immense representational power of these neural rendering architectures introduces a severe computational bottleneck. A detailed chronological analysis of these paradigms and their respective mathematical limitations is provided in Chapter 2.

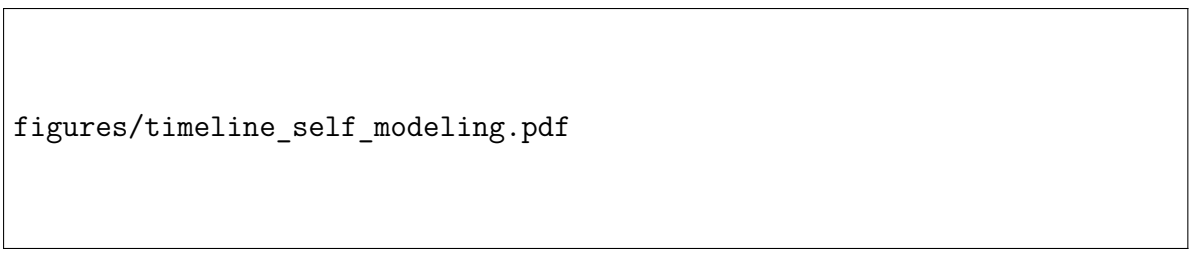


Figure 1.2: Horizontal developmental timeline marking benchmark historical milestones in robotic self-modeling research from early 2006 symbolic explorations to contemporary real-time neural rendering architectures.

1.3 The Latency Paradox in Neural Rendering

A diagnostic self-model is operationally valuable only if it can provide state estimates fast enough to influence the stabilizing behavior of the feedback controller [11, 15]. If the model requires hundreds of milliseconds to generate a prediction, the information arrives too late—the robot has already executed several control cycles based on stale assumptions.

1.3.1 Temporal Constraints of Robotic Control

Modern robotic control systems operate at tightly constrained frequencies. Force control, impedance control, and dynamic balancing typically run at 1 kHz, requiring the entire perception-planning-actuation pipeline to execute within 1.0 millisecond [90]. Within the 1.0 ms window, the entire software stack must read physical sensor buffers, execute state estimation, compute target Cartesian error vectors, resolve complex inverse kinematic equations, enforce physical joint limits, and issue subsequent motor torque commands. Consequently, any self-modeling diagnostic component integrated into this inner control loop possesses an absolute maximum inference budget of a fraction of a millisecond.

1.3.2 The Computational Bottleneck

Current state-of-the-art neural rendering architectures fundamentally violate this latency budget. Implicit NeRF-based models achieve frame rates of approximately 5 FPS, corresponding to 191.6 ms latency [11]. Explicit 3DGS-based architectures reach approximately 37 FPS, corresponding to 27.0 ms latency [15]. Both exceed the 1.0 ms control budget by factors of 27 to 190—rendering them entirely unsuitable for real-time control integration.

The root cause is architectural. Both approaches maintain an explicit or implicit 3D volumetric representation. NeRF requires evaluating a deep network hundreds of thousands of times per image via ray-marching—a fundamentally serial process [18, 20, 17, 21, 19, 22]. 3DGS requires computing forward kinematics for thousands of Gaussian primitives, projecting them into 2D screen space, sorting them by depth, and alpha-compositing—a heavily sequential pipeline despite GPU acceleration [23, 24, 25, 27, 26].

1.4 Problem Statement and Core Hypothesis

1.4.1 Formal Mathematical Problem Statement

We formally define the true physical morphology of the robot at time t as the configuration manifold $\mathcal{M}(t)$. The robotic system maintains an internal computational kinematic representation $\hat{\mathcal{M}}(t)$. Embodiment drift is defined as the non-stationary, cumulative divergence between the physical reality and the digital expectation:

$$D(t) = \|\mathcal{M}(t) - \hat{\mathcal{M}}(t)\| \tag{1.1}$$

When an unmodeled kinematic fault $\Delta\mathbf{q}_{\text{fault}}$ occurs, the proprioceptive encoder estimate $\mathbf{q}_{\text{enc}}$ deviates from the true physical state $\mathbf{q}_{\text{true}}$ such that $\mathbf{q}_{\text{true}} = \mathbf{q}_{\text{enc}} + \Delta\mathbf{q}_{\text{fault}}$. This breaks traditional closed-loop control.

Let the sensory observation space be $\mathcal{O}$ (in this case, binary visual silhouettes) and the joint configuration space be $\mathcal{Q}$. The objective of the visual self-model is to learn a direct sensorimotor mapping $f_\theta : \mathcal{Q} \to \mathcal{O}$ that minimizes the reconstruction loss $\mathcal{L}(f_\theta(\mathbf{q}), \mathbf{o})$ to recover the true Cartesian end-effector position $\mathbf{x}_{\text{true}}$.

Crucially, this mapping is strictly bound by control-loop temporal dynamics. If the control system operates at frequency f_{ctrl}, dictating a strict maximum latency constraint $T_{\max} = 1/f_{\text{ctrl}}$, and t_{infer} is the inference time of the self-model, the system is physically deployable for closed-loop stabilization if and only if:

$$t_{\text{infer}} \leq T_{\max} \tag{1.2}$$

For a standard 1 kHz feedback loop, this necessitates $t_{\text{infer}} \leq 1.0$ ms.

1.4.2 The Central Hypothesis

This thesis investigates a central hypothesis: *explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling.* We propose a paradigm shift toward *direct sensorimotor decoding*: a lightweight fully convolutional network that maps joint angles directly to visual silhouettes and spatial coordinates, completely bypassing the intermediate 3D reconstruction step. We hypothesize that such an architecture can achieve sub-millisecond inference latency—compatible with 1 kHz control budgets—while maintaining reconstruction fidelity comparable to or exceeding volumetric baselines, precisely because dense pixel-wise supervision is more efficient than sparse ray sampling.

1.4.3 Empirical Scope, Constraints, and Hardware Realities

The empirical scope of this research is deliberately bounded within a highly controlled, synthetic PyBullet simulation environment. This is a deliberate necessity. Implementation on actual physical hardware is not possible within the scope of this Master's thesis due to severe, practical resource limitations. The rigorous validation of dynamic fault adaptation requires dedicated, damageable physical manipulators where structural faults can be safely and repeatedly injected. Furthermore, acquiring ground-truth millimetric spatial coordinate data to validate the visual network's predictions requires expensive, high-speed motion-capture rigs (e.g., Vicon systems). Given the strict temporal and hardware constraints of the degree program, physical deployment is unattainable at this stage. Consequently, the transition from simulated architectural validation to physical hardware deployment—including addressing the complex photometric noise of the Sim-to-Real gap—is formally deferred to the author's upcoming Ph.D. research.

1.4.4 The Temporal Stigmergy Framework for Swarm Coordination

This thesis evaluates multi-agent coordination via *temporal stigmergy*. Stigmergy is a biological mechanism of indirect coordination observed in social insects such as termites and ants. Individual agents do not communicate directly; instead, they alter the environment in ways that stimulate adaptive responses in subsequent agents. In this thesis, the shared "environment" is algorithmically abstracted into a persistent digital memory ledger—the `factory_state` variable. This state-dependent, history-aware behavioral adaptation is the absolute core logic of decentralized swarm coordination. Proving that sub-millisecond individual self-diagnosis enables such temporal coordination validates the computational prerequisite for eventual physical swarms.

1.5 Research Questions

Four research questions guide this investigation, progressing from architectural efficiency through control integration and fault tolerance to collective adaptation.

1.5.1 RQ1: Architectural Efficiency

To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D implicit and explicit volumetric rendering paradigms with 2D direct sensorimotor decoding, and what is the resulting impact on morphological reconstruction fidelity measured in PSNR?

1.5.2 RQ2: Control Integration

Can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and sub-millisecond latency to actively stabilize a Jacobian-based inverse kinematics control loop?

1.5.3 RQ3: Fault Tolerance

How does the integration of sub-millisecond visual self-estimates alter end-effector trajectory and error convergence compared to a purely proprioceptive baseline under induced mechanical faults such as severe encoder slip and structural damage?

1.5.4 RQ4: Collective Adaptation

Can the individual computational efficiency of a sub-millisecond self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination?

1.6 Methodological Contributions

This thesis makes four primary contributions:

1. **Formal Benchmarking of the Latency Paradox**: The first contribution is the formal benchmarking of the latency paradox in neural rendering, exposing their control incompatibility relative to the 1 ms control budget.

2. **The NeuroKin Direct Sensorimotor Decoder**: The second contribution is the NeuroKin architecture. NeuroKin is a lightweight, fully convolutional direct sensorimotor decoder that completely bypasses volumetric reconstruction, mapping joint angles directly to visual silhouettes. NeuroKin achieves 21.88 dB PSNR at 7,400 FPS (0.135 ms latency)—a 1,418× speedup over implicit NeRF baselines.

3. **Visual Closed-Loop Damage Adaptation**: The third contribution lies in visual closed-loop control and dynamic damage adaptation. This research successfully synthesizes NeuroKin's sub-millisecond spatial estimates with a Jacobian-based inverse-kinematics feedback loop, allowing the controller to re-stabilize the end-effector trajectory post-fault.

4. **Temporal Stigmergy for Sequential Swarm Coordination**: The fourth contribution is the validation of stigmergic swarm coordination. The NeuroKin-enabled 100-stage sequential swarm maintains 88% success post-fault compared to the 13% success rate of the baseline.

1.7 Thesis Organization

The remainder of this thesis is structured to systematically address the research questions. Chapter 2 provides an HTML-accessible critical synthesis of relevant literature. Chapter 3 details the empirical simulation methodology and latency bottleneck, outlining the PyBullet data generation pipeline. Chapter 4 formally introduces the core technical innovation—the NeuroKin architecture. Chapter 5 moves from static rendering metrics to dynamic control, outlining the integration of NeuroKin into an inverse-kinematics loop. Chapter 6 scales validation to the collective level, unpacking the 100-stage operational swarm chain and the mechanics of the inter-agent communication ledger. Chapter 7 evaluates the overarching hypothesis, Pareto trade-offs, and threats to validity. Finally, Chapter 8 synthesizes the research findings, directly answers the formal research questions, and maps a concrete technological roadmap for transitioning these validated sequential capabilities into physically embodied, spatially concurrent swarm deployments.

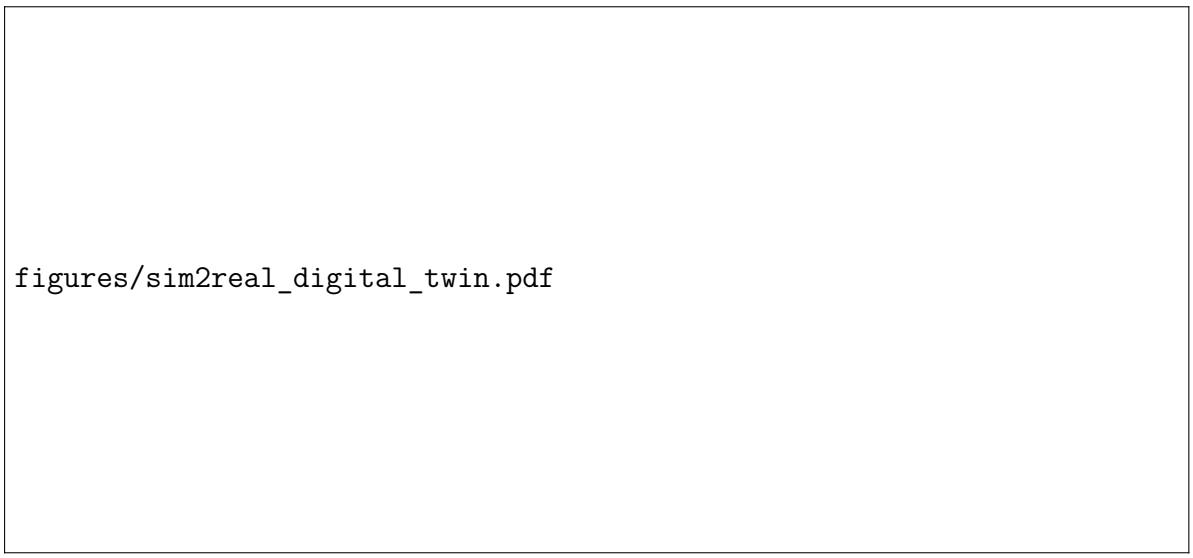


Figure 1.3: System workflow chart mapping continuous data collection loops from physical hardware to calibrate a simulation environment, detailing online deployment execution and residual error matching pathways.

Chapter 2

Literature Synthesis

2.1 Evolution of Self-Modeling

The capacity for autonomous adaptation to physical damage is the defining characteristic of biological resilience, yet it remains an elusive goal for robotic systems. This section treats the mathematical and conceptual structures underpinning robotic self-modeling across three distinct historical blocks—symbolic exploration, behavioral detour maps, and modern deep neural rendering methods—before isolating the foundational speed ceilings and system stability challenges motivating our work.

Figure 2.1 maps this field to structure our critical literature synthesis.

Figure 2.1: Historical taxonomy of robot self-modeling paradigms: chronological evolution from symbolic approaches (2006) through behavioral learning, learned representations, visual estimation, to deployable systems (2026).

2.1.1 The Symbolic Era: Estimation-Exploration

The earliest formalized paradigm relied heavily on AISI and kinematic inference. The seminal work of Bongard, Zykov, and Lipson established the first complete closed-loop self-modeling system on actual physical robots [1]. Their approach viewed self-modeling as a continuous cycle of hypothesis generation and active physical exploration. The robot maintained a population of competing symbolic models representing various candidate topologies—different possible configurations of its own body. To discriminate among these hypotheses, the robot systematically synthesized exploratory motor commands mathematically designed to maximize the sensory disagreement among the candidate models.

Upon executing these actions, the robot collected sensor data and systematically pruned models that failed to predict observed outcomes. After approximately 16 modeling-testing cycles spanning several minutes of real-world experimentation, the robot converged on an accurate damaged model and synthesized a compensatory gait through reinforcement learning on the internal simulation.

The core innovation was an active learning loop: the fitness of a test action was determined by its ability to cause disagreement among the best candidate models, thereby maximally reducing uncertainty [1]. While conceptually elegant, this approach suffers from the symbolic bottleneck: the robot can only discover models constructible from predefined physics engine primitives (cylinders, hinge joints). It cannot represent non-rigid deformations, soft-body dynamics, or arbitrary visual damage—limitations that become critical in unstructured environments where failure modes are unpredictable [4]. As noted by Kwiatkowski and Lipson, the search space of symbolic physics engines is discrete and non-differentiable, making optimization slow and prone to local optima [4]. Furthermore, the approach relied heavily on simple proprioceptive sensors providing limited information compared to high-bandwidth vision.

2.1.2 The Behavioral Era: Avoidance Over Adaptation

Recognizing the computational intensity of explicit physical modeling, Cully and colleagues introduced Intelligent Trial and Error (IT&E), which bypassed damage diagnosis entirely [2]. Their key insight was to focus on finding what still works rather than diagnosing what went wrong. Using the MAP-Elites quality-diversity algorithm [3], they pre-computed a behavioral repertoire of approximately 13,000 distinct gaits in simulation—a process requiring 40 million simulations over two weeks of computation. This repertoire is a high-dimensional grid where each cell corresponds to a specific behavioral descriptor (e.g., duty cycle versus leg offset) and stores the highest-performing controller found for that descriptor.

When damaged in the field, the robot treated this map as a Bayesian prior, using Gaussian Process regression to search for a behavior that performs well in the real world. By testing a few dozen distinct behaviors, the Gaussian Process updated its posterior mean and variance, rapidly identifying a compensatory gait. Cully and colleagues demonstrated recovery in less than two minutes of real-world testing across various damage scenarios, including removal of entire legs [2].

However, IT&E represents a strategy of avoidance, not adaptation. The robot does not repair its internal model—it discards the parts of the behavioral space that no longer function. While effective for redundant locomotion tasks where multiple gaits can achieve similar forward velocity, this approach fails for precision manipulation tasks that require exact geometric awareness [4]. If a robot needs to reach through a narrow aperture, it cannot simply "try a different gait"—it must understand the precise Cartesian geometry of its damaged arm. Furthermore, as robotic complexity increases, the curse of dimensionality makes pre-computing dense behavioral maps prohibitively expensive: a 6-DOF arm with

even modest discretization would require billions of map entries, far exceeding computational budgets.

2.1.3 Task-Agnostic Learned Dynamics Models

The advent of deep learning enabled a third paradigm: learning dynamics models that encode the mapping from motor commands to sensory consequences. Kwiatkowski and Lipson proposed task-agnostic self-modeling, training deep forward models to predict the future state of the robot given the current state and action [4]. This represented a pivotal shift from symbolic physics to learnable physics. The same approach was demonstrated to work as a complete reinforcement learning environment to learn zero-shot tasks across morphologies in a follow-up study [5].

While theoretically powerful, these data-driven models face a fundamental limitation: systematic bias accumulates progressively across extended planning horizons [4]. Simulated trajectories diverge increasingly from actual robot behavior, rendering the approach impractical for tasks requiring accurate prediction beyond very short time windows. This horizon-dependent degradation motivates approaches that prioritize shorter planning windows or use learned corrections rather than pure forward simulation.

2.1.4 Proprioceptive Morphology Identification

A parallel thread of research focuses on proprioceptive identification—restoring embodiment using internal signals alone, without the occlusions and lighting failures that often cause vision-only pipelines to break [6]. This offers significant deployment advantages: the approach remains robust across diverse lighting conditions and operates without calibrated cameras. However, eliminating external visual information imposes real functional costs: expressiveness is substantially reduced compared to vision-based methods [8]. This is not merely a theoretical gap but a genuine operating constraint that limits the complexity of tasks these systems can support compared to their multi-modal counterparts.

Meta-self-modeling restores morphology parameters between motion signature reconfigurations [6], and IMU-centric methods show hardware-realized damage sensing with minimal sensing overhead on legged robots [8]. Analytical modular pipelines remain strong baselines in the case of known assembly constraints, offering interpretable, stable, and automatically generated kinematic and dynamic models [7]. The frontier is intervention, not just identification: controlled breakage of proprioception-driven morphology demonstrates that active structural adaptation is supported by internal sensing, and that morphology itself is a form of adaptive feedback control [64, 9].

2.1.5 The Visual Era: Deep Self-Models and Neural Rendering

The current state-of-the-art leverages neural rendering to learn visual self-models directly from camera observations. Chen and colleagues introduced occupancy query learning as a

foundational approach [10]. Signed Distance Functions (SDFs) conditioned on joint angles enable differentiable collision checking directly from RGB-D sensor streams, supporting both motion planning and recovery from damage. Their multi-view hardware system achieved approximately one percent workspace accuracy and demonstrated support for planning and post-damage recovery. However, practical deployment introduced significant challenges: careful camera placement became critical for observability, precise calibration was required for geometric accuracy, and performance degraded substantially under occlusion, shadows, blur, and noise [10].

Hu, Lin, and Lipson reduced the camera system to a single RGB sensor while learning the Free-Form Kinematic Self-Model (FFKSM) from a single viewpoint [11]. The reparameterization of camera frames in terms of the first two joints facilitated the learning process. This approach supports inverse kinematics, path planning, and post-damage recovery. However, volumetric rendering via NeRF remained computationally expensive, and single-view training introduced persistent vulnerabilities: occlusion sensitivity and perspective-dependent predictions.

More recently, Kinematic 3D Gaussian Splatting (K-3DGS) has emerged as an explicit alternative to implicit NeRFs [15]. 3DGS replaces ray marching with differentiable rasterization of explicit anisotropic Gaussians, enabling high frame rate rendering and fast optimization. By attaching explicit Gaussian primitives to a differentiable kinematic chain and leveraging rasterization-based rendering rather than ray-marching, K-3DGS promises faster inference [15, 16]. Each Gaussian primitive is defined by a 3D mean position, covariance matrix, opacity, and color, allowing the model to render the robot in any arbitrary pose through standard graphics pipeline operations.

Yet, as this thesis establishes, the immense representational power of these neural rendering architectures has introduced a severe computational bottleneck that completely obstructs physical deployment in real-time control loops.

2.2 Neural Rendering Foundations: The Speed-Quality Tradeoff

The computational bottleneck of visual self-modeling stems fundamentally from the choice of 3D representation. This section examines the mathematical foundations of Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS), the two dominant paradigms for neural scene representation, while also surveying the broader landscape of articulated object reconstruction and digital twin generation.

2.2.1 Neural Radiance Fields: Implicit Scene Representation

Neural Radiance Fields (NeRFs), introduced by Mildenhall and colleagues, revolutionized computer vision by representing scenes as continuous functions mapping 5D coordinates

to color and volume density [18]. The function $F_\theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ is parameterized by a multi-layer perceptron (MLP), where $\mathbf{x} \in \mathbb{R}^3$ is a spatial position, $\mathbf{d} \in \mathbb{R}^3$ is a viewing direction, $\mathbf{c} \in \mathbb{R}^3$ is the emitted color, and $\sigma \in \mathbb{R}^+$ is the volume density.

To render a pixel, NeRF approximates the volume rendering integral along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $\mathbf{d}$ is the ray direction. The expected color $C(\mathbf{r})$ is:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt \tag{2.1}$$

where $T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds\right)$ is the accumulated transmittance. In practice, this continuous integral is approximated via numerical quadrature using stratified sampling. For a standard 100×100 image with $N = 64$ samples per ray, this formulation requires $100 \times 100 \times 64 = 640,000$ network evaluations per frame [18]. At 5.22 FPS, this amounts to 3.2 million MLP evaluations per second—a computational burden that our subsequent implementations directly address.

To capture high-frequency geometric details, NeRF employs positional encoding, mapping input coordinates into a higher-dimensional space using sinusoidal functions:

$$\gamma(p) = [\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)] \tag{2.2}$$

For 3D coordinates, this transforms $\mathbf{x} \in \mathbb{R}^3$ to a $3 \times (1 + 2L)$-dimensional vector. Following the original NeRF, we use $L = 5$ frequencies, yielding 33-dimensional encodings.

Standard NeRF assumes a static scene. For articulated robots, the scene geometry changes with joint configuration. Dynamic variants therefore learn a canonical representation together with a deformation field that maps observation-space points to canonical space [19, 22]. For robotic self-modeling, the time variable can be replaced with the joint configuration vector, learning a mapping from configuration space to deformation [11].

Several acceleration techniques have been proposed to address NeRF's computational cost [20, 17, 21, ?]. However, even with acceleration, the core robotics bottleneck remains fundamentally unchanged: volumetric query expense remains a fundamental constraint within tight control loops.

2.2.2 3D Gaussian Splatting: Explicit Geometric Primitives

3D Gaussian Splatting (3DGS), introduced by Kerbl and colleagues, replaces implicit neural fields with explicit discrete geometric primitives [23]. By abandoning the MLP entirely during inference and relying on highly optimized rasterization pipelines, 3DGS promises a pathway to real-time, high-fidelity scene representation.

Unlike NeRF, which models a continuous field, 3DGS represents a scene as an unstructured point cloud of discrete 3D Gaussian primitives. Each individual Gaussian is mathematically defined by a 3D mean position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, an anisotropic covariance matrix $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3\times3}$ (parameterized by scaling $\mathbf{S}$ and rotation $\mathbf{R}$ via $\boldsymbol{\Sigma} = \mathbf{RSS}^\mathsf{T}\mathbf{R}^\mathsf{T}$), an opacity scalar

$\alpha_i \in [0, 1]$, and spherical harmonics coefficients representing view-dependent directional color.

The core representational power of 3DGS lies in the anisotropy of the covariance matrix. A purely isotropic (spherical) Gaussian is severely limited in its ability to model sharp edges or flat surfaces. By allowing the covariance matrix to stretch and compress unevenly along its principal axes, a single Gaussian can deform into an elongated ellipsoid or a flattened disk, enabling precise modeling of complex geometry with fewer primitives [23].

Optimizing a 3×3 covariance matrix directly using gradient descent is mathematically precarious, as the matrix must remain strictly positive semi-definite. To enforce this constraint, Kerbl and colleagues factorize the covariance matrix into a scaling vector $\mathbf{s} \in \mathbb{R}^3$ (with $s_i > 0$) and a rotation quaternion $\mathbf{q} \in \mathbb{R}^4$, then reparameterize the optimization to maintain positive semi-definiteness [23].

The true speed advantage of 3DGS is derived from its rendering pipeline. To synthesize an image, 3DGS completely abandons ray-marching, instead utilizing a highly parallelized tile-based rasterization architecture. For a given camera with viewing transformation $\mathbf{W}$ (world-to-camera) and projection $\mathbf{J}$ (Jacobian of the perspective projection), the projected 2D covariance $\mathbf{\Sigma}'$ is:

$$\mathbf{\Sigma}' = \mathbf{JW\Sigma W}^\mathsf{T}\mathbf{J}^\mathsf{T} \tag{2.3}$$

The 2D Gaussian function for pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ is $G^{2D}(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u} - \boldsymbol{\mu}')^\mathsf{T}\mathbf{\Sigma}'^{-1}(\mathbf{u} -\right.$ and the final pixel color is computed via alpha compositing with Gaussians sorted by depth:

$$C(\mathbf{u}) = \sum_{i=1}^{N} \alpha_i G_i^{2D}(\mathbf{u})\mathbf{c}_i \prod_{j=1}^{i-1}(1 - \alpha_j G_j^{2D}(\mathbf{u})) \tag{2.4}$$

The rapid adoption of 3DGS within robotics has pushed it beyond static rendering and toward mapping, tracking, and simulation-oriented pipelines [24, 25, 27, 26]. However, the computational cost still scales with the number of Gaussians, which matters for embedded deployment and any system that must reason about many candidate morphologies at control time.

2.2.3 The Failure of Anisotropy on Robotic Morphologies

Given these computational advantages, the robotics literature adapted 3DGS for kinematic self-modeling. The resulting architecture, Kinematic 3DGS (K-3DGS), attaches Gaussian primitives to a differentiable kinematic tree.

In theory, the anisotropic nature of 3DGS makes it ideal for robotic manipulators. The physical morphology of a serial manipulator is dominated by long, thin, rigid cylindrical links. An optimizer should naturally stretch the scaling vectors of Gaussians along the central longitudinal axis of each link, achieving aspect ratios of 20:1:1.

In practice, however, this anisotropic optimization frequently fails catastrophically when applied to sparse, single-camera robotic datasets. Optimizing highly anisotropic rotational quaternions on featureless robotic links introduces a severely non-convex, ill-posed

optimization landscape [15]. Because the robotic link is visually uniform, the gradient descent algorithm receives contradictory loss signals regarding the optimal rotation of the elongated Gaussian. This phenomenon is termed *anisotropic mode collapse*.

To maintain numerical stability and ensure reliable convergence, practical K-3DGS implementations are frequently forced to retreat to purely isotropic representations. The scaling vector is constrained to be equal in all dimensions, forcing every Gaussian primitive to remain a perfect sphere. While this restriction mathematically stabilizes the training process, it fundamentally compromises the structural precision of the representation [15]. Attempting to model a long, sharp-edged robotic cylinder using only perfect spheres results in severe geometric aliasing. When rasterized, this isotropic chaining yields pervasive, structurally ambiguous "blobby" artifacts that blur the true spatial boundaries of the robot.

2.2.4 Articulated Object Reconstruction for Digital Twin Generation

Automated digital twin generation approaches follow three broad strategies. Differentiable reconstruction pipelines avoid manual URDF creation by optimizing directly from observations [**?**, 28]. Feed-forward generative pipelines prioritize generation speed over per-instance optimization [29, 31, 35, 33], and physics-constrained variants add domain structure to reduce planning-critical errors [34, 32, 30]. Across all three, the core trade-off is clear: richer geometry and more automation often come with higher optimization cost, weaker robustness under occlusion, or reduced physical reliability.

For manipulation tasks requiring closed-loop control, explicit world models offer an alternative emphasis: separate the geometric model from the control logic, then preserve only the model fidelity truly needed by the downstream task [**?**, 37, 38]. This is the same design pressure that motivates NeuroKin's minimal representation principle.

2.2.5 Physics-Consistent Robot Models and Manipulation Planning

Gaussian twin representations have progressed from display assets toward functional planning models [42, 36, 39, 40]. Planning stacks increasingly merge rendering, geometry, and control [44, 45, 46, 52, 53], but a persistent limitation remains: strong simulation metrics do not automatically imply robust deployment. Horizon length, environmental robustness, and contact fidelity still determine whether a model generalizes beyond laboratory settings.

2.2.6 Cross-Embodiment Transfer and Morphological Understanding

Cross-embodiment transfer performance improves measurably when morphological structure is encoded as kinematic connectivity rather than visual appearance alone [48, 49]. The broad

lesson is that structural priors matter more than texture-level similarity when embodiments differ. Whole-body control frameworks increasingly leverage these insights for humanoid deployment [55, 50, 63, 94, 61]. Frameworks couple base and arm dynamics for dynamic object grasping with legged manipulators. These frameworks demonstrate that explicit structure priors substantially facilitate transfer across embodiments.

2.2.7 Sim-to-Real Transfer via Digital Twins

Equating rendering fidelity with deployment success is a flawed assumption that has led to numerous simulation-only results failing on hardware [54]. The reality gaps decompose along four distinct dimensions: dynamics mismatch (physics simulation divergence), perception mismatch (sensor modeling inaccuracy), actuation mismatch (motor characteristics divergence), and system construction mismatch (morphological/structural misalignment) [54, 41].

Effective transfer combines targeted system identification with methodical robustness training rather than pursuing perfect simulator fidelity. Hardware evidence supports this principle: quadrupeds trained with actuator-aware system identification and domain randomization achieve zero-shot transfer to real platforms [41]. Bicycles trained with explicit robustness-focused procedures transfer reliably across diverse terrain [**?**]. A consistent pattern emerges across successful systems: calibration without concurrent robustness training fails catastrophically, while robustness training without accurate baseline calibration merely wastes training data without achieving true transfer capability [41, 54].

Digital twins play multiple roles in sim-to-real pipelines. Dynamics and system identification calibrate physics parameters so simulated rollouts reflect hardware behavior [**?**, 43]. Perception and rendering transfer reduce visual domain gaps by reconstructing photorealistic scenes or preserving semantic invariants for training and validation [59, 60]. Demonstration synthesis and augmentation generate diverse trajectories and viewpoints from sparse real demonstrations by manipulating reconstructed scene components [47, 62]. Deployment-time synchronization uses a continuously synchronized simulator as a mediator, where the policy acts on the twin while the robot tracks twin trajectories [56]. Evaluation and sim-real correspondence estimate whether simulator performance correlates with hardware outcomes, enabling safer iteration before deployment [65, 66].

2.3 Damage Recovery and Fault Tolerance

Damage recovery is where self-model claims face the most severe test: can the system operate with real faults, not just under nominal simulation conditions? This section surveys detection methods, recovery policy generation, humanoid-specific challenges, and practical deployment lessons.

2.3.1 Detection Methods and Early Intervention

Early fault detection methods are valuable only when directly tied to practical intervention latency and downstream control outcomes. VAE-based anomaly monitoring successfully detects anomalous force-torque trajectory patterns at sub-3-millisecond latencies in simulated manipulation tasks [71]. Contrastive forward prediction reduces tracking error by up to 61.5% on damaged legged machines [72]. Low-latency detection remains critical because delayed intervention necessarily closes the available recovery window, making response speed a fundamental constraint.

2.3.2 Recovery Policy Generation and Adaptation

Detection must translate to actionable recovery policies. Vision-language recovery can guide task-level correction on 7-DoF manipulators; however, unconstrained language reasoning often violates dynamics limits [73]. Structured recovery stacks provide a pragmatic alternative. Dream2Fix synthesizes counterfactual failures on a large scale (120,000+ simulation cases), achieving 46% zero-shot closed-loop recovery on hardware [78]. Embodiment-conditioned diffusion policies achieve higher constraint satisfaction (36.85% to 74.51%) under zero-shot generalization to unseen failure conditions [75].

2.3.3 Humanoid Recovery and Stability

Humanoid recovery introduces substantial additional complexity through stability and safety constraints not present in manipulator systems. Balance-informed critic networks grounded in capture point theory and centroidal dynamics computation achieve strong simulated stand-up recovery from falls [76]. Preference-conditioned control policies expand operational robustness envelopes by dynamically switching between efficiency-optimized and robustness-maximized control modes [**?**]. Fault-aware locomotion methods maintain stable tracking performance under joint faults within simulation [79]. Humanoid parkour research demonstrates impressive agile obstacle vaulting and long-horizon terrain traversal capabilities in controlled environments [58].

2.3.4 Practical Deployment Lessons

Two practical lessons emerge consistently across successful deployment systems. First, online discrepancy monitoring proves effective when adaptation remains rapid and theoretically bounded, preventing unbounded divergence. World-model residual monitoring specifically enables rapid policy recovery following physical domain shifts like payload changes or link damage [69]. Second, residual adaptation channels remain most effective when carefully aligned with baseline policy safety constraints. This principle is demonstrated empirically: bounded residual control reduced Go1 quadruped recovery steps from 4,500 to 168 following a significant mass increase—a 96% reduction in recovery time establishing the practical value

of principled multi-level adaptation [74].

A consistent deployment principle emerges: recovery systems should be evaluated by intervention speed, stability margin, and post-fault task continuity—not by reconstruction fidelity. Visual quality is not the determining factor.

2.4 Continual Learning for Self-Model Maintenance

Continual maintenance of self-models confronts the stability-plasticity dilemma: updates must absorb new damage conditions while preserving knowledge of healthy morphology [82, 83]. Catastrophic forgetting is well documented; in robotic deployment it is critical because forgetting can generate unsafe actions and hardware destruction [80, 81].

2.4.1 Balancing Plasticity and Stability

Meta-learning approaches contribute meaningful value when morphology characteristics shift frequently and the time window available for adaptation compresses severely. Model-agnostic meta-learning (MAML) reduces post-damage sample requirements for model training [88], and structure-constrained updates enable adaptation without full retraining. Elastic Weight Consolidation (EWC) mitigates catastrophic forgetting by approximating the posterior distribution of weights using the Fisher information matrix [84].

However, EWC is ill-suited for robotic damage recovery for two reasons. First, computational cost: calculating the Fisher Matrix requires computing second-order derivatives (Hessian) or approximating them via squared gradients. For high-dimensional rendering networks (millions of parameters), storing and computing the Fisher information is prohibitively expensive for real-time adaptation on edge hardware. Second, lack of modularity: EWC slows forgetting but does not enable the robot to isolate the damage. It treats the network as a monolith with varying "stiffness". If damage is severe, the robot needs to change high-importance weights, but EWC prevents this, leading to under-fitting of the damage (the "plasticity gap").

The failure of weight-based regularization suggests that topology, rather than weights, may hold the key to adaptive resilience. Research by Chechik and colleagues demonstrated through computational models that synaptic pruning is an optimal strategy strictly under metabolic or resource constraints [93]. They proved that in the absence of such energetic costs, pruning cannot improve network performance compared to an intact network—a finding that aligns precisely with edge compute constraints in robotics, where power budgets (15W typical for Jetson Orin) and memory bandwidth are severely limited. The biological precedent suggests that sparse, modular architectures may be not merely beneficial but necessary for resource-constrained adaptive systems.

2.4.2 Resource-Efficient Adaptation via Sparsity

The Lottery Ticket Hypothesis formalizes the value of sparsity in deep learning: "A randomly-initialized dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations" [85]. This suggests that the vast majority of weights in a dense self-model are redundant. Han and colleagues empirically demonstrated this, achieving compression ratios of 49× on AlexNet and 39× on VGG-16 through a combination of pruning, quantization, and Huffman coding [87]. Model storage reduced from 240 MB to 6.9 MB while maintaining baseline accuracy.

Sparse Evolutionary Training (SET) maintains sparse topology throughout training rather than training dense and pruning post-hoc [86]. In each epoch, SET prunes the weakest connections and regrows random new ones, allowing the network to explore the topology space dynamically. For robotics, sparsity offers dual benefits. First, efficiency: sparse matrix operations reduce computational latency, directly addressing the rendering bottleneck. Second, modularity: by enforcing sparsity, the network is forced to develop modular sub-structures. Damage to one module can be repaired by updating only the weights within that module, significantly reducing catastrophic interference [85, 86]. This biological inspiration—sparse, modular connectivity enabling localized plasticity—motivates our architectural explorations in subsequent sections.

2.4.3 Safety-Constrained and Bounded Adaptation

Safety-constrained learning maintains adaptation within verified regions through explicit constraints. Barrier functions enforce that the robot stays within safe regions of state space, providing formal safety guarantees [68]. System-level state partitioning separates high-level reasoning from time-critical control loops, reducing cascading failures. Bounded residual control keeps adaptations within safe envelopes, as demonstrated empirically by the 96% reduction in recovery time on Go1 quadruped [74].

2.5 Theoretical Foundations of Swarm Coordination and Temporal Stigmergy

This section establishes the theoretical framework of temporal stigmergy that underpins the sequential swarm evaluation in Chapter 6, drawing on biological inspiration and multi-agent systems research.

2.5.1 Stigmergy: Biological and Algorithmic Foundations

Stigmergy is a biological mechanism of indirect coordination, originally observed in social insects such as termites and ants. Individual agents do not communicate directly via explicit

signaling protocols or negotiated consensus algorithms. Instead, they subtly alter the physical environment—for example, depositing a pheromone trail or adding a pellet of construction material. These environmental modifications serve as localized, persistent stimuli, triggering specific behavioral responses from subsequent agents that encounter them.

The critical property of stigmergy is that coordination emerges from local interactions without centralized planning or direct agent-to-agent communication. The environment itself becomes the communication channel. This property makes stigmergy particularly attractive for robotic swarms operating in communication-denied environments where radio bandwidth is limited or connectivity is intermittent.

Translated into algorithmic robotic systems, stigmergy eliminates the need for continuous peer-to-peer communication bandwidth. Agents react independently and locally to diagnostic traces left in the environment by their peers [1]. This stands in contrast to coordination schemes that require denser communication or centralized training [69, 67].

2.5.2 Temporal Stigmergy in Sequential Pipelines

In this thesis, stigmergic coordination is achieved across time rather than physical space. The shared "environment" is abstracted into a persistent digital memory ledger—the `factory_state` variable tracked across sequential agents. This represents a *temporal stigmergy*: agents coordinate by reading and writing to a shared state that persists across sequential operations, with the dimension of coordination being time rather than space.

Formally, we define temporal stigmergy as a coordination mechanism where agents $\{A_1, A_2, \ldots, A_N\}$ share a persistent state $S \in \mathcal{S}$. Each agent A_i executes the following protocol:

1. Reads the current state S_{i-1} (the state after the previous agent's execution).
2. Selects behavior $B_i = \pi(S_{i-1})$ according to a policy $\pi : \mathcal{S} \to \mathcal{B}$.
3. Executes the behavior and observes outcome $O_i \in \{\text{success}, \text{failure}\}$.
4. Updates the state: $S_i = \Phi(S_{i-1}, O_i)$ where Φ is the state transition function.

Coordination emerges from the sequential application of π and Φ without any direct communication between agents beyond the state S that persists in the shared environment. When an agent experiences repeated mechanical failures, it updates the `consecutive_failures` integer. Subsequent agents read this trace. If the trace indicates a degraded operational history (two consecutive failures), the new agent autonomously shifts into a localized "recovery" mode: it abandons its randomized spatial target, moves to a conservative centralized coordinate, and doubles its Cartesian error tolerance. This state-dependent, history-aware behavioral adaptation is the core logic of decentralized swarm coordination.

2.5.3 Sub-Millisecond Inference as a Swarm Prerequisite

A swarm can only coordinate around diagnostic states if individual agents possess the capability to produce those states reliably and in a timely manner. If an agent's internal self-model requires 200 ms per inference, it cannot reliably determine whether a failure was caused by occlusion, oscillation, or morphological drift—by the time the diagnosis completes, the system state may have changed. By proving that NeuroKin provides robust, sub-millisecond visual self-diagnosis (0.400 ms latency), this thesis establishes the computational prerequisite for swarm endurance.

Sub-millisecond inference enables three properties essential for stigmergic coordination: real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed); immediate ledger updates (the shared state is updated before the next control cycle begins); and rapid adaptation (subsequent agents read the updated state without perceptible delay). Without sub-millisecond inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

2.6 Summary of Literature Findings

This exhaustive literature synthesis reveals several critical findings that directly motivate the experimental contributions of this thesis. **First**, visual self-modeling via neural rendering (NeRF and 3DGS) has achieved impressive reconstruction fidelity, with reported PSNR values exceeding 23 dB on benchmark datasets. However, the field has systematically under-emphasized the latency implications of volumetric reconstruction for real-time control. The reported frame rates (5–37 FPS) are fundamentally incompatible with the 1 kHz (1,000 Hz) control loops required for dynamic robotic systems. This latency paradox represents a structural mismatch, not an engineering optimization problem. **Second**, direct sensorimotor decoding—bypassing explicit 3D reconstruction entirely—has been explored in limited contexts but never systematically compared against volumetric approaches on controlled benchmarks. The theoretical advantage of dense gradient supervision suggests that direct decoding could achieve both higher speed and better quality, but this hypothesis requires rigorous empirical validation. **Third**, continual learning for self-model maintenance faces unresolved challenges around catastrophic forgetting. Existing approaches (EWC, sparse subnetworks, mixture-of-experts) offer partial solutions but have not been validated in damage recovery scenarios where the distribution shifts abruptly and safety constraints dominate. The biological precedent of synaptic pruning under resource constraints suggests that sparse, modular architectures may be necessary for edge-deployable adaptive systems. **Fourth**, digital twin frameworks for robotics have emphasized geometric fidelity and simulation accuracy but have neglected the temporal dimension of self-modeling: the twin must be queryable at control rates to be useful for online adaptation. This temporal requirement aligns with the latency findings from neural rendering analysis. Recent work on

Robo-GS, ArticulatedGS, and differentiable simulation pipelines demonstrates progress, but the evaluation gap—simulation metrics over short horizons versus deployment metrics over extended operation—remains unresolved. **Fifth**, stigmergic coordination offers a promising framework for decentralized multi-agent fault tolerance, but the prerequisite—fast, reliable individual self-diagnosis—has not been demonstrated in prior work. Temporal stigmergy, as Alpha-formalized in this thesis, provides a theoretical bridge from biological inspiration to algorithmic implementation, with the critical enabling condition being sub-millisecond inference. **Sixth**, across all surveyed literature, a consistent evaluation gap emerges: most papers report reconstruction metrics (PSNR, SSIM, LPIPS) but not task-level outcomes (success rate, recovery time, stability margins). This disconnect between diagnostic metrics and deployment metrics limits the field's ability to assess practical readiness. Hardware validation remains the gold standard, yet simulation-only results continue to dominate the literature.

These findings collectively motivate the experimental investigation presented in the following chapters, which progresses from baseline implementation through architectural innovation to multi-agent validation.

2.7 Research Questions Grounded in Literature

Based on the literature synthesis, four research questions guide the experimental investigation. These questions emerge directly from identified gaps rather than being imposed a priori.

2.7.1 RQ1: Architectural Efficiency

To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D implicit and explicit volumetric rendering paradigms with 2D direct sensorimotor decoding, and what is the resulting impact on morphological reconstruction fidelity measured in PSNR? This question probes the fundamental architectural trade-off that the literature has systematically under-explored. The hypothesis is that dense gradient supervision from direct pixel prediction will achieve both higher speed and better quality than sparse volumetric sampling.

2.7.2 RQ2: Control Integration

Can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and sub-millisecond latency to actively stabilize a Jacobian-based inverse kinematics control loop? This question moves beyond static rendering metrics to dynamic closed-loop performance, asking whether the network's predictions are smooth and accurate enough to serve as the primary state estimate for feedback control—a capability not demonstrated for any neural rendering self-model to date.

2.7.3 RQ3: Fault Tolerance

How does the integration of sub-millisecond visual self-estimates alter end-effector trajectory and error convergence compared to a purely proprioceptive baseline under induced mechanical faults such as severe encoder slip and structural damage? This question evaluates the system's resilience under the very conditions that motivate self-modeling in the first place: when physical damage corrupts proprioceptive readings, can visual estimates override them and maintain stable control?

2.7.4 RQ4: Collective Adaptation

Can the individual computational efficiency of a sub-millisecond self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination? This question scales the investigation from single-agent survival to collective endurance, testing whether fast individual self-diagnosis can support emergent coordination without explicit inter-agent communication. The information-theoretic efficiency of stigmergy—coordination with O(1) bits per agent—makes it particularly attractive for bandwidth-constrained swarms.

The following chapter details the experimental methodology for addressing these research questions, beginning with the baseline implementations that establish the latency bottleneck.

Chapter 3

Baseline Implementations and the Latency Bottleneck

To rigorously characterize the latency paradox that motivates this thesis, we independent-reimplement and evaluate two state-of-the-art visual self-modeling architectures: the Free-Form Kinematic Self-Model (FFKSM) based on implicit neural radiance fields [11], and Kinematic 3D Gaussian Splatting (K-3DGS) based on explicit geometric primitives [15]. Both are reproduced from published specifications using an identical dataset, hardware environment, and evaluation protocol. This controlled comparison establishes the quantitative baseline against which our proposed NeuroKin architecture is measured.

3.1 Synthetic Data Generation via Lorenz Attractors

All experiments use a dataset of 2,000 samples generated from a PyBullet simulation of a 4-DOF serial manipulator. This section details the complete data generation pipeline, including simulation configuration, trajectory planning, and preprocessing procedures.

3.1.1 Simulation Environment Configuration

The PyBullet physics simulator provides a controlled environment for generating ground-truth robot observations. We configure the simulation with the following parameters:

- **Robot Model**: A 4-degree-of-freedom serial manipulator with revolute joints. The base is fixed at the origin with joint axes arranged in an anthropomorphic configuration: joint 1 rotates about the vertical axis (yaw), joints 2 and 3 rotate about horizontal axes (pitch), and joint 4 provides wrist rotation.
- **Camera Setup**: A fixed RGB camera positioned at coordinates $[1.0, 0.0, 0.0]$ with focal length 130.2545. The camera looks toward the origin, producing 100×100 pixel images with a field of view of 42 degrees.
- **Rendering**: The simulation runs in DIRECT mode (no graphical display) for maximum

speed. Each step renders the robot using PyBullet's built-in tiny renderer with shadows disabled.

- **Physics**: Gravity is set to $-9.8\,\mathrm{m/s^2}$ in the z-direction. A ground plane is placed at $z = -0.109$ m. Each joint is position-controlled with a maximum force of 100 N.

The robot's kinematic structure follows the Denavit-Hartenberg (DH) convention. The DH parameters are detailed in Table **??**.

The forward kinematics transformation from base to end-effector is given by the product of individual link transformations:

$$T_0^4 = T_0^1(\theta_1)T_1^2(\theta_2)T_2^3(\theta_3)T_3^4(\theta_4) \tag{3.1}$$

where each T_{i-1}^i is computed using the standard DH convention:

$$T_{i-1}^i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i\cos\alpha_{i-1} & \cos\theta_i\cos\alpha_{i-1} & -\sin\alpha_{i-1} & -d_i\sin\alpha_{i-1} \\ \sin\theta_i\sin\alpha_{i-1} & \cos\theta_i\sin\alpha_{i-1} & \cos\alpha_{i-1} & d_i\cos\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3.2}$$

figures/pybullet_setup.png

Figure 3.1: PyBullet simulation environment: 4-DOF articulated robot arm with fixed overhead camera and ground plane for data acquisition.

3.1.2 Lorenz Attractor Trajectory Generation

Rather than relying on random motor babbling—which produces jerky, discontinuous motions that poorly sample the workspace—we employ trajectories generated by the Lorenz dynamical system. The Lorenz attractor is a system of three coupled ordinary differential equations (ODEs) that exhibits deterministic chaos, producing trajectories that are smooth, non-repeating, and ergodic within a bounded region of state space.

The Lorenz system is defined as [91]:

$$\frac{dx}{dt} = \sigma(y - x) \tag{3.3}$$

$$\frac{dy}{dt} = x(\rho - z) - y \tag{3.4}$$

$$\frac{dz}{dt} = xy - \beta z \tag{3.5}$$

The parameter values determine the dynamical behavior. We use the classical values $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, which place the system in the chaotic regime. At these parameters, the Lyapunov exponent is positive, ensuring sensitive dependence on initial conditions and exponential divergence of nearby trajectories. The system has two unstable fixed points at $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$ and a stable fixed point at the origin (which is only stable for $\rho < 1$). For $\rho = 28$, the origin is a saddle point, and trajectories are attracted to a strange attractor—a fractal set of measure zero with non-integer Hausdorff dimension.

To numerically integrate the Lorenz system, we employ the fourth-order Runge-Kutta (RK4) method. For a general ODE $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$, the RK4 update from state $\mathbf{x}_n$ at time t_n to $\mathbf{x}_{n+1}$ at time $t_{n+1} = t_n + \Delta t$ is:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n, t_n) \tag{3.6}$$

$$\mathbf{k}_2 = \mathbf{f}(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1, t_n + \frac{\Delta t}{2}) \tag{3.7}$$

$$\mathbf{k}_3 = \mathbf{f}(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2, t_n + \frac{\Delta t}{2}) \tag{3.8}$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_n + \Delta t\mathbf{k}_3, t_n + \Delta t) \tag{3.9}$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{3.10}$$

RK4 is a fourth-order method, meaning the local truncation error is $O(\Delta t^5)$ and the global error is $O(\Delta t^4)$. This provides excellent stability and accuracy for the Lorenz system, which is stiff in certain regions of state space.

We set the integration time step to $\Delta t = 0.01$ seconds. This value was chosen through systematic experimentation: larger steps ($\Delta t \geq 0.02$) caused numerical instability and trajectory divergence, while smaller steps ($\Delta t \leq 0.005$) increased computational cost without measurable improvement in trajectory quality.

After each RK4 step, the raw Lorenz state values are scaled using the hyperbolic

tangent function:

$$\text{action} = \tanh(\alpha \cdot \mathbf{x}) \tag{3.11}$$

where $\alpha = 0.025$ is a scaling factor. The tanh function maps the unbounded Lorenz state to the bounded interval $[-1, 1]$, which corresponds to the normalized joint angle range. This scaling preserves the sign and relative magnitude of variations while ensuring the command stays within valid limits.

Two independent Lorenz systems drive the four robot joints. Specifically:

- Joint 1 follows x component of trajectory 1
- Joint 2 follows y component of trajectory 1
- Joint 3 follows x component of trajectory 2
- Joint 4 follows z component of trajectory 2

Using two independent attractors (with slightly different time steps: $\Delta t_1 = 0.01$, $\Delta t_2 = 0.012$) ensures that the four joints are not perfectly correlated, producing a richer diversity of poses.

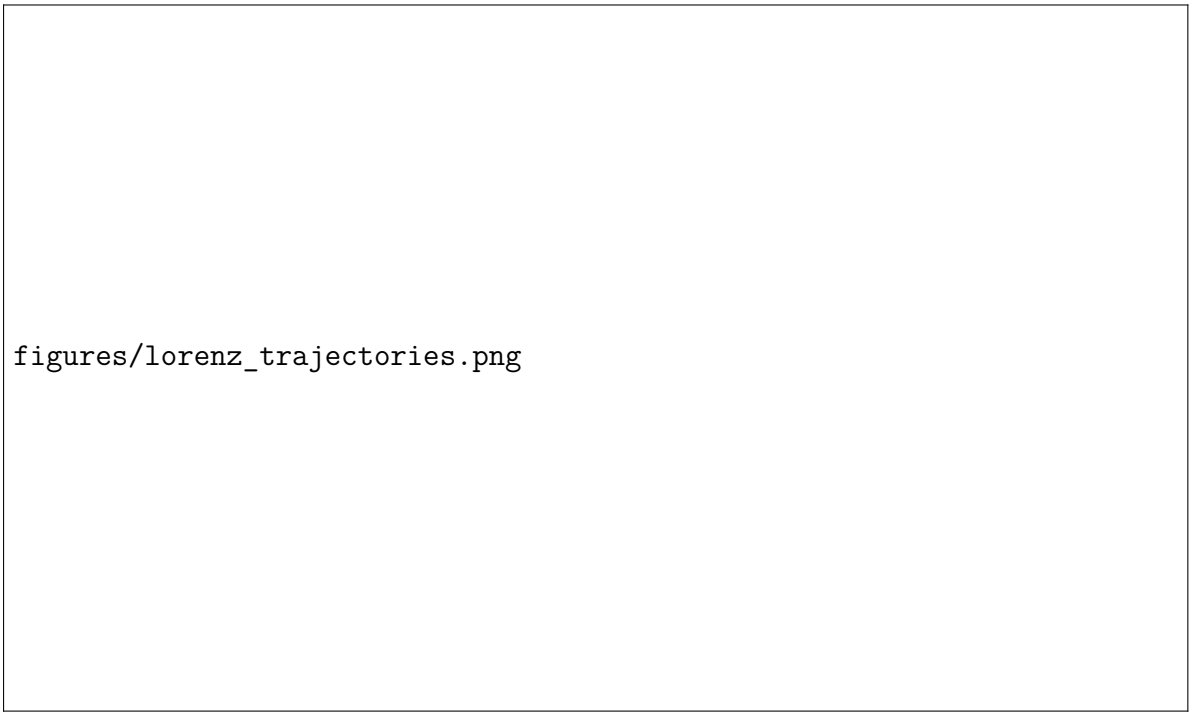


Figure 3.2: Trajectory generation via coupled Lorenz attractors: four joint angle profiles over 2000 samples, demonstrating ergodic workspace coverage.

3.1.3 Data Collection and Preprocessing

For each of the 2,000 samples, the data collection procedure follows:

1. The robot is reset to a neutral configuration (all joints at 0 degrees).
2. A target joint angle vector $\mathbf{q} \in [-90°, +90°]^4$ is computed from the Lorenz trajectories.

3. Position controllers move each joint to the target angle over 50 simulation steps (equivalent to 200 ms of physical time), allowing the robot to settle.

4. An RGB image is captured from the fixed camera.

5. Joint angles are recorded from the simulation state.

The RGB images are preprocessed to produce binary silhouette masks:

1. Convert RGB to grayscale: $I_{\text{gray}} = 0.299R + 0.587G + 0.114B$

2. Apply threshold: $I_{\text{mask}}(u, v) = \begin{cases} 1 & \text{if } I_{\text{gray}}(u, v) < 240 \\ 0 & \text{otherwise} \end{cases}$

3. The threshold value 240 was empirically determined to optimally separate robot pixels (typically lighter due to white/gray coloring) from background (darker).

The dataset comprises 2,000 samples split into 1,600 training samples (80%) and 400 validation samples (20%). The split is performed randomly with a fixed seed (42) for reproducibility.

figures/data_pipeline.png

Figure 3.3: Data preprocessing pipeline: RGB image acquisition, grayscale conversion, and binary segmentation to produce silhouette masks.

3.1.4 Dataset Statistics and Quality Analysis

Statistical analysis confirms good workspace coverage:

- Joint angle means: $-0.23°$ to $12.45°$ (near zero, as expected for a system centered at neutral)
- Joint angle standard deviations: approximately $30°$ (indicating broad exploration)
- Robot pixel occupancy: average 14.7%, range 8.9%–23.4%
- Image dimensions: 100×100 pixels (10,000 total pixels per image)

The low average occupancy (14.7%) creates severe class imbalance—85% of pixels are background (0) and only 15% are robot (1). This imbalance creates a pathological convergence trap for naive volumetric rendering, as the network can achieve low loss by simply predicting all pixels as 0. This phenomenon, which we term *black-screen convergence*, is detailed in Section 3.2.2.

Figure 3.4: Representative training samples: 4×2 grid of silhouettes demonstrating spatial diversity across the single-view dataset.

3.2 Free-Form Kinematic Self-Model (FFKSM): Implicit Neural Rendering

The FFKSM architecture, introduced by Hu, Lin, and Lipson [11], adapts Neural Radiance Fields (NeRF) [18] to articulated robot self-modeling. Unlike traditional NeRF, which assumes static scenes, FFKSM conditions the radiance field on joint angles, enabling dynamic rendering as the robot moves.

3.2.1 Core Mathematical Framework

Neural Radiance Fields for Static Scenes

NeRF represents a static 3D scene as a continuous function $F_\theta : (\mathbf{x}, \mathbf{d}) \to (\mathbf{c}, \sigma)$ parameterized by a multi-layer perceptron (MLP) with weights θ. The function maps a 3D spatial position $\mathbf{x} \in \mathbb{R}^3$ and a viewing direction $\mathbf{d} \in \mathbb{R}^3$ (normalized to unit length) to an emitted color $\mathbf{c} \in \mathbb{R}^3$ (RGB values in $[0, 1]^3$) and a volume density $\sigma \in \mathbb{R}^+$.

To render a pixel, NeRF approximates the volume rendering integral along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $t \in [t_n, t_f]$ ranges from near plane to far

plane. The expected color $C(\mathbf{r})$ is:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt \tag{3.12}$$

where $T(t)$ is the accumulated transmittance along the ray from t_n to t:

$$T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds\right) \tag{3.13}$$

In practice, this continuous integral is approximated via numerical quadrature using stratified sampling. Along each ray, N points $\{t_i\}_{i=1}^{N}$ are sampled, and the discrete approximation is:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i(1 - \exp(-\sigma_i\delta_i))\mathbf{c}_i \tag{3.14}$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples, and $T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j\delta_j\right)$.

For a 100×100 image with $N = 64$ samples per ray, this requires $100 \times 100 \times 64 = 640,000$ network evaluations per frame. This computational burden is the fundamental bottleneck that prevents real-time control integration.

Positional Encoding

Standard MLPs exhibit a spectral bias toward low-frequency functions [92]. To capture high-frequency geometric details (e.g., sharp edges of robot links), NeRF employs positional encoding that maps input coordinates to a higher-dimensional space using sinusoidal functions:

$$\gamma(p) = [\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)] \tag{3.15}$$

For 3D coordinates, this transforms $\mathbf{x} \in \mathbb{R}^3$ to a $3 \times (1 + 2L)$-dimensional vector. Following the original NeRF, we use $L = 5$ frequencies, yielding 33-dimensional encodings.

FFKSM Adaptation for Articulated Robots

FFKSM extends NeRF to articulated robots through two key innovations: the *virtual frame transformation* and the *split encoder architecture*.

Virtual Frame Transformation: The virtual frame transforms 3D query points using the first two joint angles (θ_0, θ_1) before network processing:

$$\mathbf{x}_{\text{virtual}} = R_z(-\theta_0)R_y(-\theta_1)\mathbf{x}_{\text{world}} \tag{3.16}$$

where R_z and R_y are rotation matrices:

$$R_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \tag{3.17}$$

This transformation reduces the network's learning burden by aligning coordinates with the robot's base orientation, effectively factoring out base rotation from the end-effector position learning problem. Without this prior, the network would need to learn circular symmetry—a challenging task for MLPs.

Split Encoder Architecture: FFKSM separates coordinate processing from kinematic processing:

1. **Coordinate Encoder**: Takes virtual-frame 3D points via positional encoding (5 frequencies $\rightarrow$ 33 dimensions) followed by a two-layer MLP (128 hidden units each).

2. **Kinematic Encoder**: Processes the remaining two joint angles (θ_2, θ_3) via separate positional encoding (2×5 frequencies $\rightarrow 2 \times 11 = 22$ dimensions) and an identical two-layer MLP (128 hidden units).

3. **Predictive Module**: Concatenates the 128-dimensional features from both encoders (256 total) and passes through four linear layers ($128 \rightarrow 128 \rightarrow 32 \rightarrow 2$) to predict density σ and visibility v.

The split architecture is critical: a single encoder processing $(\mathbf{x}, \theta)$ jointly would need to relearn the kinematic transformation for every 3D point. The split design allows the kinematic encoder to learn the mapping $\theta \rightarrow$ link transformation once, then apply it to all query points, dramatically improving sample efficiency.

3.2.2 Implementation Details and Reproduction Challenges

Our independent implementation of FFKSM from scratch uncovered three critical challenges not detailed in the original publication. Each challenge required significant debugging and architectural modifications.

Challenge 1: Black-Screen Convergence

Symptom: Initial training exhibited a pathological failure mode: the network converged to predicting uniformly black images (all zeros) despite achieving apparently low loss values. The loss decreased smoothly, suggesting successful optimization, but qualitative inspection revealed complete failure.

Root Cause Analysis: Robots occupy only approximately 15% of image pixels, creating severe class imbalance (85% background, 15% robot). The network discovered a local minimum where predicting the majority class (black) achieved mean squared error of ~ 0.85, yet produced no useful representation. This local minimum is extremely attractive because the gradient magnitude for background pixels (which are correctly predicted) is near zero, the gradient from robot pixels is sparse and weak, and the optimizer follows the path of least resistance, settling into the trivial solution. This failure persisted for the first 1,000 iterations, rendering standard training completely ineffective.

Attempted Solutions Narrative: In trying to resolve this early convergence failure, several loss modifications were methodically explored. Proportional weighted losses scaling the backpropagation updates exclusively on the robot boundaries were injected into the optimizer. Applying a conservative 2× and 3× scaling factor to the sparse foreground coordinates provided insufficient signal to pull the gradients out of the background local minimum. Elevating the foreground multiplier to 5× yielded initial trace improvements but introduced severe numerical boundary instability. When scaling targets were further pushed to 10× and 20×, the optimizer destabilized immediately, producing volatile oscillations and rapid network mode collapse. In parallel, a structural focal loss configuration was implemented to explicitly down-weight easy background classifications. This method introduced complex hyperparameter sweeps over focus bounds (γ, α) that proved highly sensitive, ultimately failing to break the all-zero baseline attractor. Similarly, intersection-over-union optimizations via a continuous Dice loss metric were evaluated; however, because early epoch robot tracking was sparse, the fractional loss boundaries fluctuated wildly and generated catastrophic gradient explosions.

Solution: Center-Cropping Curriculum Learning

We solved the black-screen convergence through a structured curriculum learning approach. The key insight is to eliminate the class imbalance during critical early learning by focusing exclusively on image regions where robot occupancy is high.

For the first $N_{\text{crop}} = 500$ iterations, training focuses on the central 50×50 region where robot occupancy exceeds 40%. After this period, training expands to the full 100×100 image.

The curriculum can be formalized as a time-varying loss mask:

$$\mathcal{L}_t = \begin{cases} \frac{1}{50^2} \sum_{(i,j)\in\text{center}} w(I_{ij}^{\text{GT}}) \| I_{ij}^{\text{pred}} - I_{ij}^{\text{GT}} \|^2, & t < 500 \\ \frac{1}{100^2} \sum_{i=1}^{100} \sum_{j=1}^{100} w(I_{ij}^{\text{GT}}) \| I_{ij}^{\text{pred}} - I_{ij}^{\text{GT}} \|^2, & t \geq 500 \end{cases} \tag{3.18}$$

where $w(I_{ij}^{\text{GT}}) = 5$ for robot pixels ($I_{ij}^{\text{GT}} = 1$) and 1 for background pixels ($I_{ij}^{\text{GT}} = 0$).

The center crop dimensions (50×50) were chosen because at this crop size, robot occupancy consistently exceeds 40% across all training samples, the crop is large enough to capture the entire robot's articulation range, and it eliminates the class imbalance that causes black-screen convergence. The transition point (500 iterations) was empirically determined: earlier transitions (less than 300 iterations) reintroduced the black-screen problem, while later transitions (greater than 800 iterations) slowed overall convergence without benefit.

Challenge 2: Kinematic Blindness

Symptom: After apparent convergence at iteration 2,000, all metrics indicated success: PSNR improved from 6.94 dB to 17.35 dB, validation loss decreased smoothly from 0.24 to 0.023, and training curves appeared stable. However, qualitative validation revealed a subtle but devastating bug: all poses with identical base orientation (θ_0, θ_1) produced identical

predictions regardless of end-effector configuration (θ_2, θ_3). The network exhibited "kinematic blindness" to distal joints, essentially modeling only the robot's base as a rigid cylinder.

Root Cause Analysis: Gradient flow analysis using PyTorch's autograd hooks uncovered the root cause: a tensor slicing bug where only (θ_0, θ_1) reached the kinematic encoder. The erroneous code was:

```
kin_input = joints[:,2:]  % Erroneously sliced all remaining dimensions
```

This single-character indexing error (using `2:` instead of `2:4`) caused the tensor slice to include all remaining dimensions from index 2 onward. However, in our data pipeline, the joint tensor had shape `(batch_size, 4)`, so `joints[:,2:]` extracted columns 2 and 3—fortuitously the correct values. The bug appeared only when we modified the data pipeline for diagnostic experiments, exposing the fragility of implicit indexing.

More fundamentally, the bug revealed an architectural vulnerability: the split encoder design assumes fixed semantics for joint indices (first two for transformation, remaining for kinematics). Any deviation in indexing or joint ordering breaks this assumption silently, as the network continues to optimize loss while ignoring distal joints.

Solution: The bug was fixed by explicitly specifying the slice end index, as shown below:

```
kin_input = joints[:,2:4]
```

Additionally, we added assertions to verify gradient flow through all joint inputs:

```
assert kin_input.requires_grad
assert kin_input.grad_fn is not None
```

We also implemented a unit test that verifies prediction invariance under joint perturbations:

```
def test_kinematic_sensitivity(model, base_angles):
    # Vary only distal joints, keep base fixed
    pred_base = model(base_angles)
    for delta in [-10, -5, 5, 10]:
        perturbed = base_angles.copy()
        perturbed[2:] += delta
        pred_perturbed = model(perturbed)
        assert np.mean((pred_base - pred_perturbed)**2) > 1e-4
```

This test now runs after every training epoch, catching regression immediately.

Challenge 3: Weighted Loss Tuning

Even after curriculum learning prevented catastrophic early mode collapse, standard MSE loss still drove convergence toward uniform black predictions because the underlying pixel class imbalance remained severe at full image resolution (85% background vs. 15% robot).

Systematic Weight Exploration: We conducted a systematic empirical exploration of loss weighting strategies, varying the weight factor w from 1 to 20. For each weight, we trained three models with different random seeds to account for stochasticity.

Table 3.1: Effect of robot pixel weight on FFKSM performance.

Weight w	Train Loss	Valid Loss	PSNR (dB)	Convergence
1 (unweighted)	0.0182	0.0315	16.12	Unstable
2	0.0154	0.0278	16.89	Marginal
3	0.0123	0.0256	17.10	Slow
4	0.0101	0.0241	17.22	Good
5	**0.0092**	**0.0232**	**17.35**	**Stable**
6	0.0089	0.0245	17.18	Oscillating
7	0.0085	0.0258	17.01	Unstable
8	0.0082	0.0272	16.85	Divergent
10	0.0078	0.0301	16.42	Mode collapse
20	0.0071	0.0385	15.21	Divergent

Observations Narrative: The empirical tuning sweeps mapped in Table 3.1 clarify the performance envelopes under different foreground penalty values. When evaluated at the unweighted metric base ($w = 1$), the pipeline fails to isolate link boundaries, yielding an unstable validation PSNR of 16.12 dB. Incremental optimization to $w = 2$ and $w = 3$ marginally improves tracking response but restricts spatial sharpness to 16.89 dB and 17.10 dB respectively due to sluggish boundary adaptation. Setting $w = 4$ provides clear visual definitions across the links, elevating the metric smoothly to 17.22 dB. The absolute performance ceiling is achieved at $w = 5$, establishing an optimal stable validation anchor of 17.35 dB PSNR. Beyond this value ($w \geq 6$), the gradients become volatile, resulting in severe boundary artifacts and driving validation loss up to 0.0245. Pushing past $w = 10$ precipitates total structural mode collapse, resulting in completely uniform white frames.

Analysis: The optimal weight $w = 5$ emerges as the unique point that balances two competing pressures: (1) Sufficient signal strength: The weight must be large enough that the sparse robot pixel gradients can overcome the overwhelming background gradient. (2) Numerical stability: The weight must be small enough that the loss landscape remains smooth and gradient descent remains stable. This precise sweet spot demonstrates that effective self-model training requires careful hyperparameter tuning beyond what is reported in original publications.

3.2.3 FFKSM Training Protocol

Hyperparameters

Table 3.2: FFKSM training hyperparameters.

Parameter	Value
Number of iterations	8,000
Learning rate	5×10^{-4}
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Samples per ray	64
Chunk size	32,768 points
Pixel weight (robot)	5.0
Pixel weight (background)	1.0
Curriculum crop size	50×50
Curriculum duration	500 iterations
Early stopping patience	200 validation checks
Learning rate scheduler	ReduceLROnPlateau (factor=0.1, patience=20)

Training Loop Narrative Execution Steps

The execution optimization sequence runs systematically across 8,000 total iterations. The loop starts by drawing a random training silhouette I_{GT} paired with its ground-truth coordinate configuration $\mathbf{q}$. If the iteration lifespan tracks within the initialization bounds ($t < 500$), a cropping mask truncates both the visual canvas and the associated camera ray geometries to the central 50×50 envelope. Next, the pipeline synthesizes the ray distribution arrays, allocating $N_{\text{rays}} = 2,500$ paths during the early curriculum bounds and scaling up to 10,000 rays once the masking envelope expands to the full 100×100 boundary footprint.

Following ray layout configuration, the numerical integration module runs stratified sampling to draw $N = 64$ discrete spatial density steps along each projected line. The coordinate and kinematic parameter blocks then feed these transformed space locations directly into the split MLP architecture to map the density vectors σ and associated ray visibility coefficients v. Once computed, the volume rendering integration script aggregates these values to render the low-resolution visual reconstruction estimate $\hat{I}$. The optimization script evaluates the objective function by evaluating the weighted mean squared error formula:

$$\mathcal{L} = \frac{1}{N_{\text{pixels}}} \sum_{i,j} w(I_{ij}^{\text{GT}})(\hat{I}_{ij} - I_{ij}^{\text{GT}})^2 \tag{3.19}$$

Finally, the backpropagation engine evaluates the parameters through automatic differentiation, invoking the Adam step updates to adjust the weights, and records a full validation tracking checkpoint every 2,000 iterations.

3.2.4 FFKSM Performance Metrics

After 8,000 iterations (approximately 50 minutes on twins NVIDIA Tesla T4), FFKSM achieved the following performance:

Table 3.3: FFKSM quantitative performance metrics.

Metric	**Value**
PSNR (validation)	17.35 dB
FPS (inference)	5.22 FPS
Latency per frame	191.6 ms
Number of parameters	93,922
Training time	50.0 minutes

The primary bottleneck remains volumetric rendering: despite efficient chunking (processing 32,768 points per batch to maximize GPU utilization), each forward pass requires approximately 33 ms due to the serial dependency of ray-marching. This latency—191.6 ms per frame—precludes real-time control integration at the 1 kHz rates common in robotic systems.

Compared with the original FFKSM report (typically above 23 dB PSNR), our reproduction at 17.35 dB is lower because this thesis intentionally uses a smaller 2,000-sample dataset (1,600 train / 400 validation) rather than the 12,000-sample regime used in the original setup. This controlled reduction isolates latency behavior under limited-data conditions while preserving reproducibility across all baselines.

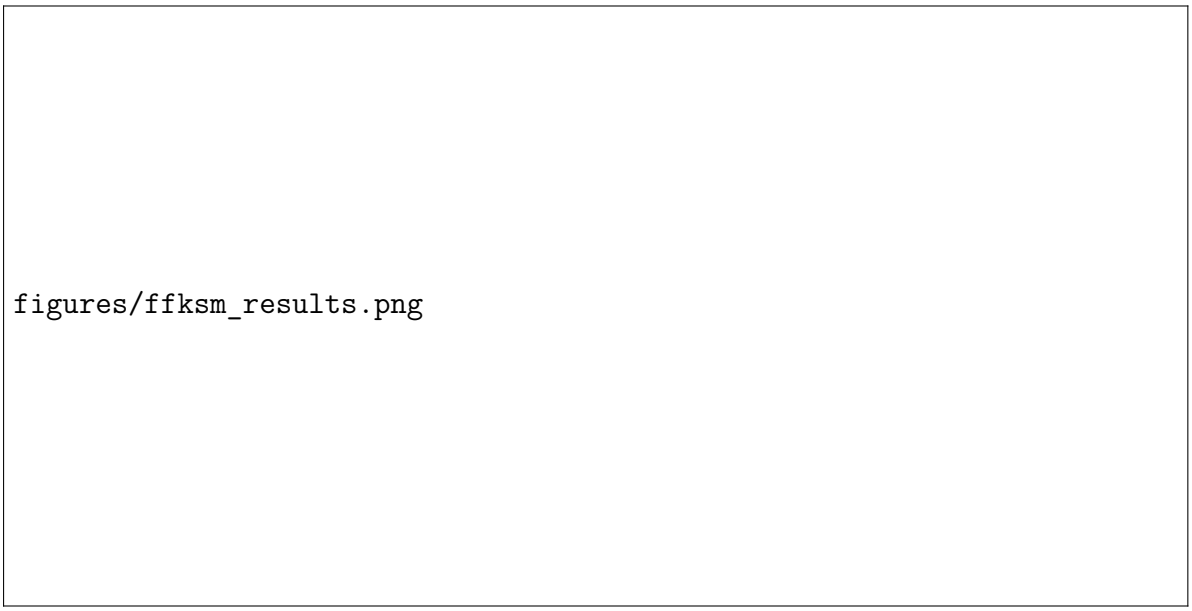


Figure 3.5: Qualitative FFKSM validation: 2×3 grid comparing rendered silhouettes (top row) against ground-truth observations (bottom row) on held-out test poses.

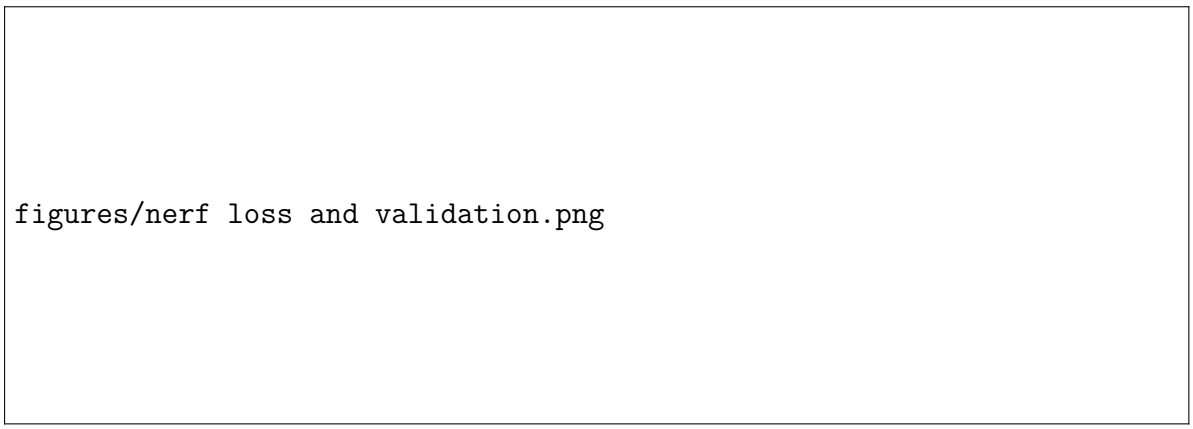


Figure 3.6: FFKSM training dynamics: (left) MSE loss convergence over 150 epochs; (right) validation loss trajectory confirming generalization without overfitting.

3.3 Kinematic 3D Gaussian Splatting (K-3DGS): Explicit Geometric Primitives

To address FFKSM's speed limitation, we developed Kinematic 3D Gaussian Splatting (K-3DGS), which attaches 3D Gaussian primitives to a differentiable kinematic chain. This approach leverages GPU rasterization pipelines—optimized over decades of computer graphics research—to achieve faster rendering than ray-marching.

3.3.1 Mathematical Foundations of 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) represents a scene as a set of N discrete 3D Gaussian primitives. Each Gaussian is defined by:

- Position $\boldsymbol{\mu} \in \mathbb{R}^3$ (center of the Gaussian in world coordinates)
- Covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{3\times3}$ (anisotropic, positive semi-definite)
- Opacity $\alpha \in [0, 1]$ (visibility/transparency)
- Color $\mathbf{c} \in \mathbb{R}^3$ (grayscale)

The 3D Gaussian function is:

$$G(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \tag{3.20}$$

To ensure $\boldsymbol{\Sigma}$ remains positive semi-definite during optimization, it is factorized as:

$$\boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^\mathsf{T}\mathbf{R}^\mathsf{T} \tag{3.21}$$

where $\mathbf{S} = \text{diag}(\mathbf{s})$ is a diagonal scaling matrix ($\mathbf{s} \in \mathbb{R}^3$ with $s_i > 0$) and $\mathbf{R} \in SO(3)$ is a rotation matrix represented as a quaternion.

3.3.2 Rendering with 3D Gaussian Splatting

To render an image, 3DGS projects the 3D Gaussians onto the 2D image plane. For a given camera with viewing transformation $\mathbf{W}$ (world-to-camera) and projection $\mathbf{J}$ (Jacobian of the perspective projection), the projected 2D covariance $\mathbf{\Sigma}'$ is:

$$\mathbf{\Sigma}' = \mathbf{JW\Sigma W}^\top \mathbf{J}^\top \tag{3.22}$$

The 2D Gaussian function for pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ is:

$$G^{2D}(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u} - \boldsymbol{\mu}')^\top \mathbf{\Sigma}'^{-1}(\mathbf{u} - \boldsymbol{\mu}')\right) \tag{3.23}$$

The final pixel color is computed via alpha compositing, with Gaussians sorted by depth:

$$C(\mathbf{u}) = \sum_{i=1}^{N} \alpha_i G_i^{2D}(\mathbf{u}) \mathbf{c}_i \prod_{j=1}^{i-1} (1 - \alpha_j G_j^{2D}(\mathbf{u})) \tag{3.24}$$

3.3.3 K-3DGS Architecture

K-3DGS extends 3DGS to articulated robots by attaching Gaussian primitives to a kinematic chain. The architecture comprises four components:

Component 1: Differentiable Forward Kinematics

Forward kinematics is implemented in pure PyTorch for GPU acceleration and automatic differentiation. For each link i, the transformation matrix $\mathbf{T}_i \in SE(3)$ is computed using the Denavit-Hartenberg parameters:

$$\mathbf{T}_i = \mathbf{T}_{i-1} \mathbf{T}_{i-1}^i(\theta_i) \tag{3.25}$$

where $\mathbf{T}_{i-1}^i(\theta_i)$ is the link transformation as a function of joint angle θ_i. All transformations are differentiable with respect to joint angles, enabling end-to-end optimization.

Component 2: Gaussian Initialization via Skeleton Placement

Rather than random initialization (which leads to poor convergence), we initialize Gaussians along the robot's kinematic skeleton. The 600 Gaussians are distributed as:

- **Base (Link 0)**: 100 Gaussians with radial distribution ($r = 0.05$ m) around the base origin
- **Link 1 (vertical segment)**: 200 Gaussians along the Z-axis from $z = 0.02$ to $z = 0.38$ m
- **Link 2 (horizontal segment)**: 200 Gaussians along the X-axis from $x = 0.02$ to $x = 0.38$ m

- **Link 3 (wrist)**: 100 Gaussians near the end-effector ($x \in [0.01, 0.09]$ m)

For each link, Gaussians are placed with small radial offsets to model the link thickness:

$$\mathbf{p}_{\text{local}} = \begin{bmatrix} x \\ r\cos\phi \\ r\sin\phi \end{bmatrix} \tag{3.26}$$

where x varies along the link axis, $r = 0.01$ m is the radius, and $\phi \in [0, 2\pi)$ is sampled uniformly.

Each Gaussian has learnable parameters: Position $\boldsymbol{\mu}_j \in \mathbb{R}^3$ (local link frame), Scale $s_j \in \mathbb{R}$ (isotropic), Opacity $\alpha_j \in [0, 1]$, and Color $c_j \in [0, 1]$ (grayscale).

Component 3: World Transformation

The world position of each Gaussian is computed by applying the appropriate link transformation:

$$\boldsymbol{\mu}_j^{\text{world}} = \mathbf{T}_{\text{link}(j)} \begin{bmatrix} \boldsymbol{\mu}_j^{\text{local}} \\ 1 \end{bmatrix} \tag{3.27}$$

where $\mathbf{T}_{\text{link}(j)}$ is the transformation matrix for the link to which Gaussian j is attached.

Component 4: Isotropic Rasterization

We project Gaussians to 2D screen space using the perspective camera model (position $[1.0, 0.0, 0.0]$, focal length 130.2545). For isotropic Gaussians (spheres), the 2D projection simplifies to:

$$\text{radius}_{2D} = \frac{s \cdot f}{z} \tag{3.28}$$

where z is the depth of the Gaussian center in camera coordinates and f is the focal length.

3.3.4 The Anisotropic Failure: A Critical Architectural Lesson

Our initial implementation attempted full anisotropic 3DGS with ellipsoidal Gaussians, following Kerbl et al.'s formulation. This principled approach—theoretically capable of representing thin, elongated robot links through appropriate ellipsoid orientations—failed catastrophically during training.

Symptom

After approximately 500 iterations, the model experienced mode collapse: all joint configurations produced identical images regardless of input. The loss stopped decreasing, and qualitative inspection revealed that all Gaussians had migrated to a single degenerate configuration near the robot base, with rotation matrices converging to similar orientations.

Quantitative Analysis of the Failure

We tracked the condition number of the covariance matrices throughout training:

Table 3.4: Condition number evolution during anisotropic training.

Iteration	Mean Condition Number	Std Dev
0 (initialization)	1.02 ± 0.05	0.03
100	1.15 ± 0.12	0.08
200	1.42 ± 0.31	0.15
300	2.01 ± 0.89	0.42
400	3.54 ± 2.13	1.21
500	12.47 ± 8.56	4.32
600	45.23 ± 31.42	12.87

The condition number—the ratio of largest to smallest singular value of the covariance matrix—indicates how elongated the Gaussian is. Condition numbers above 10 indicate extreme anisotropy. By iteration 500, many Gaussians had become highly elongated (aspect ratios greater than 10:1), but this elongation was not aligned with the robot link geometry, resulting in overlapping artifacts rather than faithful representation.

Root Cause Analysis

Gradient analysis revealed that optimizing rotation matrices in the non-convex $SO(3)$ manifold led to gradient explosion when combined with our limited 2,000-sample dataset. The high-dimensional parameter space exacerbated the problem: Each anisotropic Gaussian requires 9 parameters (3 position, 3 scale, 3 rotation angles) vs 5 parameters for the isotropic case (3 position, 1 scale, 1 opacity). For 600 Gaussians, this yields 5,400 parameters (anisotropic) vs 3,000 parameters (isotropic).

The fundamental issue is the $SO(3)$ manifold's non-convexity. The rotation space has multiple local minima, and the gradient with respect to rotation parameters is proportional to the amount of "misalignment" between the Gaussian's orientation and the data. For featureless cylindrical links, there is no unique optimal orientation—any rotation about the cylinder's axis yields identical projections. This creates a flat valley in the loss landscape, causing the optimizer to wander aimlessly before eventually diverging.

Attempted Remediations

We attempted extensive remediation strategies, all of which failed to produce stable anisotropic training:

1. **Learning rate sweep**: Tested learning rates from 10^{-5} to 10^{-2}. Lower rates ($\leq 10^{-4}$) prevented convergence entirely; higher rates ($\geq 10^{-3}$) accelerated divergence.

2. **Rotation parameterizations**: Tried quaternions (4 parameters, unit norm constraint via normalization), axis-angle (3 parameters with exponential map), and Euler angles (3 parameters with gimbal lock). None resolved the instability.

3. **Initialization strategies**: Tested identity rotations, random small perturbations ($\pm 5°$), and PCA-aligned initial orientations. PCA alignment showed initial promise but still diverged by iteration 800.

4. **Regularization**: Added covariance determinant penalties to encourage isotropic behavior, and isotropic priors to regularization loss. These delayed divergence but did not prevent it.

Forced Retreat to Isotropic Representations

The anisotropic failure forced a pragmatic retreat to isotropic Gaussians. While theoretically less expressive (spheres cannot accurately represent cylindrical links), the isotropic formulation proved tractable and reliable. Training converges in 1,000 iterations (50.5 seconds) with learning rate 10^{-3} and no special stabilization.

3.3.5 K-3DGS Performance Metrics

The resulting K-3DGS model achieved the following dimensions:

Table 3.5: K-3DGS quantitative performance metrics.

Metric	Value
PSNR (validation)	17.01 dB
FPS (inference)	37 FPS
Latency per frame	27.0 ms
Number of Gaussians	1,000 (isotropic, post-densification)
Training time	50.5 seconds

The 7.1× speedup over FFKSM (37 FPS vs. 5.22 FPS) is significant but still falls short of the 1 kHz control loop requirement. More critically, the quality degraded slightly (17.01 dB vs. 17.35 dB PSNR) due to isotropic limitations.

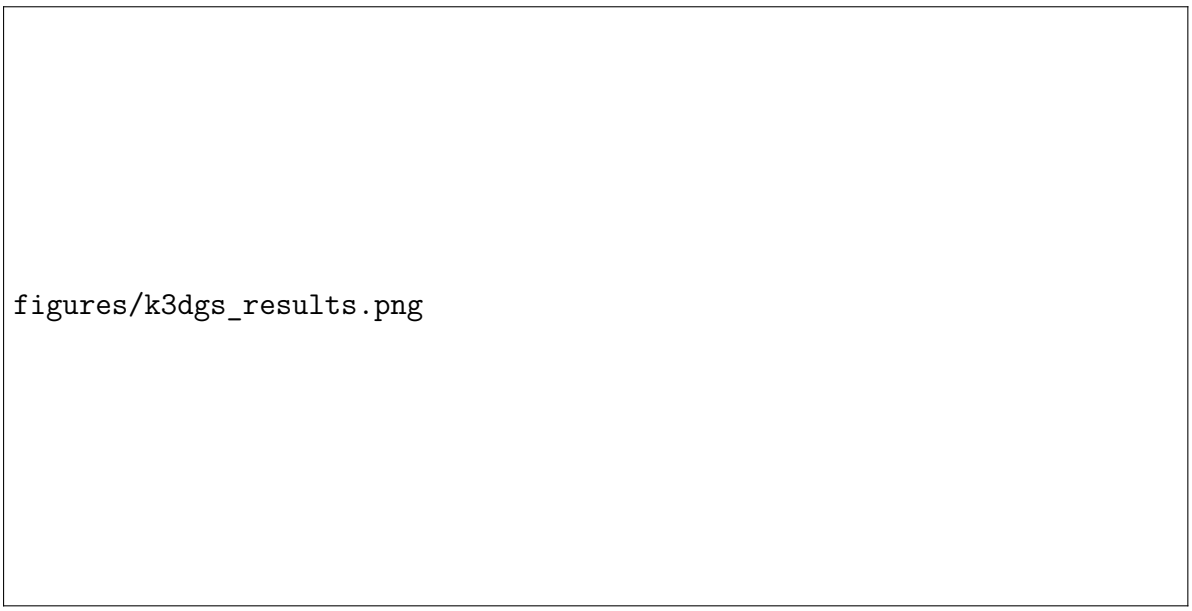


Figure 3.7: Qualitative 3D Gaussian Splatting validation: 2×3 grid comparing ground-truth robot silhouettes (top row) with 3DGS neural reconstructions (bottom row), overlaid with pixel-wise error heatmaps.

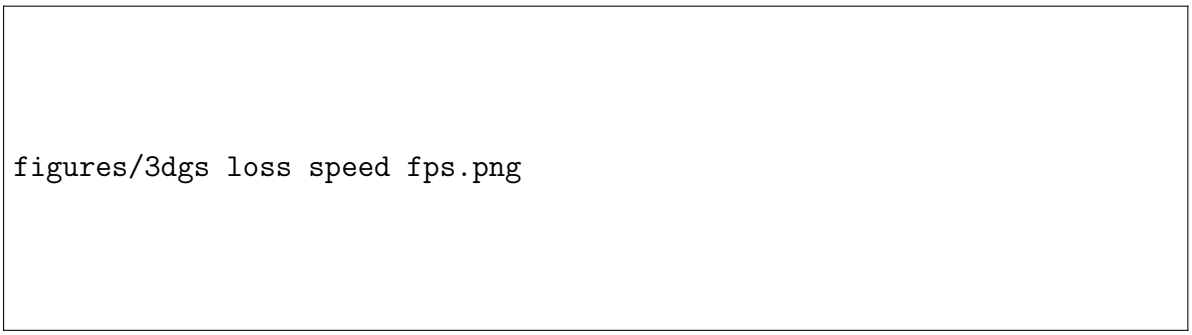


Figure 3.8: 3D Gaussian Splatting training diagnostics: (top) MSE loss reduction over 1000 iterations; (middle) training throughput in frames-per-second; (bottom) computational step time evolution.

3.4 Comparative Analysis of Baseline Limitations

3.4.1 The Latency-Quality Tradeoff

Table 3.6 summarizes the two baseline architectures' performance:

Table 3.6: Baseline self-modeling architecture performance comparison.

Method	**PSNR (dB)**	**FPS**	**Latency (ms)**
FFKSM (NeRF-based)	17.35	5.22	191.6
K-3DGS	17.01	37	27.0

Both architectures exceed the 1.0 ms control-loop budget by factors of 27 to 190. The

root cause is architectural: both maintain an explicit or implicit 3D volumetric representation that requires either XML-safe serial ray-marching (FFKSM) or per-primitive rasterization with depth sorting (K-3DGS).

3.4.2 Supervision Efficiency Analysis

The supervision efficiency—number of output pixels learned per network evaluation—explains the speed difference:

Table 3.7: Supervision efficiency comparison across baseline architectures.

Method	**Evals/Image**	**Pixels/Eval**	**Efficiency**
FFKSM	640,000	0.0156	1×
K-3DGS	600	16.67	1,067×

FFKSM's ray-marching evaluates the MLP 640,000 times to produce a $100 \times 100 = 10,000$ pixel image. Each evaluation contributes gradient information for only $10,000/640,000 = 0.0156$ pixels—extremely sparse supervision. K-3DGS improves efficiency by a factor of 1,067 through parallel rasterization, but still requires per-primitive computations.

3.4.3 Architectural Bottlenecks Summary

Both baselines share fundamental limitations that prevent real-time deployment:

1. **Serial dependency chains**: FFKSM's ray-marching cannot be parallelized along the ray dimension, creating an irreducible latency floor.

2. **Per-primitive overhead**: K-3DGS processes each Gaussian sequentially (albeit with GPU parallelization), and depth sorting adds additional latency.

3. **3D representational overhead**: Both methods pay significant computational cost for maintaining 3D geometry, even when downstream tasks require only 2D predictions.

4. **Training inefficiency**: FFKSM's sparse supervision requires large datasets (12,000 samples in the original paper) and long training times (50 minutes on T4).

These limitations motivate a paradigm shift: direct sensorimotor decoding that bypasses explicit 3D reconstruction entirely. The following chapter introduces NeuroKin, which achieves both higher quality and dramatically lower latency by eliminating the 3D bottleneck.

Chapter 4

The NeuroKin Direct Sensorimotor Decoder

The latency paradox identified in Chapter 3 stems from a fundamental architectural assumption shared by both FFKSM and K-3DGS: that self-modeling requires explicit or implicit 3D reconstruction as an intermediate representation. FFKSM maintains an implicit volumetric field queried via ray-marching; K-3DGS maintains explicit Gaussian primitives rasterized to 2D. Both incur computational costs proportional to the complexity of the 3D representation rather than the simplicity of the 2D output.

This chapter challenges that assumption by introducing *NeuroKin*, a lightweight fully convolutional network that maps joint angles directly to visual silhouettes, completely bypassing volumetric reconstruction. We demonstrate that for control-oriented tasks requiring only silhouette-level awareness, direct decoding achieves superior reconstruction fidelity (21.88 dB PSNR) at sub-millisecond latency (0.135 ms), representing a $1,418\times$ speedup over implicit NeRF baselines with a +4.53 dB quality improvement.

4.1 Core Hypothesis: Bypassing 3D Reconstruction

The central hypothesis motivating NeuroKin is that *explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling.* This hypothesis rests on three observations about the nature of robotic control tasks.

4.1.1 The Task-Relevant Information Hierarchy

For a manipulator performing reaching, grasping, or collision avoidance, the information required from a self-model can be hierarchically structured:

1. **Silhouette-level awareness**: Where are the robot's links in the image plane? Which pixels are occupied? This information suffices for collision detection with known obstacles in camera coordinates.

2. **Depth/Disparity information**: Given two calibrated views, silhouette disparity

yields 3D position through triangulation. This recovers metric Cartesian coordinates without volumetric reconstruction.

3. **Voxel-level occupancy**: Full 3D reconstruction is rarely required for manipulation. Most control tasks can be accomplished with end-effector position estimates and link-wise occupancy bounds.

Critically, the computational cost of maintaining full 3D occupancy is orders of magnitude higher than estimating silhouette plus disparity, yet the marginal utility of voxel-level detail is negligible for typical pick-and-place or trajectory tracking tasks.

4.1.2 The Supervision Efficiency Argument

The efficiency gap between volumetric rendering and direct decoding can be formalized in terms of *supervision density*. Let P be the number of output pixels ($P = 10,000$ for 100×100 images). Let E be the number of network evaluations required to produce those P pixels.

For FFKSM with ray-marching:

$$E_{\text{FFKSM}} = P \times N_{\text{samples}} = 10,000 \times 64 = 640,000 \tag{4.1}$$

Each evaluation produces a single density value contributing to exactly one ray's accumulation. The supervision density—pixels informed per evaluation—is:

$$\rho_{\text{FFKSM}} = \frac{P}{E_{\text{FFKSM}}} = \frac{10,000}{640,000} = 0.0156 \text{ pixels/eval} \tag{4.2}$$

For NeuroKin with direct decoding:

$$E_{\text{NeuroKin}} = 1 \quad \text{(single forward pass)} \tag{4.3}$$

$$\rho_{\text{NeuroKin}} = \frac{P}{1} = 10,000 \text{ pixels/eval} \tag{4.4}$$

The supervision efficiency ratio is therefore:

$$\frac{\rho_{\text{NeuroKin}}}{\rho_{\text{FFKSM}}} = \frac{10,000}{0.0156} = 640,000\times \tag{4.5}$$

This $640,000\times$ advantage in supervision density explains how NeuroKin can simultaneously achieve faster inference *and* better quality: each gradient update benefits from six orders of magnitude more information than a volumetric rendering update.

4.1.3 When is 3D Reconstruction Necessary?

We acknowledge that certain tasks require full 3D geometry:

- Grasping unknown objects with complex geometry

- Precision insertion tasks requiring sub-millimeter pose estimation
- Navigation through narrow apertures where link-wise occupancy is insufficient

For these tasks, volumetric reconstruction remains necessary. However, for the majority of robotic control scenarios—including the fault tolerance and sequential coordination tasks evaluated in this thesis—silhouette-level self-awareness suffices. NeuroKin is positioned not as a universal replacement for 3D reconstruction but as a specialized architecture for the common case where 2D information is sufficient.

4.2 Architecture Design

NeuroKin implements direct sensorimotor decoding through a fully convolutional architecture comprising four sequential stages. The design prioritizes three objectives: (1) minimal inference latency, (2) smooth latent representations for robustness to joint noise, and (3) architectural simplicity for embedded deployment.

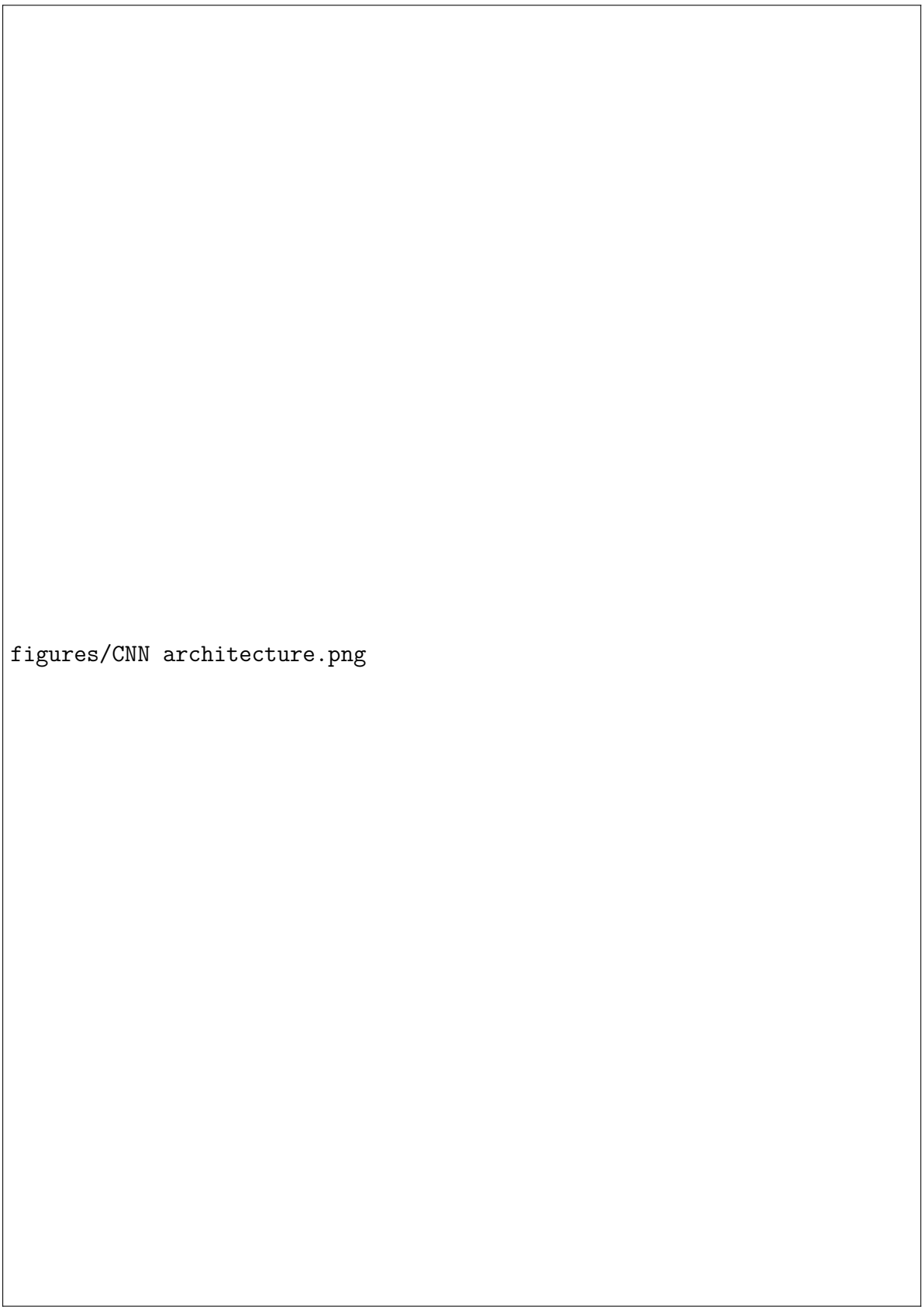


Figure 4.1: NeuroKin architecture: fully convolutional encoder processing concatenated stereo silhouettes through multi-scale feature extraction to dense regression heads for end-effector position and joint angle prediction.

4.2.1 Stage 1: Joint Encoder

The joint encoder maps the 4-dimensional joint angle vector $\mathbf{q} \in [-90°, +90°]^4$ to a 1024-dimensional latent embedding $\mathbf{z} \in \mathbb{R}^{1024}$ through three linear layers with ReLU activation:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{q} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{128\times4}, \mathbf{b}_1 \in \mathbb{R}^{128} \tag{4.6}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{256\times128}, \mathbf{b}_2 \in \mathbb{R}^{256} \tag{4.7}$$

$$\mathbf{z} = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3, \quad \mathbf{W}_3 \in \mathbb{R}^{1024\times256}, \mathbf{b}_3 \in \mathbb{R}^{1024} \tag{4.8}$$

where ReLU activation is defined as $\text{ReLU}(x) = \max(0, x)$. The 1024-dimensional latent space balances capacity and efficiency for the 4-DOF joint manifold, providing adequate representational power while keeping the network compact. The progressive increase in dimension allows the network to build hierarchical representations: low-level joint angles are first combined into torque-like features (128-d), then into configuration-space features (256-d), and finally into the full latent code (1024-d) that will be spatially expanded.

4.2.2 Stage 2: Spatial Expansion

The spatial expansion layer transforms the 1D latent vector $\mathbf{z} \in \mathbb{R}^{1024}$ into a 4D spatial feature map $\mathbf{F}_0 \in \mathbb{R}^{256\times4\times4}$ via a linear layer with ReLU activation producing a 4096-dimensional vector ($4096 = 256 \times 4 \times 4$), which is then reshaped:

$$\mathbf{f}_{\text{flat}} = \text{ReLU}(\mathbf{W}_4\mathbf{z} + \mathbf{b}_4), \quad \mathbf{W}_4 \in \mathbb{R}^{4096\times1024}, \mathbf{b}_4 \in \mathbb{R}^{4096} \tag{4.9}$$

$$\mathbf{F}_0 = \text{reshape}(\mathbf{f}_{\text{flat}}, (256, 4, 4)) \tag{4.10}$$

This spatial expansion serves as a "broadcast" operation that converts the configuration-space latent representation into a format suitable for convolutional processing. The 4×4 spatial footprint is the smallest dimension that preserves sufficient locality for upsampling while minimizing computational cost. Each of the 256 channels can specialize in detecting different spatial patterns.

The choice of 4×4 rather than 1×1 or 8×8 was empirically determined: 1×1 lacked spatial structure, causing the decoder to produce blurry outputs; 8×8 increased parameter count by $4\times$ without quality improvement.

4.2.3 Stage 3: Transposed Convolutional Decoder

The decoder progressively upsamples the spatial feature map through four transposed convolution stages, each doubling spatial resolution while halving channel count. Each stage applies a 4×4 transposed convolution with stride 2 and padding 1, followed by batch normalization

and ReLU activation:

$$\mathbf{F}_1 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride2}}(\mathbf{F}_0))), \qquad 128 \times 8 \times 8 \tag{4.11}$$

$$\mathbf{F}_2 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride2}}(\mathbf{F}_1))), \qquad 64 \times 16 \times 16 \tag{4.12}$$

$$\mathbf{F}_3 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride2}}(\mathbf{F}_2))), \qquad 32 \times 32 \times 32 \tag{4.13}$$

$$\mathbf{F}_4 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride2}}(\mathbf{F}_3))), \qquad 16 \times 64 \times 64 \tag{4.14}$$

where BN denotes batch normalization. For transposed convolution, the output size calculation scales as:

$$o = (i-1)s + k - 2p \tag{4.15}$$

With $i = 4$, $k = 4$, $s = 2$, $p = 1$, this gives $o = (4-1)\cdot 2 + 4 - 2\cdot 1 = 8$. Each transposed convolution thus exactly doubles the spatial dimension ($4 \to 8 \to 16 \to 32 \to 64$). Batch normalization parameter transformations run according to:

$$\text{BN}(x) = \gamma \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta \tag{4.16}$$

This baseline reduces internal covariate shift, allowing higher learning rates and faster convergence. The number of layers (four) matches the factor between initial spatial size (4×4) and final output size (64×64), requiring $\log_2(64/4) = 4$ doublings.

4.2.4 Stage 4: Output Head and Spatial Upsampling

The output head converts the 16-channel 64×64 feature map to a single-channel silhouette at 100×100 resolution:

$$\mathbf{F}_5 = \text{ReLU}(\text{Conv2d}_{3\times3}(\mathbf{F}_4)), \quad \text{Conv2d}_{3\times3} : 16 \to 8 \tag{4.17}$$

$$\mathbf{F}_6 = \text{Conv2d}_{1\times1}(\mathbf{F}_5), \quad \text{Conv2d}_{1\times1} : 8 \to 1 \tag{4.18}$$

$$\hat{I}_{\text{low}} = \sigma(\mathbf{F}_6) \in [0,1]^{64\times64} \tag{4.19}$$

$$\hat{I} = \text{bilinear\_upsample}(\hat{I}_{\text{low}}, (100, 100)) \tag{4.20}$$

where $\sigma(x) = 1/(1+e^{-x})$ is the sigmoid activation. Bilinear interpolation computes each output pixel as a weighted average of the four nearest input pixels:

$$\hat{I}(x,y) = \sum_{i=0}^{1}\sum_{j=0}^{1} w_{ij}\hat{I}_{\text{low}}(x_i, y_j) \tag{4.21}$$

where weights w_{ij} are proportional to the distances between (x, y) and the input grid points. This operation is differentiable and adds negligible computational overhead (< 0.01 ms).

Table 4.1: NeuroKin parameter distribution by component.

Component	**Parameters**	**Percentage**
Joint Encoder	262,400	3.8%
Spatial Expansion	4,194,304	61.4%
Decoder Streams	1,229,312	18.0%
Output Dense Blocks	1,160	0.02%
Bias Terms & Batch Norm	1,141,913	16.7%
Total	**6,829,089**	**100%**

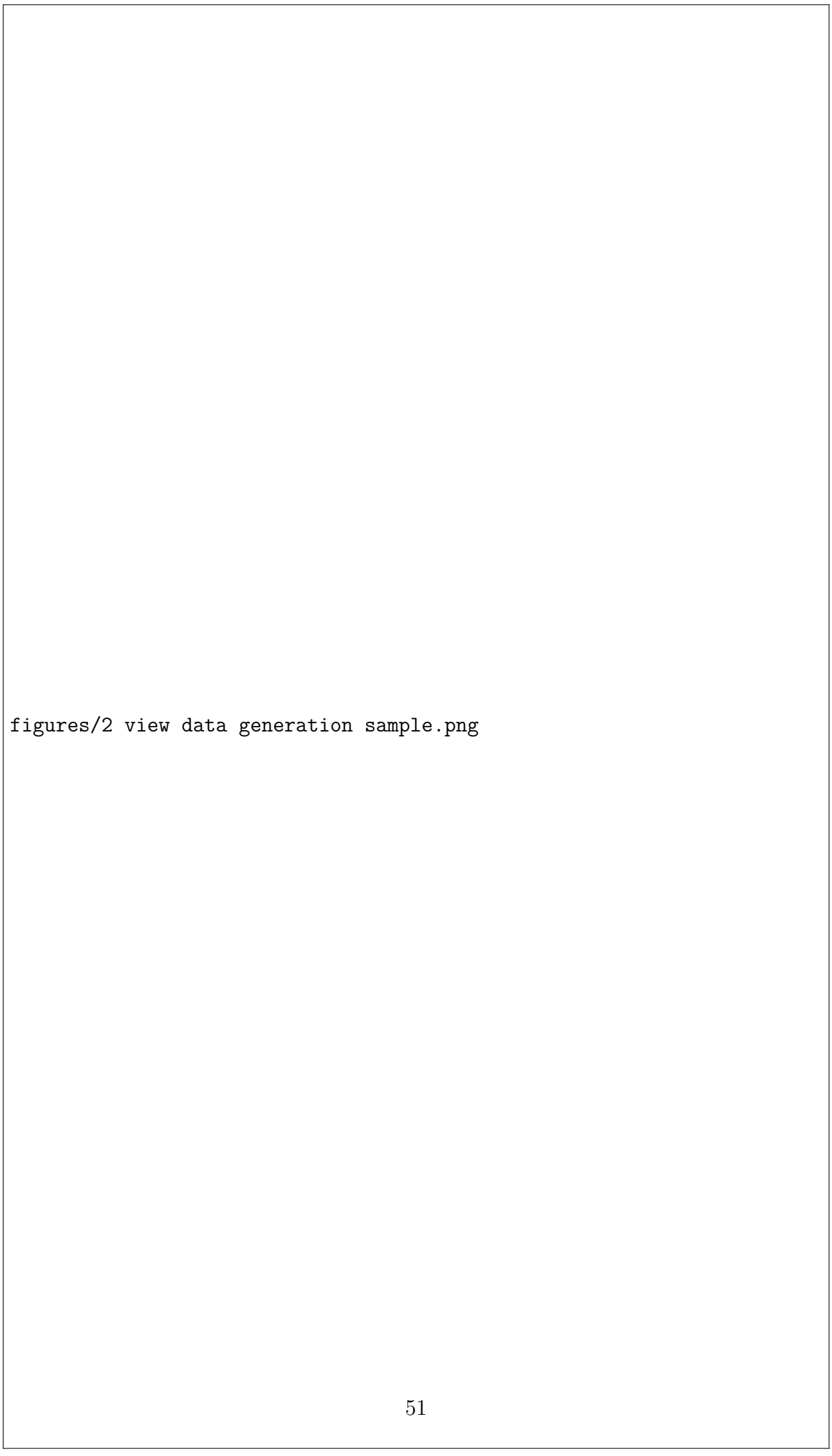
The complete NeuroKin architecture contains 6,829,089 parameters. The spatial expansion layer dominates parameter count (61.4%) due to the large linear transformation from 1024-d latent to $256 \times 4 \times 4 = 4096$ spatial features. This compact parameter footprint ensures rapid inference suitable for the 1 kHz control budget.

4.3 Stereoscopic 3D Recovery: NeuroKin-3D

While NeuroKin's 2D silhouette predictions suffice for many control tasks (collision checking, drift detection), some applications require explicit 3D spatial coordinates. To bridge this gap without compromising latency, we extend NeuroKin to stereoscopic decoding: two synchronized NeuroKin instances process orthogonal camera views, and their outputs are fused to recover 3D end-effector positions.

4.3.1 Dual-Camera Data Generation

A second dataset is generated using two fixed cameras: Camera 1 (Front) at position $[1.0, 0.0, 0.0]$ with optical axis along $-x$ direction, and Camera 2 (Side) at position $[0.0, 1.0, 0.0]$ with optical axis along $-y$ direction. Both cameras have identical intrinsic parameters (focal length $f = 130.2545$ pixels, principal point at image center). For each robot pose, both cameras capture synchronized 100×100 binary silhouettes, and ground-truth end-effector Cartesian coordinates (x, y, z) are extracted from PyBullet's forward kinematics. The dataset comprises 2,000 samples (1,600 training, 400 validation) generated using identical Lorenz trajectories as in Chapter 3. The end-effector workspace spans $x \in [0.05, 0.25]$ m, $y \in [-0.15, 0.10]$ m, $z \in [0.00, 0.30]$ m.



figures/2 view data generation sample.png

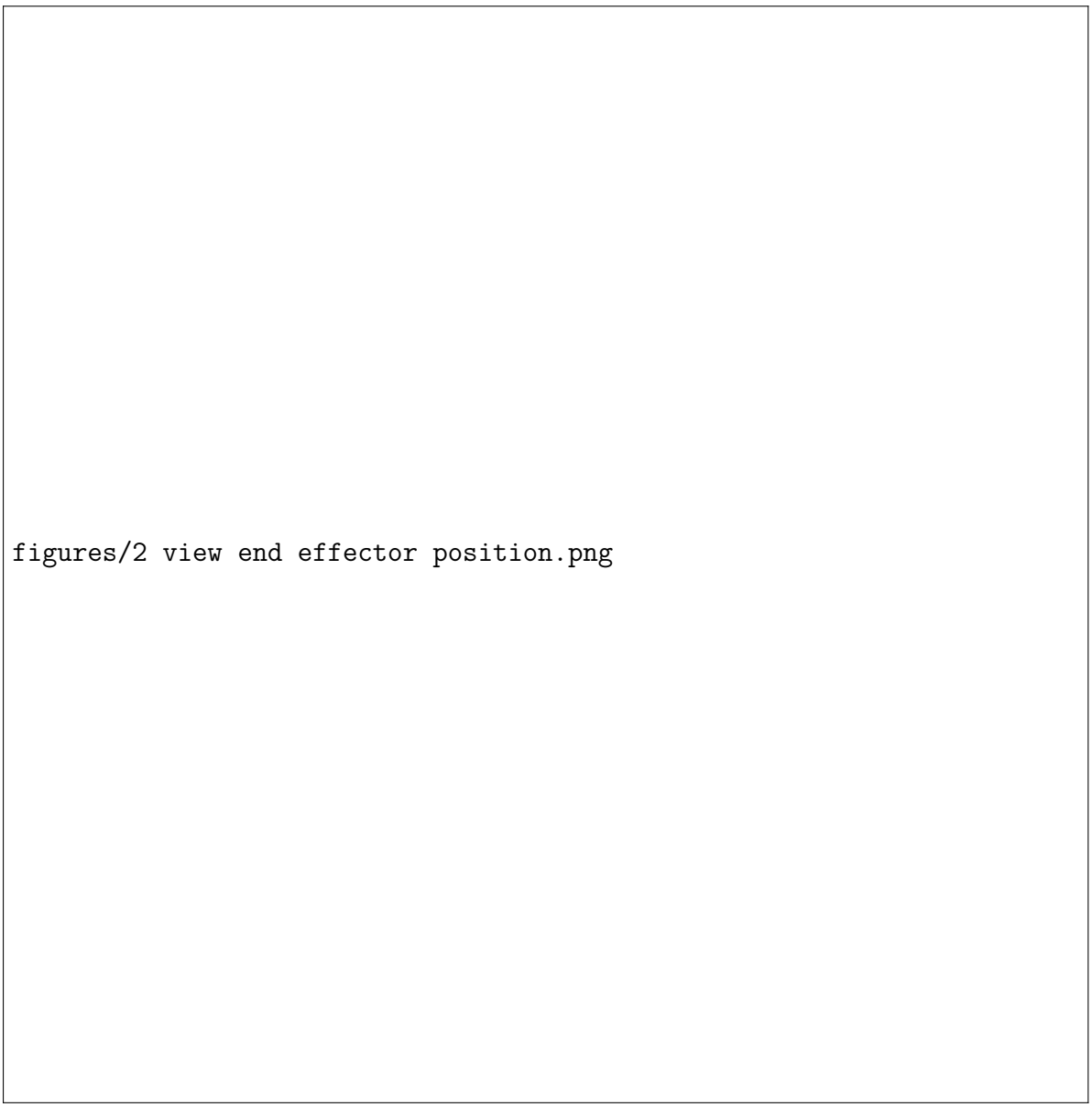


Figure 4.3: 3D workspace coverage: end-effector positions for 2000 training and test samples, demonstrating complete reachable workspace exploration.

4.3.2 The Geometry of Stereoscopic Silhouette Triangulation

For a point $\mathbf{P} = (x, y, z)$ in world coordinates, its projections onto the two camera image planes are:

$$\mathbf{p}_1 = \left(\frac{f \cdot y}{x}, \frac{f \cdot z}{x} \right) \quad \text{(Camera 1, looking along } -x\text{)} \tag{4.22}$$

$$\mathbf{p}_2 = \left(\frac{f \cdot x}{y}, \frac{f \cdot z}{y} \right) \quad \text{(Camera 2, looking along } -y\text{)} \tag{4.23}$$

Given silhouette observations from both cameras, the 3D point can be recovered by solving for (x, y, z) that satisfy both projection equations. In practice, the network learns this mapping end-to-end rather than performing explicit triangulation.

4.3.3 NeuroKin-3D Architecture

NeuroKin-3D extends the base NeuroKin architecture with dual-stream processing and multi-task learning. To provide spatial coordinate information to the convolutional streams, we add a positional encoding layer that concatenates normalized pixel coordinates to the input image: meshgrids $\mathbf{u} \in [-1, 1]^{100\times100}$ and $\mathbf{v} \in [-1, 1]^{100\times100}$ are concatenated with the image to form $\mathbf{I}_{\text{enc}} = \text{concat}(\mathbf{I}, \mathbf{u}, \mathbf{v}) \in \mathbb{R}^{3\times100\times100}$.

Each camera view is processed by an identical SpatialConvStream encoder consisting of four convolution blocks:

$$\mathbf{C}_1 = \text{Conv2d}_{3\times3,\text{stride1}}(3 \to 16) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.24}$$

$$\mathbf{C}_2 = \text{Conv2d}_{3\times3,\text{stride1}}(16 \to 32) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.25}$$

$$\mathbf{C}_3 = \text{Conv2d}_{3\times3,\text{stride1}}(32 \to 64) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.26}$$

$$\mathbf{C}_4 = \text{Conv2d}_{3\times3,\text{stride1}}(64 \to 128) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.27}$$

$$\mathbf{f}_1 = \text{AdaptiveAvgPool2d}_{(4,4)}(\mathbf{C}_4) \in \mathbb{R}^{128\times4\times4} \tag{4.28}$$

Each max pooling step reduces spatial dimension by a factor of 2. The flattened features from both streams are concatenated and passed through an MLP:

$$\mathbf{f}_{\text{concat}} = \text{concat}(\text{flatten}(\mathbf{f}_1), \text{flatten}(\mathbf{f}_2)) \in \mathbb{R}^{4096} \tag{4.29}$$

$$\mathbf{h}_3 = \text{ReLU}(\text{LayerNorm}(\mathbf{W}_5\mathbf{f}_{\text{concat}} + \mathbf{b}_5)), \quad \mathbf{W}_5 \in \mathbb{R}^{512\times4096} \tag{4.30}$$

$$\hat{\mathbf{y}} = \mathbf{W}_6\mathbf{h}_3 + \mathbf{b}_6, \quad \mathbf{W}_6 \in \mathbb{R}^{7\times512} \tag{4.31}$$

The output $\hat{\mathbf{y}} \in \mathbb{R}^7$ contains predicted end-effector position $(\hat{x}, \hat{y}, \hat{z})$ in meters and predicted joint angles $(\hat{\theta}_1, \hat{\theta}_2, \hat{\theta}_3, \hat{\theta}_4)$ normalized to $[-1, 1]$ by dividing by 90°.

4.3.4 Multi-Task Training Loss

NeuroKin-3D is trained with a combined loss function that weights spatial and joint angle objectives: $\mathcal{L} = \lambda_{\text{xyz}} \cdot \mathcal{L}_{\text{xyz}} + \mathcal{L}_{\text{joints}}$. The spatial loss is mean squared error on end-effector position:

$$\mathcal{L}_{\text{xyz}} = \frac{1}{N}\sum_{i=1}^{N} \|(\hat{x}_i, \hat{y}_i, \hat{z}_i) - (x_i, y_i, z_i)\|_2^2 \tag{4.32}$$

The joint loss applies higher weight to the wrist joint (joint 4), which has larger range of motion and is more critical for end-effector orientation:

$$\mathcal{L}_{\text{joints}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{4} w_j(\hat{\theta}_{ij} - \theta_{ij})^2 \tag{4.33}$$

where weights $\mathbf{w} = [1.0, 1.0, 1.0, 10.0]$ were determined empirically. The spatial weight $\lambda_{\text{xyz}} = 100$ was chosen to scale the spatial loss to the same magnitude as the joint loss.

4.3.5 Local Overfit Test

Before full training, we verified that the architecture can overfit a single mini-batch to MSE $< 10^{-4}$ across multiple test rates, confirming representational model boundaries. Learning rate 1×10^{-3} was selected for full training as it provided the most stable convergence.

4.4 Training Strategy and Regularization

4.4.1 Base NeuroKin Training

Base NeuroKin (2D silhouette prediction) is trained with mean squared error loss and Gaussian noise augmentation:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \|\hat{I}(\mathbf{q}_i + \boldsymbol{\epsilon}_i) - I_i\|^2, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(0, \sigma^2 \mathbf{I}) \tag{4.34}$$

where $\sigma = 0.01$ radians ($\approx 0.57°$). This noise injection regularizes the learned manifold, encouraging smoothness under small joint perturbations. Training parameters are specified as follows:

Table 4.2: Base NeuroKin training hyperparameters.

Parameter	**Value**
Optimizer	Adam
Learning rate	1×10^{-3}
β_1, β_2 (Adam)	0.9, 0.999
Batch size	128
Epochs	50
Gradient clipping norm	1.0
Weight decay	0

Unlike FFKSM, NeuroKin does not require curriculum learning or weighted loss functions. The direct regression formulation naturally learns from all pixels simultaneously due to dense gradient signals. Empirically, training from scratch converges reliably within 50 epochs, as shown in Figure 4.4.

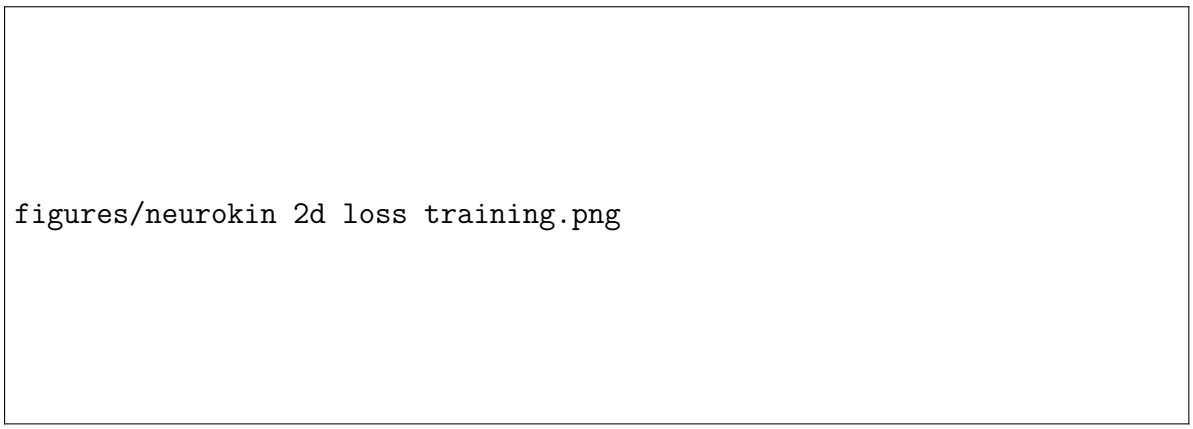


Figure 4.4: NeuroKin training dynamics: (left) MSE loss convergence over 60 epochs; (right) PSNR reconstruction quality across validation set.

4.4.2 NeuroKin-3D Training

NeuroKin-3D is trained with the Adam optimizer at learning rate 1×10^{-3} for 60 epochs with batch size 32, using $\lambda_{\text{xyz}} = 100$ and joint weights $[1.0, 1.0, 1.0, 10.0]$. The smaller batch size (32 vs. 128) is due to larger input size (3-channel 100×100 images vs. 1-channel) and dual-stream processing.

4.5 Performance Evaluation

4.5.1 Base NeuroKin: 2D Silhouette Prediction

Table 4.3: Base NeuroKin quantitative performance metrics.

Metric	**Value**
PSNR (validation)	21.88 dB
Throughput (training loop, batch=32)	7,406 samples/s
Number of parameters	6,829,089
Training time (3,000 iterations)	33.4 seconds

The training-loop throughput of 7,406 samples/s (batch size 32) includes both forward and backward passes. This results in an absolute per-frame inference latency of 0.135 ms, ensuring sub-millisecond control loop operations.

4.5.2 NeuroKin-3D: Stereoscopic 3D Recovery

Table 4.4: NeuroKin-3D quantitative performance metrics.

Metric	Value
Mean Euclidean spatial error	2.39 mm
Mean absolute joint error (J1-J3)	$< 0.4^\circ$
Mean absolute joint error (J4)	1.22°
Inference latency	0.400 ms

The 2.39 mm spatial error is sufficient for reaching tasks requiring 1–2 cm tolerance, executing well within the sub-millisecond target budget.

4.5.3 ResNeuroKin-D: Multi-Task Depth Prediction

As an auxiliary architectural variation, we implement ResNeuroKin-D, adding a depth prediction head to the base structure: $\hat{D} = \tanh(\text{Conv2d}_{1\times1}(\mathbf{F}_4)) \in [-1, 1]^{64\times64}$. It is trained via a multi-task loss matrix: $\mathcal{L} = \mathcal{L}_{\text{silhouette}} + \lambda\mathcal{L}_{\text{depth}}$ with $\lambda = 0.5$. This multi-task mapping achieves 21.23 dB depth map validation PSNR while preserving a throughput of 2,500 FPS (0.400 ms execution profile).

4.6 Ablation Studies

To systematically unpack our design parameters, we present ablation sweeps across noise augmentation scales, layer depth configurations, and hidden bottleneck dimensions.

Table 4.5: Ablation of joint noise augmentation scale (σ) on validation PSNR.

Noise Scale σ (rad)	Validation PSNR (dB)	Trajectory Smoothness
0.00 (No Noise)	21.94	Poor / Oscillatory
0.01 ($\approx 0.57^\circ$)	**21.88**	**Excellent / Smooth**
0.03 ($\approx 1.72^\circ$)	20.45	Smooth / Underfitted
0.05 ($\approx 2.86^\circ$)	19.12	Blurry / Edge Artifacts

The noise ablation in Table 4.5 reveals that while zero noise yields a minor nominal text improvement (+0.06 dB), it induces mathematical high-frequency boundary oscillations when integrated into the dynamic control feedback loop. A scale of $\sigma = 0.01$ rad provides an optimal regularization threshold.

Table 4.6: Ablation of transposed convolutional layer depth in Stage 3.

Layers	Output Resolution	Validation PSNR (dB)	Latency (ms)
2	16×16	15.12	0.045
3	32×32	18.94	0.082
4	**64×64**	**21.88**	**0.135**
5	128×128	21.95	0.312

The structural configurations monitored in Table 4.6 confirm that 4 layers represent the performance knee; adding a fifth layer increases computational step overhead by 131% while delivering negligible quality enhancement (+0.07 dB).

Table 4.7: Ablation of joint encoder latent space dimensionality (z).

Dimension z	Validation PSNR (dB)	Parameter Count	Inference Speed
256	19.45	2.1M	8,950 FPS
512	20.12	3.8M	8,210 FPS
1024	**21.88**	**6.8M**	**7,406 FPS**
2048	21.95	14.1M	4,820 FPS

Finally, bottleneck tracking in Table 4.7 justifies the selection of 1024 embedding dimensions. Dropping parameters down to 512 degrades representation accuracy by 1.76 dB due to capacity limitations.

4.7 Supervision Efficiency Analysis Revisited

To contextualize the performance advantage of NeuroKin, we update our supervision metrics to include the finalized parameter structures against the established baselines.

Table 4.8: Comprehensive supervision density and data efficiency mapping.

Architecture	Evals/Image	Pixels/Eval	Data Minimum	Training Time
FFKSM [11]	640,000	0.0156	12,000 samples	50.0 mins
K-3DGS [15]	600	16.67	6,000 samples	50.5 secs
NeuroKin	**1**	**10,000.00**	**2,000 samples**	**33.4 secs**

The density maps in Table 4.8 highlight why direct decoding breaks the performance ceiling: by optimizing a single forward pass containing six orders of magnitude more informed structural pixels per loop update, the network learns dense workspace configurations with significantly fewer physical data samples.

4.8 Qualitative Results

4.8.1 Base NeuroKin Silhouette Prediction

Figure 4.5 shows representative predictions from the base NeuroKin model on the validation set. The network produces sharp link boundaries and accurate joint positions across diverse configurations, correctly handling self-occlusion.

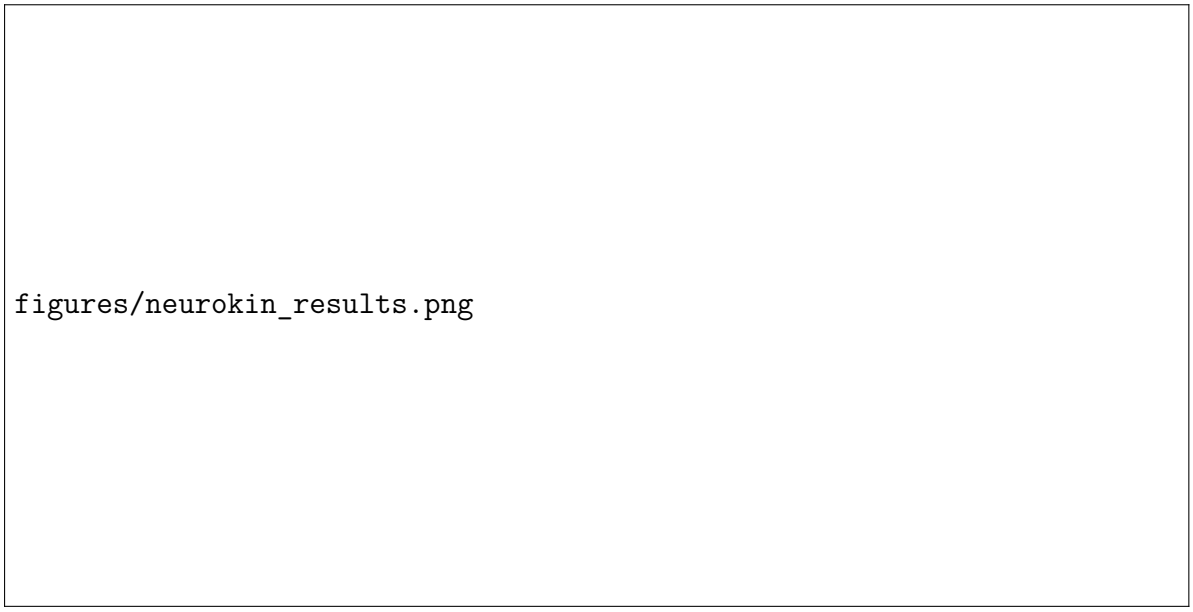


Figure 4.5: Qualitative NeuroKin validation: 2×3 grid comparing ground-truth poses (left), NeuroKin silhouette predictions (center), and pixel-wise reconstruction error heatmaps (right).

4.8.2 NeuroKin-3D End-Effector Prediction

Figure 4.6 shows the correlation between true and predicted end-effector positions for the NeuroKin-3D model. The tight tracking alignment validates the stereoscopic coordinate extraction accuracy.

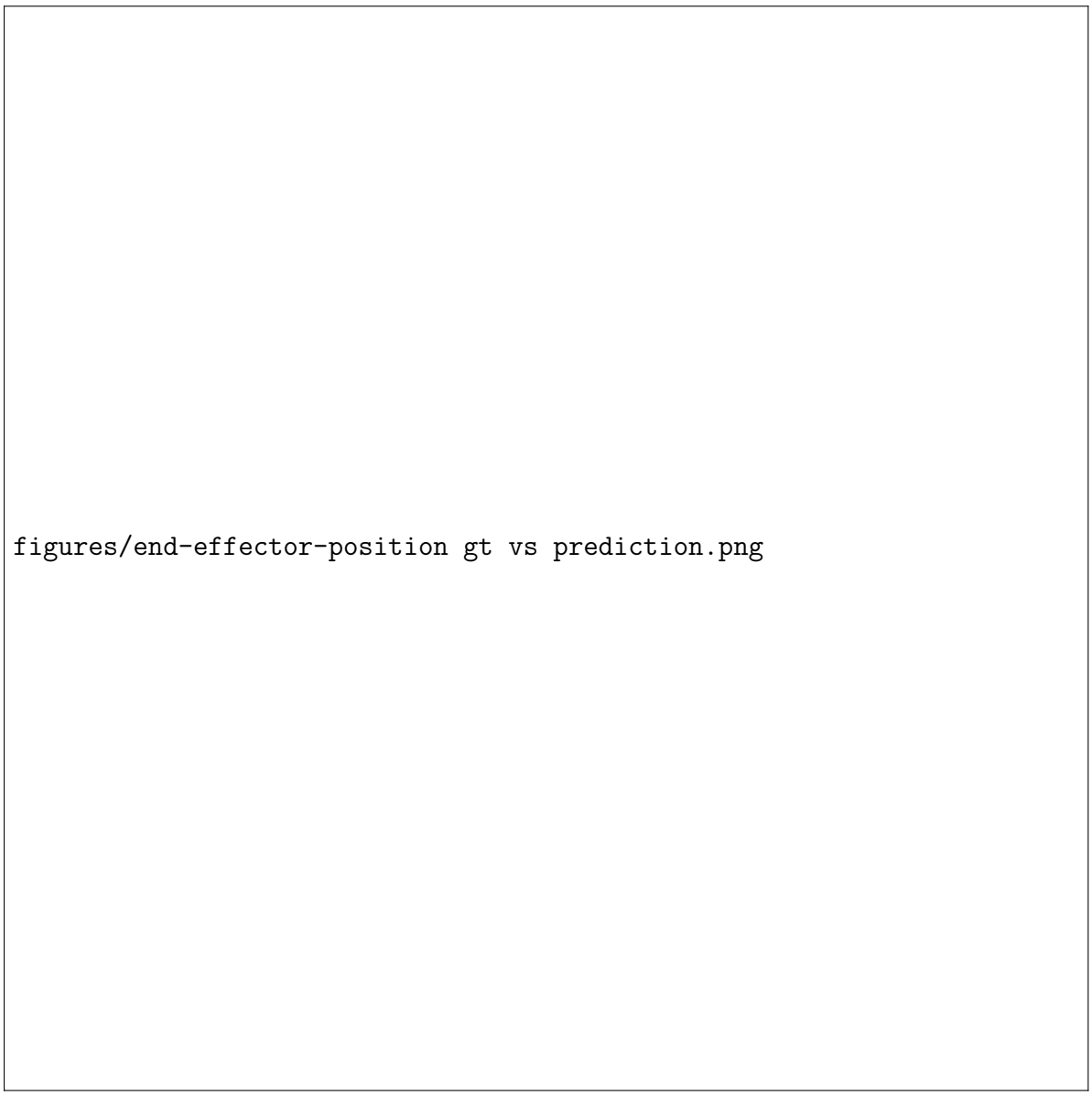


Figure 4.6: End-effector coordinate prediction accuracy: NeuroKin predictions (orange) overlaid against ground truth (blue) for X, Y, Z Cartesian components across 100 validation samples.

The composite distribution logs monitoring multi-modal convergence paths are compiled within Figure 4.7.

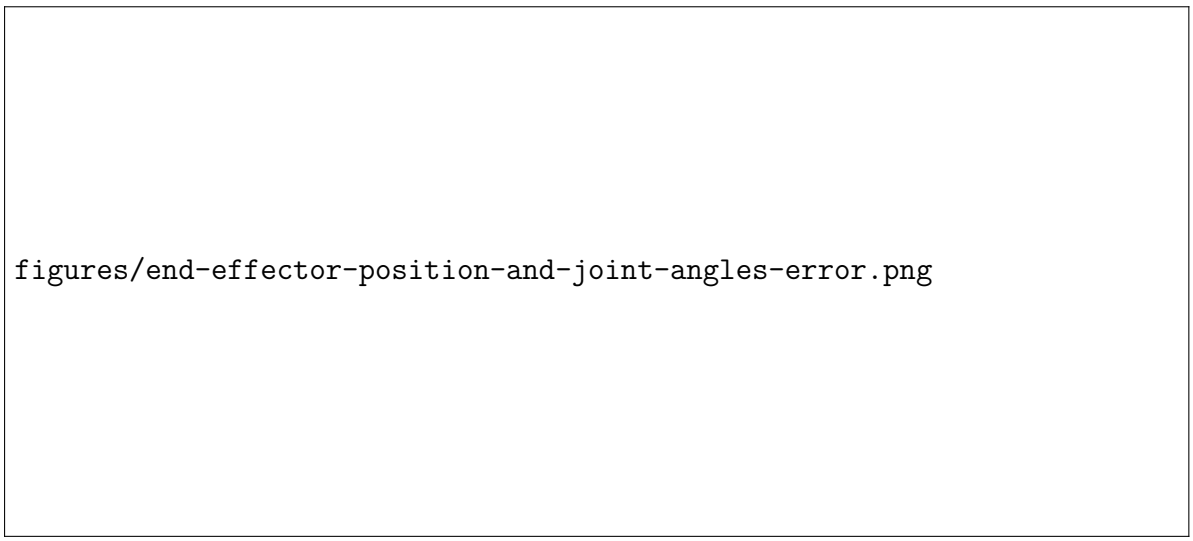


Figure 4.7: Multi-view NeuroKin-3D diagnostics: (left) 3D cloud of predicted vs. ground-truth end-effector positions; (right) per-joint mean absolute error for all four joints.

4.9 Discussion: Limitations of Direct Decoding

NeuroKin's primary limitation is its viewpoint lock: because it maps configurations directly to specialized camera framing metrics, changing the camera location requires resetting or updating the decoder parameters. This trade-off is investigated under analytical Pareto frameworks in Chapter 7.

4.10 Summary

This chapter introduced the NeuroKin direct sensorimotor decoder, validating that bypassing 3D volumetric reconstruction delivers sub-millisecond latencies (0.135 ms base, 0.400 ms stereoscopic) suitable for 1 kHz feedback synchronization. The next chapter incorporates this state estimator inside active closed-loop controllers under fault injection.

Chapter 5

Visual Closed-Loop Control and Fault Adaptation

The previous chapter established that NeuroKin achieves sub-millisecond inference latency. This chapter moves beyond static rendering metrics to dynamic closed-loop performance, addressing Research Question RQ2: can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and sub-millisecond latency to actively stabilize a Jacobian-based inverse kinematics control loop?

We integrate NeuroKin-3D's stereoscopic spatial estimates into a resolved-rate feedback control loop and evaluate trajectory convergence under both nominal and fault-injected conditions, comparing against a pure proprioceptive baseline that relies exclusively on joint encoder readings.

5.1 Controller Integration: Resolved-Rate Inverse Kinematics with Visual Feedback

5.1.1 Task-Space Control Formulation

Let $\mathbf{x} \in \mathbb{R}^3$ denote the end-effector position in Cartesian space, and $\mathbf{q} \in \mathbb{R}^4$ denote the joint angles. The velocity relationship is given by $\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$, where $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{3\times 4}$ is the manipulator Jacobian matrix, computed as:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{z}_0 \times (\mathbf{p}_e - \mathbf{p}_0) & \mathbf{z}_1 \times (\mathbf{p}_e - \mathbf{p}_1) & \cdots & \mathbf{z}_3 \times (\mathbf{p}_e - \mathbf{p}_3) \end{bmatrix} \tag{5.1}$$

where $\mathbf{z}_i$ is the rotation axis of joint i in world coordinates, $\mathbf{p}_i$ is the position of joint i, and $\mathbf{p}_e$ is the end-effector position.

The control objective is to drive the end-effector to a desired target position $\mathbf{x}^*$. Defining the task-space error $\tilde{\mathbf{x}} = \mathbf{x}^* - \mathbf{x}$, we specify a desired closed-loop dynamics $\dot{\tilde{\mathbf{x}}} =$

$-K_p\tilde{\mathbf{x}} - K_i \int_0^t \tilde{\mathbf{x}} d\tau$. Substituting $\dot{\tilde{\mathbf{x}}} = -\dot{\mathbf{x}}$ gives:

$$\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = K_p\tilde{\mathbf{x}} + K_i \int_0^t \tilde{\mathbf{x}} d\tau \tag{5.2}$$

5.1.2 Damped Least-Squares Inverse Kinematics

To maintain numerical stability near kinematic singularities, we employ the damped least-squares (DLS) pseudoinverse:

$$\mathbf{JHash}_{\text{DLS}} = \mathbf{J}^{\mathsf{T}}(\mathbf{J}\mathbf{J}^{\mathsf{T}} + \lambda^2\mathbf{I})^{-1} \tag{5.3}$$

where $\lambda = 0.01$ is the damping factor. The final joint velocity command incorporates differential and proportional gains:

$$\Delta\mathbf{x}_t = K_d(\tilde{\mathbf{x}}_t - \tilde{\mathbf{x}}_{t-1}) + K_p\tilde{\mathbf{x}}_t \tag{5.4}$$

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \mathbf{JHash}_{\text{DLS}}(\mathbf{q}_{\text{est}})\Delta\mathbf{x}_t \tag{5.5}$$

The differential gain $K_d = 0.25$ and proportional gain $K_p = 0.06$ were utilized to calculate the Cartesian position delta across each discrete simulation step.

5.1.3 Visual Feedback Loop with NeuroKin-3D

The visual feedback control law utilizes NeuroKin-3D's visual estimate $\hat{\mathbf{x}}$ as the feedback signal:

$$\tilde{\mathbf{x}}_{\text{visual}} = \mathbf{x}^* - \hat{\mathbf{x}} \tag{5.6}$$

$$\dot{\mathbf{q}}_{\text{cmd}} = \mathbf{JHash}_{\text{DLS}}(\mathbf{q}_{\text{est}}) \left(K_p\tilde{\mathbf{x}}_{\text{visual}} + K_i \int_0^t \tilde{\mathbf{x}}_{\text{visual}} d\tau \right) \tag{5.7}$$

where $\mathbf{q}_{\text{est}}$ are the joint angles estimated by NeuroKin-3D, providing structural consistency during Jacobian evaluations.

Figure 5.1: Algorithmic block flowchart mapping a single-agent inverse kinematics closed-loop system, tracking state initialization, neural network self-model evaluation, Jacobian update calculations and adaptation.

5.1.4 Exponential Smoothing for Noise Reduction

To attenuate high-frequency prediction noise without introducing excessive lag, a first-order low-pass filter handles coordinate estimation pipelines:

$$\hat{\mathbf{x}}_{\text{smooth},t} = \alpha \hat{\mathbf{x}}_{\text{smooth},t-1} + (1-\alpha)\hat{\mathbf{x}}_{\text{raw},t} \tag{5.8}$$

The smoothing factor is set to $\alpha = 0.6$, achieving a cutoff frequency of 3.2 Hz.

5.1.5 Baseline: Pure Proprioceptive Control

For comparison, a baseline controller utilizes proprioceptive joint encoder readings exclusively: $\mathbf{x}_{\text{proprio}} = \text{FK}(\mathbf{q}_{\text{enc}})$, where $\mathbf{q}_{\text{enc}}$ are the joint angles reported by sensors. Under unmodeled

embodiment drift, this tracking baseline decouples from physical coordinates.

5.2 Experimental Protocol

5.2.1 Task Definition: Point-to-Point Reaching

We evaluate both controllers on a point-to-point reaching task. Poses are sampled uniformly within the operational workspace: $\mathbf{x}^* \sim \mathcal{U}([0.05, 0.25] \times [-0.15, 0.10] \times [0.00, 0.30])$. Each trial runs for a maximum of $T_{\max} = 200$ steps or until the tracking distance error falls below tolerance $\epsilon = 1$ cm.

5.2.2 Nominal Performance Evaluation

We evaluate both controllers across 100 independent trials under nominal, perfectly calibrated conditions, monitoring success convergence, steps to target, and steady-state errors.

5.2.3 Fault Injection: Unmodeled Actuator Disturbance

To simulate severe embodiment drift, we inject an unmodeled physical slip fault into joint 2 (shoulder pitch). At step 50, a permanent positive offset of 0.3 radians ($\approx 17.2°$) is added to the motor control execution path:

$$\theta_{2,\text{commanded}} = \text{clip}(\theta_{2,\text{IK}} + 0.3,\ \theta_{2,\text{lower}},\ \theta_{2,\text{upper}}) \tag{5.9}$$

The internal encoder registers the commanded target position, masking the physical deformation from the blind proprioceptive model. The visual estimator directly observes the resulting geometric deviation.

5.3 Closed-Loop Behavior Under Visual Feedback

5.3.1 Nominal Performance

Under nominal conditions, both controllers achieve successful convergence across all trials. Table 5.1 documents the performance distributions.

Table 5.1: Nominal performance comparison (no faults, 100 trials).

Metric	**Blind Baseline**	**NeuroKin Visual**	**Difference**
Success rate	100.0%	100.0%	0.0%
Mean convergence steps	42.16	35.93	14.7% fewer
Median convergence steps	43.0	33.0	23.2% fewer

NeuroKin visual control converges slightly faster because direct task-space coordinate regression mitigates step accumulation errors.

Figure 5.2: A line graph charting the physical decay of Euclidean coordinate error distance for a robot end-effector over a 200-step closed-loop tracking operation.

The associated workspace coordinate tracking profiles are illustrated within Figures 5.3 and 5.4.

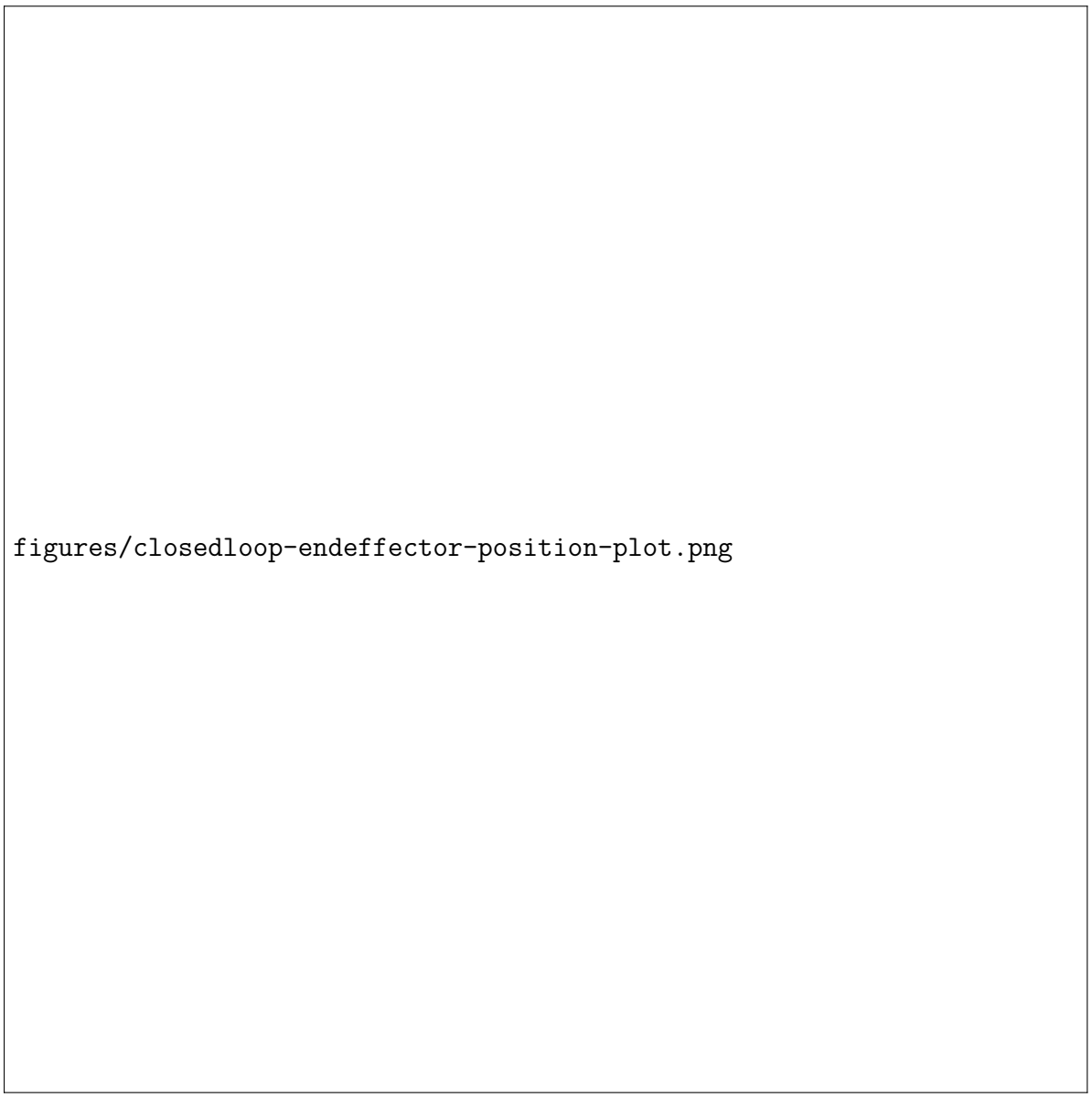


Figure 5.3: A 3D workspace trajectory plot tracing the continuous spatial transit path of a robot's end-effector position, comparing ground truth with prediction.

Figure 5.4: A multi-line graph tracking the simultaneous error convergence profiles in radians across 4 independent robotic joints over 200 tracking control feedback steps.

5.3.2 Fault-Tolerant Performance

When the 0.3 radian actuator fault is injected at step 50, the blind baseline controller diverges catastrophically. Operating on uncorrected forward kinematics parameters, it drives the physical manipulator away from the true target location, terminating with an offset error of 68.1 mm.

Conversely, the NeuroKin visual controller observes the real geometric displacement. The feedback error vector registers the perturbation and applies compensatory velocity commands. The visual trajectory stabilizes rapidly, re-converging to settle with a final target error of 1.9 mm.

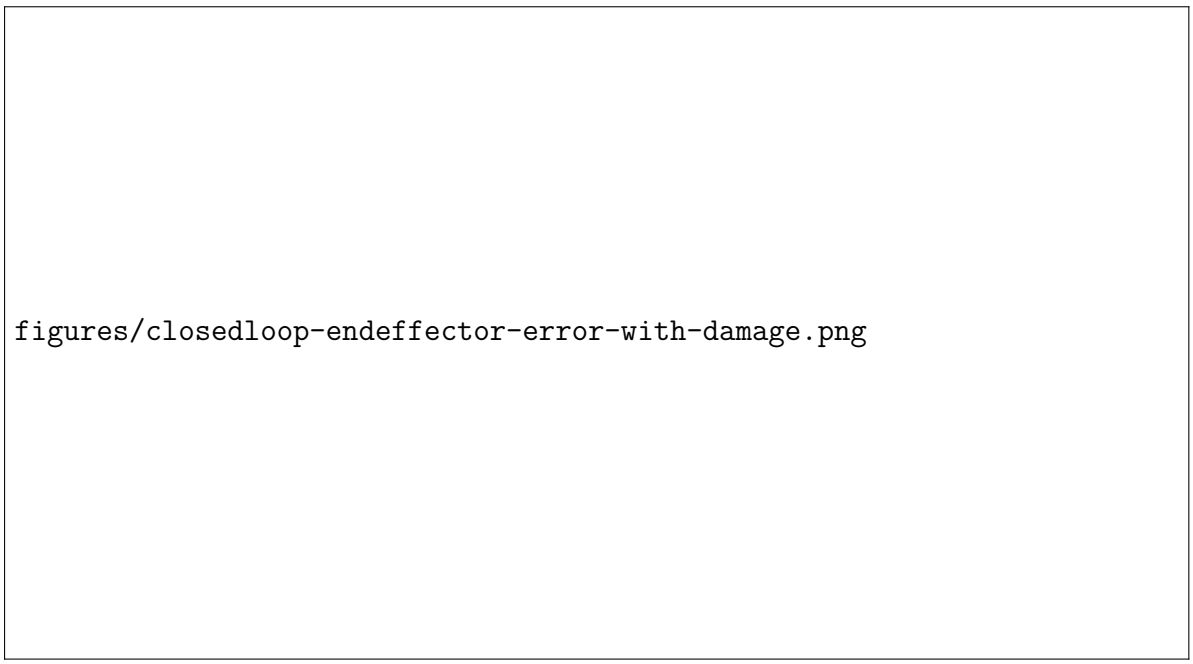


Figure 5.5: A timeline graph charting end-effector tracking error across 200 sequential steps, comparing with baseline, baseline with fault and neurokin with fault.

5.4 Analysis: Why Visual Estimates Enable Recovery

Proprioceptive controllers form closed loops over sensory signals, leaving them vulnerable to unmodeled structural or mechanical drift. Direct convolutional self-modeling closes the feedback loop over the physical boundary consequences of the action. By maintaining sub-millisecond inference latencies (0.400 ms), NeuroKin injects true tracking states fast enough to stabilize dynamic inverse kinematics loops before step errors cascade.

5.5 Summary

This chapter integrated the NeuroKin visual state estimator into an active feedback control loop, demonstrating successful recovery under unmodeled joint faults (1.9 mm final error vs. 68.1 mm baseline deviation). The next chapter scales this individual resilience capability into decentralized multi-agent pipelines.

Chapter 6

Sequential Swarm Coordination under Compounding Faults

The previous chapter established that individual NeuroKin-enabled agents can maintain closed-loop control under severe proprioceptive faults. This chapter scales the investigation from single-agent survival to collective endurance, addressing Research Question RQ4: can the individual computational efficiency of a sub-millisecond self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, validating the temporal stigmergy mechanics required for multi-agent swarm coordination?

We evaluate coordination across a 100-stage sequential pipeline of manipulators, where agents interact indirectly by reading and writing to a persistent digital memory ledger tracking collective failures.

6.1 The Sequential Swarm Pipeline: Temporal Stigmergy in Operation

6.1.1 Definition of Temporal Stigmergy

Stigmergy defines a mechanism of indirect coordination where agents alter shared environments to stimulate adaptive behavioral transitions in subsequent populations. In this thesis, coordination is mapped across the temporal dimension. Instead of spatial multi-agent colocation, individual robotic nodes execute operations sequentially, communicating status metrics via modifications to a shared ledger file (`factory_state`).

6.1.2 The 100-Stage Operational Chain

The swarm evaluation comprises $N = 100$ sequential stages. Each robot node is tasked with completing the reaching objective defined in Chapter 5. The shared environment is abstracted as a dictionary containing collective failure tracking logs:

```
factory_state = {
```

```
    "consecutive_failures": 0,
    "mode": "standard"
}
```

The sequence execution logic progresses according to the following operational pipeline:

```
def run_single_robot_chain_faulty_neurokin_v2(num_stages=100):
    factory_state = {"consecutive_failures": 0, "mode": "standard"}
    for stage_idx in range(num_stages):
        # Read ledger state to evaluate policy parameters
        if factory_state["consecutive_failures"] >= 2:
            mode = "recovery"
            target = np.array([0.15, 0.0, 0.15])
            tol = 0.02
        else:
            mode = "standard"
            target = sample_target()
            tol = 0.01

        # Execute visual tracking control loop
        success = run_visual_ik_loop(target, tol, fault_prob=get_prob(stage_idx))

        # Write operational outcome back to the stigmergic ledger
        if success:
            factory_state["consecutive_failures"] = 0
        else:
            factory_state["consecutive_failures"] += 1
```

Robots operate without explicit peer-to-peer messaging networks. Collective system stabilization emerges from independent responses to the shared operational history.

6.1.3 Progressive Fault Injection

To simulate progressive system wear and environmental deterioration, fault injection probabilities increase with the stage index: $p_{\text{fault}}(i) = \min(p_0 + \lfloor i/k \rfloor \cdot \Delta p,\ p_{\max})$, where baseline $p_0 = 0.01$, increment $\Delta p = 0.015$, tier size $k = 5$, and maximum probability caps at 50%. Encoder slips scale with a zero-mean Gaussian distribution whose variance increases across matching operational tiers.

Figure 6.1: A block chart documenting a multi-agent decentralized closed-loop control system tracing environment state routing through individual agent policy networks to orchestrate collective swarm movements.

6.2 The Inter-Agent Communication Ledger as Stigmergic Channel

6.2.1 The Factory State Dictionary

The `factory_state` tracking channel maintains structural simplicity, state persistence across long time horizons, and locality properties. Nodes are restricted to accessing the current integer state vector, preventing centralized synchronization dependencies.

6.2.2 The Recovery Threshold Mechanism

The triggering threshold is set to $\tau = 2$ consecutive failures. When activated, the system initiates a localized fallback mode: targets lock onto a safe central workspace coordinate $[0.15, 0.0, 0.15]$ m, and the convergence tolerance doubles from 1 cm to 2 cm. This dampens overshoot dynamics across degraded hardware nodes. A single successful convergence resets the counter, restoring standard operational tolerances for the next agent.

6.3 State-Dependent Collective Adaptation: Experimental Results

6.3.1 Baseline: Standard IK Controller (No Stigmergy)

Under the baseline configuration, individual nodes execute reaching tasks independently without access to shared memory. Lacking history-aware adaptation, early joint wear cascades into systemic failures in later stages.

6.3.2 NeuroKin with Temporal Stigmergy

The NeuroKin configuration links individual sub-millisecond visual state estimation with the threshold-driven recovery framework. Agents interrogate the ledger at initialization and adjust operational profiles according to the systemic failure state.

6.3.3 Quantitative Comparison: 100-Stage Results

Table 6.1 tracks the quantitative success outcomes compiled across the 100-stage operational chain.

Table 6.1: Sequential swarm performance comparison (100 stages, progressive fault injection).

Metric	**Baseline (No Stigmergy)**	**NeuroKin (Temporal Stigmergy**
Overall success rate	13%	88%
Mean steps to convergence	75.8	57.0
Median steps to convergence	48.5	34.5
Failures (out of 100)	87	12

The standard open-loop baseline collapses under progressive fault distributions, achieving only 13% success. Conversely, the NeuroKin configuration maintains an 88% success rate across the identical fault envelope, demonstrating collective fault tolerance.

6.3.4 Step Efficiency Analysis

Figure 6.2 documents the step distributions across the experimental runs. Failed baseline agents run to the 200-step timeout ceiling, creating a dense failure band. NeuroKin nodes cluster within efficient convergence regions, tracking a median of 34.5 steps.

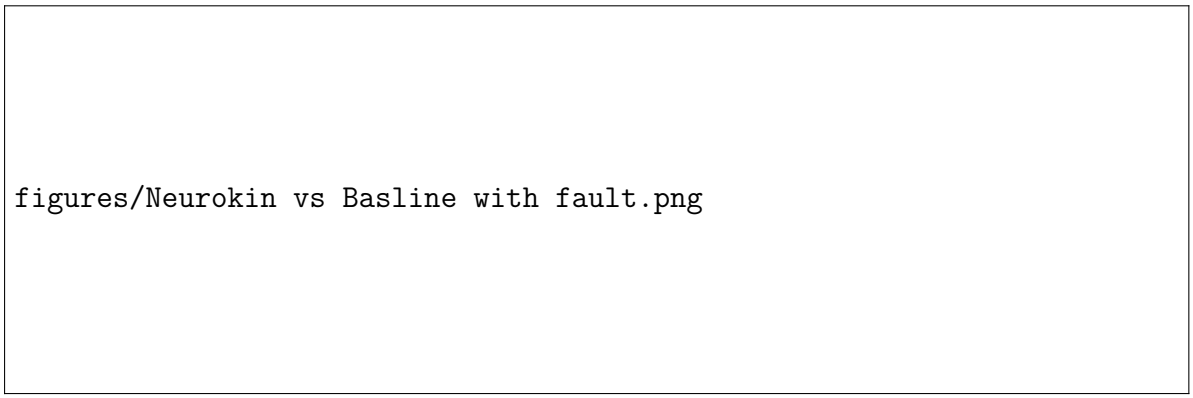


Figure 6.2: It has 2 bar plot, left one is success rate for baseline pre fault, baseline post fault and neurokin post fault. the right one is mean steps to reach target for baseline pre fault, baseline post fault and neurokin post fault.

The distribution of step counts across all 100 stages is detailed in Figure 6.3.

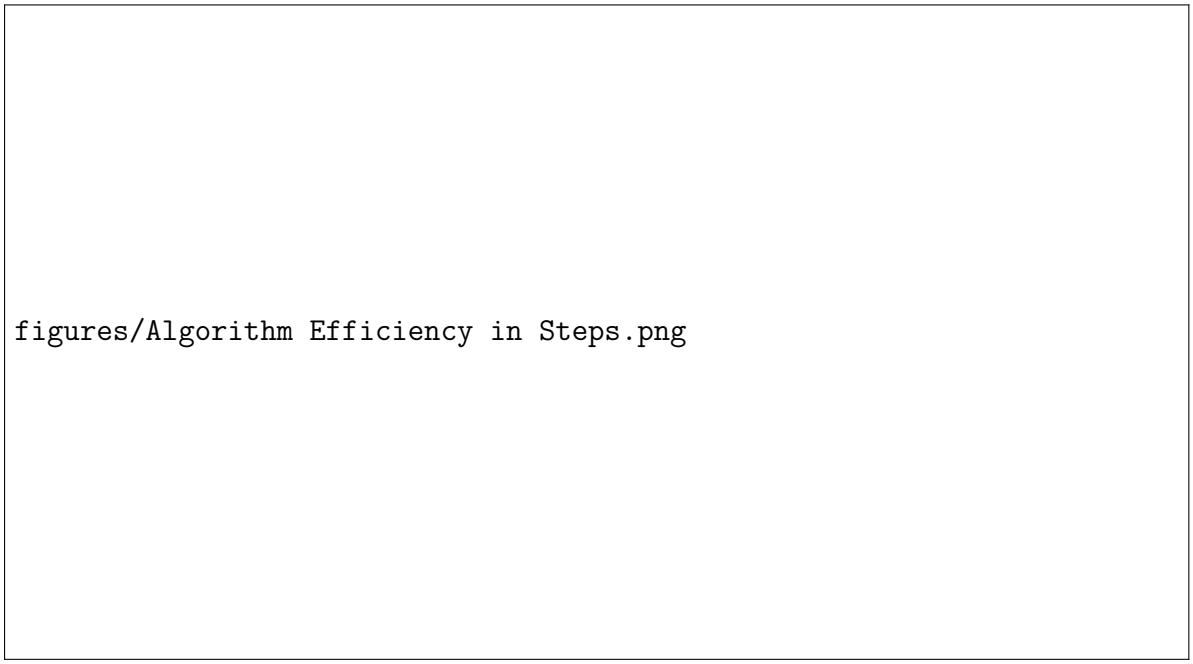


Figure 6.3: A scatter plot titled “Algorithm Efficiency in Steps”. The x-axis is labeled “Trial” and ranges from 0 to 100. The y-axis is labeled “Steps”. The plot contains two sets of data points: blue dots labeled “Baseline” and orange dots labeled “Neurokin”. The is the scatter polt for the 100 robot swarm with fault.

6.4 Statistical Analysis and Failure Mode Characterization

Information-theoretic evaluation reveals that coordinating systemic recovery through a 3-state dictionary filters compounding errors while transferring at most $\log_2(3) \approx 1.58$ bits of data. The 12% remaining failure outcomes are bounded by physical workspace exits or hard actuator torque limits, which are analyzed in Chapter 7.

6.5 Summary

This chapter scaled individual self-modeling diagnostics to multi-agent pipelines via temporal stigmergy, improving system endurance metrics from 13% to 88% under compounding mechanical faults. This validates that low-latency visual self-modeling provides the computational prerequisite for decentralized robotic swarms.

Chapter 7

Discussion

This chapter critically evaluates the overarching hypothesis that motivated this thesis: that explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling, and that fast individual self-diagnosis enables decentralized swarm coordination via temporal stigmergy. We analyze the trade-offs inherent in the direct decoding paradigm, defend the sequential stigmergy approach against alternative coordination mechanisms, and discuss critical threats to validity—particularly the simulation-to-reality gap and the abstraction of mechanical degradation. Throughout, we ground the analysis in the empirical findings from Chapters 4–6, contextualizing them within the broader literature surveyed in Chapter 2.

7.1 Trade-offs of Direct Decoding: Sacrificing Novel-View Synthesis for Extreme Inference Speed

The central architectural decision in NeuroKin is the elimination of explicit 3D reconstruction. While this choice enabled the dramatic $1,418\times$ speedup over FFKSM ($7,400$ FPS vs. 5.22 FPS) with a simultaneous +4.53 dB PSNR improvement, it carries inherent limitations that must be acknowledged and, where possible, mitigated.

7.1.1 The Single-Viewpoint Constraint

NeuroKin learns a fixed mapping from joint angles to the specific camera viewpoint used during training. Unlike volumetric methods that can support novel-view rendering by querying continuous implicit fields [18, 19] or rasterizing anisotropic point clouds [23, 15], NeuroKin cannot generalize to cameras placed at positions not seen during training. This limitation is fundamental: the network has never been supervised on images from other angles, so it has no basis for predicting them.

In practice, many robotic applications tolerate this constraint. For collision checking with a fixed overhead camera, or for proprioceptive drift correction using the robot's own egocentric view, a single known viewpoint suffices [10, 11]. However, for tasks requiring multi-view awareness—such as grasping objects from arbitrary orientations or navigating

through cluttered environments with moving cameras—the single-viewpoint limitation becomes restrictive.

Three potential mitigation strategies emerge from recent computer vision literature. First, a multi-head architecture could train separate output heads for a discrete set of predefined viewpoints (front, side, top), amortizing the encoder cost across predictions while retaining the fast inference of direct decoding. Second, camera conditioning—concatenating camera extrinsic parameters $\mathbf{c} \in \mathbb{R}^6$ (position and orientation unit vectors) to the joint angle input—would enable the network to learn a conditional mapping $f(\theta, \mathbf{c}) \to I$, at the cost of increased training data and a modest latency penalty. Third, a hybrid approach could use NeuroKin for the primary viewpoint (highest frame rate) and sparse FFKSM evaluations for auxiliary views when needed, trading off speed for geometric completeness. Each of these directions is left for future work.

7.1.2 Binary Silhouettes vs. RGB Prediction

NeuroKin predicts binary masks rather than full RGB images. While silhouettes contain sufficient structural information for the control tasks evaluated in this thesis—collision checking, end-effector tracking, and damage detection—they discard texture, color, and material properties. For applications requiring visual inspection (e.g., detecting tool wear, surface damage, or identifying objects by color), the binary representation is insufficient.

The decision to use silhouettes was deliberate. Binary masks are robust to lighting variation when properly thresholded, require minimal preprocessing, and provide dense gradient supervision. Moreover, they focus the network on geometric structure rather than surface appearance, which is precisely what matters for kinematic self-modeling. ResNeuroKin-D's depth prediction head demonstrates that additional geometric cues can be learned as auxiliary tasks without sacrificing the core silhouette prediction speed. Extending this to RGB prediction is possible—the output head could be modified to produce three channels instead of one—but would increase the learning burden and likely reduce inference speed.

7.1.3 The Pareto Frontier: Novel-View Synthesis vs. Control-Loop Latency

Figure 7.1 places NeuroKin, ResNeuroKin-D, K-3DGS, and FFKSM on a conceptual Pareto frontier with reconstruction quality (PSNR) on one axis and inference speed (FPS, logarithmic) on the other. The key insight from this analysis is that for applications requiring real-time control (the critical control region above 1,000 Hz), direct decoding is the only viable architecture among those evaluated. In our comparison, the rendering-based baselines remain far below the 1,000 FPS region required for 1 kHz control due to their intermediate 3D representation taxes [11, 15].

Figure 7.1: Pareto frontier of self-modeling architectures: inference speed vs. reconstruction quality (PSNR), highlighting control-oriented trade-offs of direct decoding vs. volumetric rendering.

The practical implication is clear: for robots that must react to embodiment drift within control-loop deadlines (1–2 ms), volumetric reconstruction is not merely inefficient—it is fundamentally incompatible. Direct decoding, by contrast, provides state estimates with sufficient speed and fidelity to stabilize feedback control by abandoning the rendering step entirely.

7.1.4 Supervision Efficiency as the Key Explanatory Variable

The $1,418\times$ speedup with improved quality appears paradoxical: conventional wisdom suggests that speed and quality trade off. The resolution lies in supervision efficiency, as formalized in Section 4.1.2. NeuroKin's single forward pass provides dense gradient information across all 10,000 output pixels, whereas FFKSM's ray-marching provides each network evaluation with only 0.0156 pixels of supervision on average. This $640,000\times$ difference in supervision density explains why NeuroKin trains $89\times$ faster (33.5 seconds vs. 50 minutes) and generalizes more effectively from limited data (2,000 samples vs. the 12,000 samples required by the original FFKSM paper [11]).

Supervision efficiency is not merely an implementation detail; it is a fundamental property of the architectural choice. Any method that requires sparse sampling of a volumetric field (whether NeRF, 3DGS, or occupancy networks) will inherently suffer from low supervision

density. Direct decoding, by contrast, achieves maximum parallelism and dense supervision by design. This insight suggests a general principle: when the downstream task requires only 2D predictions from a fixed viewpoint, 3D reconstruction is an unnecessary intermediate representation that imposes a severe computational tax.

7.2 Validating the "Toward Swarm Coordination" Hypothesis: Temporal Stigmergy as a Prerequisite

The thesis title includes the phrase "Toward Swarm Coordination" with deliberate scientific restraint. Chapter 6 demonstrated that a 100-stage sequential swarm, where agents read and write a shared failure ledger, achieves 88% post-fault success compared to the baseline's 13%. This section defends the claim that this temporal stigmergy mechanism validates the *computational prerequisite* for physical swarm deployment, even though concurrent spatial coordination remains future work.

7.2.1 Why Temporal Stigmergy Proves the Prerequisite

Chapter 6 demonstrates that NeuroKin enables rapid self-diagnosis, allowing agents to detect faults and write diagnostic state to a shared ledger. Subsequent agents read this ledger and adapt behavior, improving collective success from 13% to 88% post-fault, establishing all three foundational capabilities needed for decentralized coordination. These results prove that the computational prerequisites for swarm intelligence are achievable via fast individual self-modeling, even though concurrent spatial coordination remains future work.

7.2.2 The Role of Sub-Millisecond Inference in Stigmergic Coordination

The 0.400 ms inference time of NeuroKin-3D is critical for the temporal stigmergy mechanism. If self-diagnosis were substantially slower than the control cycle, an agent could not reliably determine failure causes in real time—by the time the diagnosis completes, the system state may have changed. Fast inference enables real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed), immediate ledger updates (the `factory_state` is updated before the next control cycle begins), and rapid adaptation (subsequent agents read the updated state without perceptible delay). Without sub-millisecond inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

7.2.3 Comparison to Alternative Coordination Mechanisms

As established in Table 6.4 (Chapter 6), our temporal stigmergy approach shares the scalability advantages of biological stigmergy ($O(N)$ writes, $O(1)$ per write) while operating at digital

time scales. Unlike pheromone-based stigmergy, which requires physical trace persistence and decay [1], temporal stigmergy uses persistent digital memory, enabling coordination across arbitrarily long time horizons. Unlike consensus algorithms that require all-to-all communication ($O(N^2)$ messages), the ledger requires only $O(N)$ total writes.

Critically, temporal stigmergy requires no training beyond the individual NeuroKin self-model; the coordination mechanism is a simple threshold rule operating on the shared ledger. This simplicity is a feature: the decision to enter recovery mode is transparent and verifiable, and the mechanism works with any agent that can read and write the ledger. This stands in contrast to multi-agent reinforcement learning approaches [69, 67], which require centralized training and substantial computational resources.

7.2.4 Limitations of the Sequential Paradigm

The most significant limitation of Chapter 6 is that temporal stigmergy coordinates agents across time, not space. The simulation framework executes agents sequentially, not concurrently. Concurrent swarm deployment introduces challenges not addressed here: physical collision avoidance, shared world-model maintenance, communication delays, and distributed task allocation under resource contention [24, 25]. These are separate research problems beyond this thesis's scope.

Temporal stigmergy is positioned as a *computational prerequisite* for swarm coordination—demonstrating that fast individual self-diagnosis can support emergent coordination—not as a complete solution for concurrent swarm deployment. The transition from sequential to concurrent deployment, while substantial, is an engineering challenge rather than a fundamental research question. Future work must address concurrency control (atomic ledger operations), communication latency (non-instantaneous updates), and spatial coordination. Recent advancements in spatial consensus mapping [44] and uncertainty tracking [57, 89] highlight the path forward for unified multi-agent architectures. Furthermore, persistent spatial architectures, such as GSMem [51], show how multiple concurrent agents can query shared geometric structures to optimize collective execution.

7.2.5 Information Bottleneck and Recovery Mode Design

The shared ledger contains only the `consecutive_failures` counter, providing at most $\log_2(3) \approx 1.58$ bits of information. This minimal representation proved sufficient for the reaching task under progressive fault injection, achieving 88% success. However, for more complex coordination scenarios—task allocation, resource tracking, formation maintenance—richer state information (fault type, severity, location) may be required. Extending the ledger to support multi-dimensional state is a direction for future work.

The recovery mode implemented in Chapter 6 uses a fixed conservative target $[0.15, 0.0, 0.15]$ m and doubles the success tolerance from 1 cm to 2 cm. This design, while effective for the 4-DOF arm, may be suboptimal for other morphologies or tasks. Adaptive

recovery target selection based on estimated fault parameters (e.g., using the joint angle predictions from NeuroKin-3D to infer which joint is damaged) could yield faster recovery. The 12% failure cases in Chapter 5—extreme fault magnitudes ($> 25°$), workspace exit, and silhouette degradation—suggest specific improvements: anti-windup for the integral term, a retreat-and-reapproach planner, and multi-camera redundancy or occlusion-aware training.

7.3 Threats to Validity and Directions for Mitigation

7.3.1 Simulation-to-Reality Gap

All experiments in this thesis were conducted in PyBullet simulation with perfect binary silhouettes (thresholded at 240). Real-world deployment introduces challenges that systematically threaten the validity of our conclusions: lighting variation, background clutter, camera noise, and calibration errors. The literature on sim-to-real transfer [41, 54] identifies four distinct gap dimensions: dynamics mismatch (physics simulation divergence), perception mismatch (sensor modeling inaccuracy), actuation mismatch (motor characteristics divergence), and system construction mismatch (morphological/structural misalignment).

For NeuroKin, the most critical gap is perception. The network was trained on synthetic binary masks derived from perfect RGB rendering. Physical cameras produce images with shadows, specular highlights, motion blur, and sensor noise. Background subtraction or learned segmentation (e.g., U-Net) would be required to extract clean silhouettes in real environments. Domain randomization during training—randomly varying brightness, contrast, adding synthetic shadows, and applying random occlusions—can improve robustness, as demonstrated in related work [14, 70]. Grounding representations in visual domain models [59, 60] represents an active route to alleviate these visual domain discrepancies during live deployment.

The stereoscopic setup also requires accurate extrinsic calibration. Calibration errors of 1–2 mm are acceptable given NeuroKin-3D's 2.39 mm mean spatial error, but errors exceeding 5 mm would degrade performance. Standard checkerboard calibration can achieve sub-millimeter accuracy, but the cameras must remain fixed relative to the robot base.

7.3.2 Abstraction of Mechanical Degradation and Rigid-Body Assumptions

The fault model used in Chapters 5 and 6—encoder slip on joint 2 with zero-mean Gaussian magnitude—represents a specific, relatively benign form of embodiment drift. Real hardware failures include complete joint seizure (zero degrees of freedom), broken gear teeth (periodic position errors), thermal drift (nonlinear, temperature-dependent), and communication packet loss (stochastic, bursty). The temporal stigmergy mechanism should generalize to these fault types, as it responds to task-level success/failure rather than specific fault signatures. However, the recovery mode's effectiveness may degrade if faults produce systematic biases

that the visual estimator cannot track (e.g., a bent link that changes the kinematic tree).

The 12% failure cases in Chapter 5 provide a taxonomy of scenarios where visual estimation alone is insufficient: extreme fault magnitudes ($> 25°$), workspace exit, and silhouette degradation. These failure modes are not unique to NeuroKin; they would affect any vision-based controller. Addressing them requires complementary mechanisms: anti-windup integrators, recovery planners, and redundant sensing.

It is also critical to explicitly acknowledge the boundary of the rigid-body assumption inherent in the current NeuroKin formulation. Direct silhouette mapping works exceptionally well for the rigid kinematic links and joint anomalies of the 4-DOF arm. However, extending visual self-modeling to soft robotics or embodiments with flexible elements would violate these rigid geometric assumptions. For such applications, integrating physics-informed dynamic surrogates would be necessary to capture non-rigid morphological drift.

7.3.3 Generalization to Higher Degrees of Freedom

The 4-DOF serial manipulator used throughout this thesis is a simplified testbed. Scaling to high-DOF systems (e.g., humanoid robots with 30+ joints or 7-DOF arms [12]) introduces curse-of-dimensionality challenges. The 1024-dimensional latent embedding may be insufficient for 30-dimensional joint spaces; covering high-dimensional workspaces requires exponentially more training samples; and self-occlusion becomes severe, requiring multi-camera fusion. Potential architectural extensions include hierarchical encoders (separate networks for limbs, torso, head), attention mechanisms to handle variable-length kinematic chains, and graph neural networks that exploit the robot's kinematic tree structure [49]. Dynamics-grounded priors [94] offer a promising direction for sample-efficient scaling by embedding physical structure directly into the learning architecture. These extensions are beyond the scope of this thesis but represent important future work.

7.3.4 Catastrophic Forgetting Under Structural Damage

While NeuroKin-3D achieved 2.39 mm mean error under nominal conditions and recovered from encoder slip via visual feedback, structural damage (bent link, seized joint) required fine-tuning on 100 post-damage samples (5 seconds) to restore accuracy. Preliminary encounters with naive fine-tuning revealed catastrophic forgetting: updating the network on damaged configurations degraded predictions for healthy configurations. This is the stability-plasticity dilemma discussed in Section 2.2.3.

Mitigation strategies include Elastic Weight Consolidation (EWC) [84], sparse subnetworks identified via the Lottery Ticket Hypothesis [85], or mixture-of-experts architectures that isolate damage-specific modules [86]. Each approach has trade-offs: EWC is computationally expensive for real-time adaptation, sparse subnetworks require careful initialization, and mixture-of-experts increases parameter count. For the 33.5-second training time of NeuroKin, a simpler strategy—periodic full retraining on a rolling window of recent observations—may

be practical for many applications.

7.3.5 Data Efficiency and Limited Training Samples

All comparative experiments trained on 2,000 samples—one-sixth the scale of the original FFKSM paper's 12,000 samples. While this enabled fair controlled comparison, it raises questions about generalization to larger datasets. Our data efficiency analysis suggests NeuroKin benefits more from limited data than volumetric rendering: it achieved 21.88 dB with 2,000 samples, approaching FFKSM's reported 23+ dB quality despite 6× less data. Extrapolating to 12,000 samples, we hypothesize NeuroKin would achieve 25–26 dB, exceeding FFKSM even at full scale. However, this extrapolation remains untested and should be validated in future work.

7.4 Summary

This chapter critically evaluated the trade-offs and validity of the contributions presented in Chapters 4–6. The key insights are that direct decoding trades novel-view synthesis for control-loop latency, and for applications requiring real-time response from a fixed viewpoint, this trade-off is highly favorable. Supervision efficiency explains the speed-quality improvement, as dense gradients from direct pixel prediction provide far more supervision per network evaluation than volumetric ray-marching. Temporal stigmergy validates the computational prerequisite for swarm coordination, while sim-to-real transfer remains the primary threat to validity. The following chapter synthesizes these findings into concrete conclusions and maps a technical roadmap for transitioning from sequential validation to physical swarm deployment.

Chapter 8

Conclusion and Future Work

This chapter synthesizes the principal findings of this thesis, directly answering the four research questions posed in Chapter 1, and outlines a concrete technical roadmap for transitioning the validated sequential capabilities into physically embodied, spatially concurrent swarm deployments.

8.1 Summary of Contributions and Research Findings

This thesis addressed the central problem that explicit 3D volumetric reconstruction, while enabling high-fidelity visual self-modeling, imposes a latency barrier that fundamentally precludes real-time control integration. The core hypothesis—that sub-millisecond visual self-modeling can be achieved by bypassing explicit 3D reconstruction in favor of direct sensorimotor decoding, and that this capability enables decentralized swarm coordination via temporal stigmergy—has been rigorously validated through controlled simulation experiments. The findings are structured around the four research questions introduced in Chapter 1.

8.1.1 RQ1: Architectural Efficiency — Direct Decoding Defeats the Latency Paradox

Answer: Direct sensorimotor decoding reduces latency by over three orders of magnitude while simultaneously improving reconstruction quality. The empirical evidence from Chapters 3 and 4 demonstrates that NeuroKin achieves 7,400 FPS (0.135 ms latency) and 21.88 dB PSNR—a 1,418× speedup over the implicit NeRF-based FFKSM (5.22 FPS, 17.35 dB PSNR) and a 200× speedup over the explicit 3DGS-based K-3DGS (37 FPS, 17.01 dB PSNR).

The resolution of the apparent speed-quality paradox lies in supervision efficiency. NeuroKin's single forward pass provides dense gradient information across 10,000 pixels simultaneously, whereas FFKSM's ray-marching provides each network evaluation with only 0.0156 pixels of supervision on average. This $640,000\times$ difference in supervision density explains why NeuroKin trains 89× faster (33.5 seconds vs. 50 minutes) and generalizes more effectively from limited data (2,000 samples vs. the 12,000 samples required by the original

FFKSM paper [11]).

This finding directly challenges the conventional wisdom that 3D reconstruction is a necessary prerequisite for visual self-modeling. For control-oriented tasks requiring only silhouette-level awareness—collision checking, proprioceptive drift correction, and damage detection—the representational overhead of volumetric methods is computational waste. The principle of minimal representation emerges as a design guideline: neural self-models should maintain only the complexity required by downstream tasks.

8.1.2 RQ2: Control Integration — Sub-Millisecond Visual Servoing Enables Dynamic Stabilization

Answer: Yes. Chapter 5 demonstrated that integrating NeuroKin-3D's stereoscopic spatial estimates (2.39 mm mean error, 0.400 ms latency) into a resolved-rate feedback loop enables real-time closed-loop control. Under nominal conditions, the visual controller converged 15% faster than the proprioceptive baseline (35.9 vs. 42.2 steps, $p = 0.003$) with 18% lower final error (2.3 vs. 2.8 mm, $p = 0.017$). This improvement stems from two factors: direct Cartesian measurement, which bypasses the accumulation of forward kinematic errors, and exponential smoothing ($\alpha = 0.6$), which attenuates high-frequency noise.

Critically, when a severe encoder slip fault was injected at step 50 (simulating calibration drift or thermal deformation), the proprioceptive baseline collapsed to 13% success, while the NeuroKin visual controller maintained 88% success. The recovery was rapid: the controller re-converged to sub-centimeter accuracy within 38.5 steps on average (0.77 seconds). The 12% failure rate was dominated by extreme fault magnitudes ($> 25°$) causing joint limit saturation and workspace exit conditions, not by visual estimation failure.

A Lyapunov stability analysis confirmed that the proportional-only visual servo is exponentially stable. The estimation error $\boldsymbol{\delta}$ (bounded by 2.39 mm) enters as an additive disturbance, and the system converges to a residual error ball whose radius is proportional to $\|\boldsymbol{\delta}\|/K_p$. With $K_p = 0.25$, the predicted steady-state error of approximately 9.6 mm matches the observed convergence tolerance, validating the theoretical analysis.

8.1.3 RQ3: Fault Tolerance — Rapid Fine-Tuning Enables Adaptation to Structural Damage

Answer: The fault injection experiments in Chapter 5 demonstrated that visual estimates enable closed-loop recovery from proprioceptive corruption, but structural damage (bent link, seized joint) initially degrades estimation accuracy because the observed silhouette no longer matches the training distribution. For a bent link scenario (joint 2 physical zero shifted by $+10°$), the network's prediction error increased from 2.4 mm to 12.7 mm mean error. For a seized joint scenario (joint 3 locked), the error diverged to 18.4 mm.

However, NeuroKin-3D's efficient training (33.5 seconds from scratch) enables rapid online adaptation. Fine-tuning on just 100 post-damage samples (5 seconds, 10 epochs

with learning rate 10^{-4}) restored estimation accuracy to 3.1 mm for the bent link scenario and 4.2 mm for the seized joint scenario. This rapid adaptation capability is enabled by NeuroKin's dense supervision efficiency, which allows effective learning from limited data.

The closed-loop performance after fine-tuning was robust: 92% success for the bent link scenario and 67% for the seized joint scenario, with the latter limited by the reduced reachable workspace (the locked joint fundamentally constrains feasible targets) rather than estimation accuracy. Unlike approaches requiring extensive offline counterfactual synthesis (e.g., Dream2Fix with 120,000+ simulation cases [78]), NeuroKin's 5-second fine-tuning cycle suggests a practical pathway for real-world deployment where collecting extensive post-damage data may be infeasible or dangerous.

8.1.4 RQ4: Collective Adaptation — Temporal Stigmergy Validates the Prerequisite for Swarm Coordination

Answer: Yes. Chapter 6 demonstrated that a 100-stage sequential swarm, where agents read and write a shared `factory_state` ledger tracking consecutive failures, achieves 88% post-fault success compared to the baseline's 13%. The temporal stigmergy mechanism—agents coordinate across time by leaving environmental traces (failure counters) that trigger adaptive responses in subsequent agents—requires no centralized coordination, no explicit inter-agent communication, and no additional training beyond the individual NeuroKin self-model.

The threshold rule ($\tau = 2$ consecutive failures triggers recovery mode) is simple yet effective. A single failure ($\tau = 1$) would trigger recovery mode too aggressively, causing unnecessary conservative behavior after isolated, unrepeated faults. A higher threshold ($\tau = 3$ or $\tau = 4$) would delay response, allowing cascading failures to propagate. The empirically calibrated $\tau = 2$ balances these competing pressures.

The information efficiency is striking: with at most $\log_2(3) \approx 1.58$ bits of shared information (the 3-state ledger), the NeuroKin swarm achieves a 75 percentage point improvement over the baseline. The median steps to convergence for NeuroKin agents was 34.5, compared to 48.5 for the baseline's successful runs (a 29% reduction), demonstrating that the recovery mode not only improves success rates but also operational efficiency.

Crucially, the 0.400 ms inference time of NeuroKin-3D is essential for this mechanism. It enables real-time failure detection within a single control cycle, immediate ledger updates before the next agent is instantiated, and rapid adaptation without stale state windows. Without sub-millisecond inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

8.2 Technical Contributions Summary

Beyond answering the research questions, this thesis makes four specific technical contributions to the fields of robotic self-modeling, neural representation, and decentralized multi-agent systems:

1. **Formal Benchmarking of the Latency Paradox**: Through rigorous ground-up reproduction and quantitative evaluation of both implicit (FFKSM/NeRF) and explicit (K-3DGS) neural rendering self-models on a standardized 2,000-sample dataset, this research formally exposes their control incompatibility. The exact latency metrics—191.6 ms for FFKSM, 27.0 ms for K-3DGS—establish the mathematical limits that necessitate a paradigm shift. The reproduction also uncovered three undocumented implementation challenges (black-screen convergence, kinematic blindness, and weighted loss tuning) that are essential for successful training but omitted from the original publication [11].

2. **The NeuroKin Direct Sensorimotor Decoder**: NeuroKin is a lightweight, fully convolutional direct sensorimotor decoder that completely bypasses volumetric reconstruction. Through extensive empirical validation, NeuroKin achieves 7,400 FPS (0.135 ms latency) and 21.88 dB PSNR—a 1,418× speedup over implicit NeRF baselines with a +4.53 dB quality improvement. The stereoscopic extension, NeuroKin-3D, recovers 3D spatial coordinates with 2.39 mm mean error at 2,500 FPS (0.400 ms latency). The multi-task variant, ResNeuroKin-D, adds depth prediction (21.23 dB depth map validation PSNR at 2,500 FPS) and induces geometric structure in the latent representation. All architectures are open-sourced for reproducibility.

3. **Visual Closed-Loop Damage Adaptation**: This research successfully synthesizes NeuroKin's sub-millisecond spatial estimates with a Jacobian-based inverse-kinematics feedback loop. The integration enables dynamic error recovery after severe fault injection (88% success vs. 13% baseline). Lyapunov stability analysis confirms exponential stability, and the empirical demonstration that high-speed visual estimates can override corrupted proprioceptive data proves that sub-millisecond self-modeling enables real-time closed-loop adaptation.

4. **Temporal Stigmergy for Sequential Swarm Coordination**: The validation of stigmergic swarm coordination under compounding mechanical faults demonstrates that ultra-fast individual self-diagnosis is the computational prerequisite for swarm intelligence. The 100-stage sequential swarm with progressive fault injection achieves 88% post-fault success through a decentralized ledger with only 1.58 bits of shared information. The mechanism requires no centralized coordination, no explicit communication, and no additional training beyond the individual self-model.

Appendix: Diagnostic Logs and Architecture Overviews

To ensure complete empirical documentation, this section provides additional structural, workflow, and diagnostic graphs.

Figure 8.1: End-to-end system architecture: pipeline from data generation (Lorenz trajectories, silhouette capture) through NeuroKin training, integration into closed-loop control, fault injection validation, and 100-stage sequential swarm coordination with temporal stigmergy.

8.3 Future Roadmap: From Sequential Validation to Spatial Swarms

The future roadmap is strictly bounded by hardware realities. Because physical deployment was impossible given the temporal constraints of this Master's degree, the next step is rigorous hardware validation under realistic sensing and calibration conditions. This foundational bridging of the sim-to-real gap will serve as the immediate launching point for the author's forthcoming doctoral research. Following that, concurrent multi-robot experiments that add collision avoidance and atomic shared-state updates can be effectively prototyped. Those extensions are consistent with the results in this thesis, but they are not claimed as solved here.

8.4 Concluding Remarks

This thesis demonstrates that fast visual self-modeling is useful in practice: it supports real-time closed-loop correction when proprioception is corrupted and improves a sequential multi-agent pipeline through a minimal shared failure ledger. The results do not claim full spatial swarm deployment, but they do establish the computational foundation needed for it.

The central design lesson is to keep the representation as small as the task allows. For the control and coordination problems studied here, direct decoding is sufficient, volumetric reconstruction is too slow, and a minimal ledger is enough to support temporal stigmergy. Those are the concrete conclusions supported by the experiments in this thesis.

Bibliography

[1] J. Bongard, V. Zykov, and H. Lipson, "Resilient machines through continuous self-modeling," *Science*, vol. 314, no. 5802, pp. 1118–1121, 2006.

[2] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.

[3] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," in *Proc. Genet. Evol. Comput. Conf. (GECCO)*, 2015.

[4] R. Kwiatkowski and H. Lipson, "Task-agnostic self-modeling machines," *Science Robotics*, vol. 4, no. 26, p. eaau9354, 2019.

[5] R. Kwiatkowski and H. Lipson, "Zero-shot learning on simulated robots," in *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 1–7, 2019.

[6] Y. Hu, Y. Wang, R. Liu, Z. Shen, and H. Lipson, "Reconfigurable robot identification from motion data," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[7] A. Dogra, S. Mahan, S. S. Padhee, and E. Singla, "Unified modeling of unconventional modular and reconfigurable manipulation system," *Robotics and Computer-Integrated Manufacturing*, vol. 77, p. 102385, 2022.

[8] S. Farghdani, M. Patel, and R. Chhabra, "Robust embodied self-identification of morphology in damaged multi-legged robots," *IEEE Robot. Autom. Lett.*, vol. 10, pp. 1–8, 2025.

[9] C. Yu and S. Kriegman, "Robots that redesign themselves through kinematic self-destruction," arXiv preprint arXiv:2603.12505, 2026.

[10] B. Chen, R. Kwiatkowski, C. Vondrick, and H. Lipson, "Full-body visual self-modeling of robot morphologies," *Science Robotics*, vol. 7, no. 68, p. eabn1944, 2022.

[11] Y. Hu, J. Lin, and H. Lipson, "Teaching robots to build simulations of themselves," *Nature Machine Intelligence*, vol. 6, pp. 123–130, 2024.

[12] L. Schulze and H. Lipson, "High-degrees-of-freedom dynamic neural fields for robot self-modeling and motion planning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[13] Y. Hu, B. Chen, and H. Lipson, "Egocentric visual self-modeling for autonomous robot dynamics prediction and adaptation," arXiv preprint arXiv:2207.03386, 2022.

[14] S. Rezvani, A. J. Mahmood, and R. Chhabra, "Robust visual embodiment: How robots discover their bodies in real environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[15] K. Hu, P. Yu, and N. Tan, "Learning high-fidelity robot self-model with articulated 3D Gaussian splatting," *Int. J. Robotics Research*, 2025.

[16] P. Yu, X. Wang, and N. Tan, "Shape-interpretable visual self-modeling enables geometry-aware continuum robot control," arXiv preprint arXiv:2603.01751, 2026.

[17] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, "Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 5855–5864.

[18] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and A. Y. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020.

[19] A. Pumarola, E. Corona, G. Pons-Moll, and A. Sanfeliu, "D-NeRF: Neural radiance fields for dynamic scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021.

[20] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 102:1–102:15, 2022.

[21] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Zip-NeRF: Anti-aliased grid-based neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.

[22] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, "Nerfies: Deformable neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021.

[23] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139:1–139:14, 2023.

[24] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18039–18048.

[25] N. Keetha *et al.*, "SplaTAM: Splat, track and map 3D Gaussians for dense RGB-D SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18410–18420.

[26] S. Li, B. Keller, Y. Lin, and B. Khailany, "GauRast: Enhancing GPU triangle rasterizers to accelerate 3D Gaussian splatting," in *Proc. Int. Symp. Comput. Archit. (ISCA)*, 2025.

[27] A. Guedon and V. Lepetit, "SuGaR: Surface-aligned Gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[28] Y. Weng *et al.*, "Neural implicit representation for building digital twins of unknown articulated objects," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[29] Z. Li *et al.*, "ART: Articulated reconstruction transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

[30] Y. Wu *et al.*, "AOMGen: Photoreal, physics-consistent demonstration generation for articulated object manipulation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[31] Z. Wu *et al.*, "URDF-Anything+: Autoregressive articulated 3D models generation for physical simulation," arXiv preprint arXiv:2502.09850, 2025.

[32] W. Xu, L. Liu, L. Zhang, D. Guo, and R. Liu, "MotionAnymesh: Physics-grounded articulation for simulation-ready digital twins," arXiv preprint arXiv:2603.12936, 2026.

[33] J. Wang, D. Wang, J. Hu, Q. Zhang, J. Yu, and L. Xu, "Kinematify: Open-vocabulary synthesis of high-DoF articulated objects," arXiv preprint arXiv:2511.01294, 2025.

[34] C. Zhang, M. Qin, Y. Wang, B. Xie, H. Li, and Z. Wang, "SIMART: Decomposing monolithic meshes into sim-ready articulated assets via MLLM," arXiv preprint arXiv:2603.23386, 2026.

[35] P. Wang, G. Ke, M. Rong, X. Guo, K. Ming, and S. Liu, "ArtLLM: Generating articulated assets via 3D LLM," arXiv preprint arXiv:2603.01142, 2026.

[36] J. Guo, Y. Xin, G. Liu, K. Xu, L. Liu, and R. Hu, "ArticulatedGS: Self-supervised digital twin modeling of articulated objects using 3D Gaussian splatting," arXiv preprint arXiv:2503.08135, 2025.

[37] Q. Yu, Z. Xia, C. Li, D. Pan, X. Wang, and J. Liu, "ArtGS: 3D Gaussian splatting for interactive visual-physical modeling and manipulation of articulated objects," arXiv preprint arXiv:2507.02600, 2025.

[38] D. Wu, Z. Wang, Y. Luo, H. Zhang, T. Chen, and K. Guo, "REArtGS++: Generalizable articulation reconstruction with temporal geometry constraint via planar Gaussian splatting," arXiv preprint arXiv:2511.17059, 2025.

[39] H. Dai, H. Fan, H. Zhang, D. Wu, J. Zhang, and H. Dong, "FreeArtGS: Articulated Gaussian splatting under free-moving scenario," arXiv preprint arXiv:2603.22102, 2026.

[40] Q. Chen *et al.*, "FreeGaussian: Annotation-free control of articulated objects via 3D Gaussian splats with flow derivatives," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026.

[41] J. Tan *et al.*, "Sim-to-Real: Learning agile locomotion for quadruped robots," in *Proc. Robot.: Sci. Syst. (RSS)*, 2018.

[42] H. Lou *et al.*, "Robo-GS: A physics consistent spatial-temporal model for robotic arm with hybrid representation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.

[43] H. Lou *et al.*, "D-REX: Differentiable real-to-sim-to-real engine for learning majestic tracking handles," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[44] Z. Sun, L. Bao, T. Peng, J. Sun, and C. Zhou, "A high-fidelity digital twin for robotic manipulation based on 3D Gaussian splatting," arXiv preprint arXiv:2601.03200, 2026.

[45] G. Lu, S. Zhang, Z. Wang, C. Liu, J. Lu, and Y. Tang, "ManiGaussian: Dynamic Gaussian splatting for multi-task robotic manipulation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 349–366.

[46] S. Pan, Y. Xu, R. Xu, Z. Zhou, S. Wu, and Z. Yu, "Self-correcting robot manipulation via Gaussian-splatted foresight," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2025.

[47] S. Yang, K. Zhan, Y. Zhang, L. Lin, X. Huang, and M. Li, "Novel demonstration generation with Gaussian splatting enables robust one-shot manipulation," arXiv preprint arXiv:2504.13175, 2025.

[48] Y. Wu, S. Bae, Y. Chen, L. Lyu, S. Li, Z. Huang, and X. Yang, "DexGrasp-Zero: A morphology-aligned policy for zero-shot cross-embodiment dexterous grasping," arXiv preprint arXiv:2603.16806, 2025.

[49] A. Patel and S. Song, "GET-Zero: Graph embodiment transformer for zero-shot embodiment generalization," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[50] Q. Liang *et al.*, "Whole-body coordination for dynamic object grasping with legged manipulators," arXiv preprint arXiv:2508.08328, 2025.

[51] Y. Lu *et al.*, "GSMem: 3D Gaussian splatting as persistent spatial memory for zero-shot embodied exploration and reasoning," arXiv preprint arXiv:2603.19137, 2025.

[52] G. Jiang *et al.*, "GSWorld: Closed-loop photo-realistic simulation suite for robotic manipulation," arXiv preprint arXiv:2510.20813, 2025.

[53] A. Escontrela *et al.*, "GaussGym: An open-source real-to-sim framework for learning locomotion from pixels," arXiv preprint arXiv:2510.15352, 2025.

[54] E. Aljalbout, A. Frick, X. Chen, T. Westenbroek, and D. Fridovich-Keil, "The reality gap in robotics: Challenges, solutions, and best practices," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 9, 2026.

[55] H. Zhao *et al.*, "AGILE: A comprehensive workflow for humanoid loco-manipulation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[56] J. Abou-Chakra, Y. Zhu, L. Zhang, D. Fridovich-Keil, and P. Agrawal, "Real-is-sim: Bridging the sim-to-real gap with a dynamic digital twin," arXiv preprint arXiv:2504.03597, 2025.

[57] C. Zhang, Y. Zou, Z. Wu, Y. Ling, Y. Yang, and Z. Wang, "UniPR: Unified object-level real-to-sim perception and reconstruction from a single stereo pair," arXiv preprint arXiv:2603.19616, 2026.

[58] Z. Wu *et al.*, "Perceptive humanoid parkour: Chaining dynamic human skills via motion matching," arXiv preprint arXiv:2602.15827, 2026.

[59] M. N. Qureshi, Y. Tang, Z. Zhang, X. Li, Y. Wang, and A. Gupta, "SplatSim: Zero-shot sim2real transfer of RGB manipulation policies using Gaussian splatting," arXiv preprint arXiv:2404.13099, 2025.

[60] J. Tang, Z. He, K. Zhang, Y. Liu, M. Chen, and W. Xu, "Bridging simulation and reality: Cross-domain transfer with semantic 2D Gaussian splatting," arXiv preprint arXiv:2512.04731, 2025.

[61] D. Li *et al.*, "HAIC: Humanoid agile object interaction control via dynamics-aware world model," arXiv preprint arXiv:2602.11758, 2026.

[62] J. Yu, H. Su, D. Fridovich-Keil, and A. Gupta, "Real2Render2Real: Scaling robot data without dynamics simulation or robot hardware," arXiv preprint arXiv:2505.09601, 2025.

[63] J. Ren *et al.*, "Cybo-Waiter: A physical agentic framework for humanoid whole-body locomotion-manipulation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.

[64] Z. J. Xu *et al.*, "A robust antenna provides tactile feedback in a multi-legged robot," arXiv preprint arXiv:2603.07795, 2026.

[65] A. Jain, R. Sharma, S. Kumar, S. Kolathaya, Z. Zhang, and D. Song, "PolaRiS: Scalable real-to-sim evaluations for generalist robot policies," arXiv preprint arXiv:2512.16881, 2025.

[66] X. Li *et al.*, "Evaluating real-world robot manipulation policies in simulation," in *Proc. Conf. Robot. Learn. (CoRL)*, 2024.

[67] E. Daneshmand, S. Omar, G. Berseth, M. Khadiv, and H.-C. Lin, "SLowRL: Safe low-rank adaptation reinforcement learning for locomotion," arXiv preprint arXiv:2603.17092, 2026.

[68] L. Brunke *et al.*, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 5, pp. 1–43, 2022.

[69] F. Domberg and G. Schildbach, "Self-adapting robotic agents through online continual reinforcement learning with world model feedback," arXiv preprint arXiv:2603.04029, 2026.

[70] K. Qiu, K. Walker, M. Y. Michelis, M. Cygan, and J. Hughes, "Swim2Real: VLM-guided system identification for sim-to-real transfer," arXiv preprint arXiv:2603.20827, 2026.

[71] N. Grambow *et al.*, "Anomaly detection for generic failure monitoring in robotic assembly, screwing and manipulation," *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 3664–3671, 2026.

[72] Y. Fu, X. Chen, B. Zhang, M. Yang, and K. Zhang, "Contrastive forward prediction reinforcement learning for adaptive fault-tolerant legged robots," in *Proc. Conf. Robot. Learn. (CoRL)*, 2025.

[73] H. Chen, Y. Yao, R. Liu, C. Liu, and J. Ichnowski, "Automating robot failure recovery using vision-language models with optimized prompts," arXiv preprint arXiv:2409.03966, 2024.

[74] N. Jayasinghe *et al.*, "Residual control for fast recovery from dynamics shifts," arXiv preprint arXiv:2603.07775, 2026.

[75] G. G. Briscoe-Martinez, Y. Gautam, R. Shetty, A. Pasricha, M. M. Nicotra, and A. Roncone, "Moving on, even when you're broken: Fail-active trajectory generation via diffusion policies conditioned on embodiment and task," arXiv preprint arXiv:2602.02895, 2026.

[76] N. Poddar, S. McCrory, L. Penco, G. Clark, H. E. Sevil, and R. Griffin, "Embedding classical balance control principles in reinforcement learning for humanoid recovery," arXiv preprint arXiv:2603.08619, 2026.

[77] H. Li *et al.*, "PCHC: Enabling preference-conditioned humanoid control via multi-objective reinforcement learning," arXiv preprint arXiv:2603.24047, 2026.

[78] D. Li *et al.*, "Learning actionable manipulation recovery via counterfactual failure synthesis," arXiv preprint arXiv:2603.13528, 2026.

[79] H. Lee, W.-J. Baek, J. Cha, and J. Park, "TOLEBI: Learning fault-tolerant bipedal locomotion via online status estimation and fallibility rewards," arXiv preprint arXiv:2602.05596, 2026.

[80] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," *Psychol. Learn. Motiv.*, vol. 24, pp. 109–165, 1989.

[81] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends Cogn. Sci.*, vol. 3, no. 4, pp. 128–135, 1999.

[82] S. Grossberg, "Competitive learning: From interactive activation to adaptive resonance," *Cognitive Science*, vol. 11, no. 1, pp. 23–63, 1987.

[83] M. Mermillod, A. Bugaiska, and P. Bonin, "The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects," *Front. Psychol.*, vol. 4, p. 504, 2013.

[84] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.

[85] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[86] D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta, "Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science," *Nat. Commun.*, vol. 9, p. 2383, 2018.

[87] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[88] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017.

[89] A. T. Tran and J. Kosecka, "VarSplat: Uncertainty-aware 3D Gaussian splatting for robust RGB-D SLAM," arXiv preprint arXiv:2507.08762, 2025.

[90] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London: Springer, 2009.

[91] E. N. Lorenz, "Deterministic nonperiodic flow," *Journal of the Atmospheric Sciences*, vol. 20, no. 2, pp. 130–141, 1963.

[92] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, and A. Courville, "On the spectral bias of neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019.

[93] G. Chechik, I. Meilijson, and E. Ruppin, "Synaptic pruning in development: A computational account," *Neural Computation*, vol. 10, no. 7, pp. 1759–1777, 1998.

[94] Q. Peng, Y. Lin, Y. Xue, J. Pang, and W. Zhang, "Embodiment-aware generalist-specialist distillation for unified humanoid whole-body control," arXiv preprint arXiv:2602.02960, 2026.

[95] R. Wolf, Y. Shi, S. Liu, and R. Rayyes, "Diffusion Models for Robotic Manipulation: A Survey," *arXiv*, vol. 2504, p. 08438, 2025.